Electrical Engineering
Computer Engineering and Information Technology Department

Smart BUS RAPID TRANSIT

A Thesis Submitted in Partial Fulfillment of the Requirements for the award of the degree of

Bachelor of Computer Engineering and Information Technology Department

Graduation Project

Academic Year 2025 – 2026

Submitted By

Name	ID
Abdelrahman Mahmoud Ali Ahmed	4220058
Abdelrehem Mohamed Abdelrehem	4220061
Anasimon Semsem Mufid	4220300
Mohamed El-Kahlawy Mahmoud	4220351
Mohamed Hatem Abdelkhalek	4220067
Ahmed Alaa El-Din Abdelazez	4200049

Supervised By

Dr. Manar Mohamed Fattouh Donia

2025 / 2026

Abstract

This project presents an integrated, AI-powered Smart Bus Rapid Transit (BRT) system designed to modernize public transportation in Egypt. Rapid urbanization and expanding commuter demands have introduced challenges in congestion management, fleet synchronization, and onboard security. To address these issues, the project develops a comprehensive ecosystem that combines digital service layers with a real-time video analytics engine.

Methods. The service layer comprises an administrative web platform providing route mapping, dynamic ticketing, and station telemetry, paired with a cross-platform Flutter mobile application for real-time bus tracking and digital ticketing. The core technical contribution lies in the deployment of deep learning and computer vision pipelines optimized via the YOLOv8 architecture for high-speed edge inference.

Eight intelligent vision subsystems are implemented: (1) Station Congestion Detection for crowd density profiling and fleet scheduling, (2) Bidirectional Passenger Counting for real-time cabin occupancy, (3) Weapon Detection for isolating concealed or drawn threats, (4) Violence and Abnormal Behavior Analysis for flagging physical altercations, and (5) Driver Behavior Assessment for monitoring operator fatigue and distraction. Additionally, a fire and smoke detection subsystem, an automated bus-lane violation and license-plate recognition module, and a biometric facial-recognition access-control gate are integrated. The violence-and-anomaly subsystem is realised by a hybrid anomaly-detection model that pairs a reconstruction-based EfficientNet-B0/U-Net autoencoder with a ResNet3D-18 spatiotemporal classifier.

Results. Experimental evaluation under variable operational and lighting conditions demonstrates robust performance across precision, recall, and inference throughput metrics. The weapon detection model achieves a mean Average Precision (mAP@0.5) of 91.2%, while the passenger counting system exceeds 95% directional accuracy. The biometric authentication system achieves sub-millisecond query latency using ArcFace embeddings matched via FAISS indexing.

The unified framework provides a scalable, production-grade blueprint for integrating Edge AI into modern public transport infrastructure, accelerating vehicle turnaround

times, minimizing operational latency, and improving the passenger experience within future smart city developments.

Acknowledgement

The authors express sincere gratitude to Allah, whose grace enabled the completion of this work. Deep thanks are extended to our families for their unwavering support throughout this journey.

Our supervisor, Dr. Manar Mohamed Fattouh Donia, deserves special recognition for her guidance, patience, and invaluable insights that shaped the direction and quality of this project. We also thank Eng. Mahmoud Al-Emirati, Eng. Rawda Tariq, and Eng. Hadeer Hossam for their consistent technical assistance.

Table of Contents

Chapter 1: Introduction *1*
1.1 Overview of Smart Bus Rapid Transit 1
1.2 Problem Statement and Motivation 3
1.3 Project Objectives 6
1.4 Project Scope 7
1.5 Hardware and Software Tools 9
1.6 Thesis Organization 10
Chapter 2: Background *10*
2.1 Convolutional Neural Networks and Computer Vision 10
2.2 YOLOv8 Architecture 11
2.3 Object Tracking: BoT-SORT and Kalman Filter 13
2.4 ArcFace and Biometric Recognition 14
2.5 FAISS: Fast Approximate Nearest Neighbor Search 16
2.6 Anomaly Detection in Video Surveillance 16
2.7 Software Engineering Foundations 17
Chapter 3: Literature Review *17*
3.1 Bus Rapid Transit Systems 17
3.2 AI and Computer Vision in Public Transportation 18
3.3 Passenger Counting Technologies 19
3.4 Weapon and Threat Detection 19
3.5 Biometric Access Control 20
3.6 Summary of Related Work 21
3.7 Research Gaps 22
Chapter 4: Proposed Work *23*
4.1 System Overview 23
4.2 Module I: Station Congestion Detection 27
4.4 Module III: Violence and Abnormal Behavior Detection 30
4.5 Module IV: Weapon Detection System 34
4.6 Module V: Accident Detection and Prediction System 35
4.7 Module VI: Fire Detection System 37
4.9 Module VIII: Bus Lane Monitoring and Violation Detection System 41
4.10 System Use Case Analysis 43
4.11 Operational Use Case Analysis (AI & Automation) 60
4.12 Entity Relationship Diagram (ERD) 62
4.13 Master Entity Relationship Summary 67

4.14 Computer Vision Pipeline Architecture **69**
4.15 BRT Mobile Application **74**
4.16 Web Application **84**
Chapter 5: Results and Discussion ***93***
5.1 Chapter Introduction **93**
5.2 Testing Methodology and Evaluation Metrics **96**
5.3 Experimental Results **98**
5.4 Per-Module Detailed Results **100**
5.5 Comparative Benchmarking Against Literature **114**
5.6 Digital Application Evaluation **115**
Chapter 6: Critical Discussion, System Limitations, and Future Roadmap ***136***
6.1 Discussion of Strengths and Weaknesses **136**
6.2 System Limitations **139**
6.3 Future Work Suggestions **140**
Chapter 7: Final Conclusion ***144***
7.1 Project Summary **144**
7.2 Key Findings and Empirical Milestones **145**
7.3 Limitations and Critical Assessment **146**
7.4 Broader Impact and Core Engineering Contributions **147**
References ***148***

Table of Figures

FIGURE 1: THE KEY COMPONENTS OF A STANDARD CNN ARCHITECTURE 11
FIGURE 2: YOLOV8 ARCHITECTURE 12
FIGURE 3: BOT-SORT AND KALMAN FILTER TRACKING PIPELINE 13
FIGURE 4: BASIC SYSTEM ELEMENTS OF THE SMART BRT PLATFORM AND THE DATA FLOW BETWEEN THEM 23
FIGURE 5: THE STRUCTURAL FLOW OF THE PROPOSED ECOSYSTEM OPERATES ACROSS A SEQUENTIAL LOGICAL PROGRESSION 24
FIGURE 6: SOFTWARE DEVELOPMENT LIFE CYCLE (ITERATIVE / AGILE) ADOPTED FOR THE SMART BRT PLATFORM 25
FIGURE 7: OPERATIONAL PIPELINE AND DECISION FLOW FOR MODULE I (STATION CONGESTION DETECTION.) 27
FIGURE 8: DIAGRAM OF THE BUS PASSENGER COUNTING SYSTEM (ENTRY EVENT FLOW) 28
FIGURE 9: DECISION LOGIC AND COMPUTATIONAL PIPELINE FOR MODULE II (REAL-TIME PASSENGER COUNTING SYSTEM) 30
FIGURE 10: DECISION LOGIC AND COMPUTATIONAL PIPELINE FOR MODULE III (VIOLENCE AND ABNORMAL BEHAVIOR DETECTION SYSTEM) 31
FIGURE 11: MODULE III HYBRID ANOMALY-DETECTION MODEL — TWO-BRANCH ARCHITECTURE (EFFICIENTNET-B0/U-NET RECONSTRUCTION AUTOENCODER AND RESNET3D-18 CLASSIFIER), PREPROCESSING, AND THE INFERENCE CASCADE 33
FIGURE 12: DEEP LEARNING ARCHITECTURE AND POST-PROCESSING PIPELINE FOR MODULE IV (WEAPON DETECTION SYSTEM) 34
FIGURE 13: KINEMATIC HAZARD EVALUATION AND PREDICTIVE DECISION FLOW FOR MODULE V (ACCIDENT PREDICTION SYSTEM) 36
FIGURE 14: DETERMINISTIC VERIFICATION PIPELINE AND SPATIAL-TEMPORAL FILTERING LOGIC FOR MODULE VI 37
FIGURE 15: MODULE VII (BIOMETRIC GATES) 38
FIGURE 16: MULTI-TIER HIERARCHICAL VERIFICATION AND PHYSICAL HARDWARE CONTROL PIPELINE FOR MODULE VII (BIOMETRIC ACCESS GATES) 39
FIGURE 17: BUS LANE MONITORING AND VIOLATION DETECTION SYSTEM 40
FIGURE 18: MULTI-STAGE ENFORCEMENT PIPELINE AND ALPHANUMERIC OCR LOGIC FOR MODULE VIII (BUS LANE MONITORING SYSTEM) 41
FIGURE 19: UNIFIED UML USE-CASE DIAGRAM FOR THE SMART BRT PLATFORM 46
FIGURE 20: TECHNICAL UML SEQUENCE DIAGRAM MAPPING THE LOW-LATENCY MESSAGING LIFECYCLE OF UC-11 (TRIGGER EMERGENCY SOS ALERT) 56
FIGURE 21: TECHNICAL UML SEQUENCE DIAGRAM MAPPING THE AUTOMATED THREAT DETECTION AND HUMAN-IN-THE-LOOP VALIDATION LIFECYCLE OF UC-16 (REVIEW AUTOMATED CV SECURITY ALERTS) 56
FIGURE 22: SMART BRT - AI AND AUTOMATION SERVICES 60
FIGURE 23: ENTITY RELATIONSHIP DIAGRAM (ERD) 62
FIGURE 24: ENTITY-RELATIONSHIP DIAGRAM — MODULE 1: IDENTITY & ACCESS 63
FIGURE 25: ENTITY-RELATIONSHIP DIAGRAM — MODULE 2: PAYMENTS & TICKETING 63
FIGURE 26: ENTITY-RELATIONSHIP DIAGRAM — MODULE 3: TRANSIT NETWORK & FLEET 63
FIGURE 27: ENTITY-RELATIONSHIP DIAGRAM — MODULE 4: DEMAND ANALYTICS & OPERATIONS 64
FIGURE 28: ENTITY-RELATIONSHIP DIAGRAM — MODULE 5: COMPUTER VISION & SAFETY 64
FIGURE 29: MULTI-STAGE COMPUTER-VISION INFERENCE PIPELINE 69
FIGURE 30: CV DATA PROCESSING SEQUENCE FLOWCHART 70
FIGURE 31: PHYSICAL DATABASE SCHEMA (MICROSOFT SQL SERVER 2022) 72
FIGURE 32: COMPUTER-VISION SECURITY SUBSYSTEM — ENTITY-RELATIONSHIP DIAGRAM 74
FIGURE 33: MOBILE APPLICATION SYSTEM CONTEXT DIAGRAM 76
FIGURE 34: SEQUENCE DIAGRAM: DIGITAL TICKETING FLOW 78
FIGURE 35: DOMAIN LAYER CORE ENTITIES CLASS DIAGRAM 79
FIGURE 36: LIVE OPENAPI (SWAGGER) DOCUMENTATION OF THE SMART BRT REST API ... 80
FIGURE 37: DATA LIFECYCLE FLOWCHART 81

FIGURE 38: WEB APPLICATION ARCHITECTURE DIAGRAM 85
FIGURE 39: REACT COMPONENT TREE DIAGRAM 86
FIGURE 40: SAMPLE VALIDATION PREDICTIONS FROM THE PERSON/OCCUPANCY DETECTION DATASET (MODULES I–II), WITH DETECTED INDIVIDUALS BOUNDED AND CONFIDENCE-SCORED 94
FIGURE 41: REPRESENTATIVE VALIDATION IMAGE FROM THE WEAPON-DETECTION DATASET (MODULE IV) — THE "PLIERS / TOOLS" THREAT CLASS 94
FIGURE 42: REPRESENTATIVE VALIDATION IMAGE FROM THE FIRE-DETECTION DATASET (MODULE VI) — A VEHICLE FIRE CAPTURED AT NIGHT 95
FIGURE 43: REPRESENTATIVE FRAME FROM THE BUS-LANE / LICENSE-PLATE DATASET (MODULE VIII), SHOWING A VEHICLE AND ITS READABLE PLATE WITHIN THE MONITORED LANE 96
FIGURE 44: THE STANDARD CONFUSION-MATRIX STRUCTURE (TRUE/FALSE POSITIVE AND NEGATIVE QUADRANTS) USED TO EVALUATE PER-MODULE CLASSIFICATION PERFORMANCE 98
FIGURE 45: COMPARATIVE F1-SCORE AND MEAN AVERAGE PRECISION (MAP) PROFILES ACROSS THE AI MODULES 100
FIGURE 46: REAL-TIME PASSENGER COUNTING 101
FIGURE 47: COMPUTER-VISION TEMPORAL RULE ENGINE 102
FIGURE 48: ANOMALY MODEL — RESNET3D-18 CLIP-CLASSIFICATION BRANCH ON A NORMAL SCENE (SEATED, READING PASSENGERS) 103
FIGURE 49: ANOMALY MODEL — RESNET3D-18 BRANCH CLASSIFYING THE CLIP AS ABNORMAL DURING A PHYSICAL ALTERCATION BETWEEN PASSENGERS 104
FIGURE 50: ANOMALY MODEL — AUTOENCODER BRANCH WITH PER-PASSENGER DEEPSORT TRACKING; EACH TRACKED IDENTITY IS SCORED NORMAL WITH A LOW RECONSTRUCTION ERROR 104
FIGURE 51: ANOMALY MODEL — AUTOENCODER BRANCH FLAGGING AN INDIVIDUAL AS ABNORMAL (RED BOX, ELEVATED RECONSTRUCTION ERROR) WHILE SURROUNDING PASSENGERS REMAIN NORMAL 105
FIGURE 52: REAL-TIME ADAS VISION SYSTEM FOR VEHICLE DISTANCE ESTIMATION AND COLLISION RISK DETECTION 108
FIGURE 53: LIVE FIRE-DETECTION TEST — THE SYSTEM RAISES A 'FIRE DETECTED' STATUS AND LOCALISES EACH ACTIVE FLAME WITH A 'FIRE SOURCE' BOUNDING BOX ON A BURNING VEHICLE 109
FIGURE 54: LIVE FIRE-DETECTION TEST DURING SUPPRESSION — THE FLAME REGION REMAINS CORRECTLY LOCALISED WHILE A FIREFIGHTER EXTINGUISHES THE VEHICLE FIRE 110
FIGURE 55: LIVE FACIAL-RECOGNITION TEST — A REGISTERED PASSENGER IS MATCHED AGAINST THE FAISS GALLERY AND ADMITTED, WITH THE RECOGNISED IDENTITY AND SIMILARITY SCORE DISPLAYED (GREEN BOX) 111
FIGURE 56: MULTI-SUBJECT RECOGNITION TEST — SUBSCRIBED PASSENGERS ARE IDENTIFIED IN GREEN WHILE AN UNREGISTERED INDIVIDUAL IS FLAGGED IN RED AS 'NOT SUBSCRIBED', ENFORCING GATE ACCESS CONTROL 112
FIGURE 57: ARCHITECTURAL OVERVIEW OF AN END-TO-END AUTOMATIC LICENSE PLATE RECOGNITION SYSTEM 114
FIGURE 58: COMPARATIVE PERFORMANCE EVALUATION OF THE PROPOSED SMART BRT SYSTEM VERSUS EXISTING LITERATURE 115
FIGURE 59: MOBILE APP — HOME DASHBOARD WITH TRIP PLANNING, NEARBY LINES AND THE NEAREST-STATION CARD 116
FIGURE 60: LIVE ARRIVALS EXPANDED VIEW — SHOWING REAL-TIME BUS ARRIVAL TIMES WITH SEAT AVAILABILITY STATUS (EMPTY/CROWDED) AND QUICK BOOK DESTINATIONS 117
FIGURE 61: MOBILE APP — CONFIRM TICKET SCREEN SHOWING FARE, DURATION, ORIGIN AND DESTINATION, CROWD LEVEL AND WALLET BALANCE BEFORE PAYMENT 119
FIGURE 62: MOBILE APP — MY TICKETS SCREEN WITH AN ACTIVE QR TICKET (ME7WAR EL MOSHEER TO EL TROLLEY, 39:52 REMAINING, 27 EGP) AND THE GATE-SCAN CODE 120
FIGURE 63: MOBILE APP — SEASON PASS (PASSES TAB) SHOWING THE ACTIVE WEEKLY SAVER PLAN (150 EGP, 14 RIDES OVER 7 DAYS, 12 OF 14 REMAINING) 121

FIGURE 64: MOBILE APP — WALLET SCREEN WITH THE AVAILABLE BALANCE, A SAVED VISA CARD, A LINKED VODAFONE-CASH WALLET AND A FAWRY CASH TOP-UP OPTION *122*
FIGURE 65: MOBILE APP — SOS CONFIRMATION SCREEN (ALERT SENT) WITH THE SECURITY TEAM NOTIFIED AND LIVE-LOCATION TRACKING ACTIVATED *123*
FIGURE 66: MOBILE APP — SUPPORT AND REPORTS SCREEN WITH THE EMERGENCY-SOS SHORTCUT AND INCIDENT CATEGORIES *124*
FIGURE 67: MOBILE APP — MY REPORTS SCREEN LISTING SUBMITTED REPORTS WITH STATUS BADGES (RESOLVED, PENDING) AND A NEW-REPORT ACTION *125*
FIGURE 68: WEB PORTAL — HOME AND ABOUT VIEWS *127*
FIGURE 69: BRT PUBLIC WEBSITE — LOCATION VIEW: THE INTERACTIVE MAP PROMPTS THE PASSENGER TO SHARE THEIR POSITION TO LOCATE THE NEAREST BRT STATION *129*
FIGURE 70: WEB PORTAL — LOCATION VIEW GEOSPATIAL WAYFINDING INTERFACE *129*
FIGURE 71: BRT PUBLIC WEBSITE — MOBILE APPLICATION VIEW ADVERTISING THE PASSENGER COMPANION APP AND ITS FEATURES *130*
FIGURE 72: BRT PUBLIC WEBSITE — CONTACT & COMPLAINTS VIEW FOR SUBMITTING PASSENGER FEEDBACK *131*
FIGURE 73: OPERATOR DASHBOARD - HOME OVERVIEW WITH LIVE FLEET STATUS, STATION MANAGEMENT, AND TICKETING *132*
FIGURE 74: OPERATOR DASHBOARD - SECURITY CENTRE AND COMPUTER-VISION RUNTIME ANALYSIS OF DETECTED INCIDENTS *133*
FIGURE 75: OPERATOR DASHBOARD - SOS EMERGENCY-ALERT HANDLING AND PASSENGER ACCOUNT RECORDS *134*
FIGURE 76: ADDITIONAL MOBILE APPLICATION SCREENS - AUTHENTICATION, LIVE BUS MAP, AND PASSENGER PROFILE *135*
FIGURE 77: PROPOSED UNIFIED EDGE–CLOUD TELEMETRY ARCHITECTURE INTEGRATING SMART BRT OPERATIONAL MODULES AND CENTRAL SCHEDULING INTELLIGENCE *138*
FIGURE 78: ARCHITECTURE OF THE PROPOSED VISION TRANSFORMER (VIT)-BASED VIOLENCE DETECTION MODEL *141*
FIGURE 79: PROPOSED 5G-ENABLED MEC-BASED FEDERATED LEARNING ARCHITECTURE FOR SMART BRT SYSTEMS *142*
FIGURE 80: AI-POWERED SMART BRT ECOSYSTEM *148*

Table of Tables

TABLE 1: PROJECT OPERATIONAL DOMAINS 8
TABLE 2: COMPARATIVE SUMMARY OF LITERATURE REVIEW MATRIX 21
TABLE 3: SMART BRT SYSTEM ACTOR SPECIFICATIONS 43
TABLE 4: MASTER SYSTEM USE CASE REGISTRY 48
TABLE 5: USE CASE SPECIFICATION - UC-07: PURCHASE DIGITAL TICKET 51
TABLE 6: USE CASE SPECIFICATION - UC-11: TRIGGER EMERGENCY SOS ALERT 52
TABLE 7: USE CASE SPECIFICATION - UC-16: REVIEW AUTOMATED CV SECURITY ALERTS 54
TABLE 8: USE CASE SPECIFICATION - UC-22: MANAGE FLEET DYNAMIC SCHEDULING 55
TABLE 9: ENTITY DEFINITIONS — IDENTITY BOUNDED CONTEXT 65
TABLE 10: ENTITY DEFINITIONS — PAYMENTS BOUNDED CONTEXT 65
TABLE 11: ENTITY DEFINITIONS — TICKETING BOUNDED CONTEXT 66
TABLE 12: ENTITY DEFINITIONS — TRANSPORT ASSETS BOUNDED CONTEXT 66
TABLE 13: ENTITY DEFINITIONS — TELEMETRY BOUNDED CONTEXT 66
TABLE 14: ENTITY DEFINITIONS — CV SECURITY BOUNDED CONTEXT 67
TABLE 15: ENTITY DEFINITIONS — SAFETY & REPORTING BOUNDED CONTEXT 67
TABLE 16: MASTER ENTITY-RELATIONSHIP REGISTRY: BOUNDED CONTEXT MAPPING 68
TABLE 17: CV ENTITY DATA MODEL 71
TABLE 18: CV ENTITY DATA MODEL — CVDETECTORCONFIGURATION 71
TABLE 19: CV ENTITY DATA MODEL — CVALERT 71
TABLE 20: TECHNOLOGY STACK 73
TABLE 21: CV SUBSYSTEM — ADMINISTRATIVE USE CASE SPECIFICATIONS 73
TABLE 22: MOBILE APPLICATION — GLOSSARY OF ACRONYMS AND TECHNICAL TERMS 75
TABLE 23: MOBILE APPLICATION — CORE TECHNOLOGIES AND PACKAGE DEPENDENCIES 76
TABLE 24: EDGE AI PERFORMANCE METRICS 80
TABLE 25: MOBILE APPLICATION — MINIMUM AND RECOMMENDED HARDWARE CONSTRAINTS 83
TABLE 26: WEB APPLICATION — PAGE ROUTING AND CONTENT SUMMARY 84
TABLE 27: WEB APPLICATION — ARCHITECTURAL CLASSIFICATION PROFILE 85
TABLE 28: WEB APPLICATION — FRONTEND TECHNOLOGY STACK 86
TABLE 29: WEB APPLICATION — SECURITY LAYER IMPLEMENTATION 91
TABLE 30: WEB APPLICATION — FRONTEND PERFORMANCE OPTIMIZATION TECHNIQUES 91
TABLE 31: WEB APPLICATION — PAGE LOAD PERFORMANCE TARGETS 92
TABLE 32: OVERALL PERFORMANCE SUMMARY — ALL SMART BRT AI MODULES 99
TABLE 33: WEAPON DETECTION SYSTEM DATASET DISTRIBUTION MATRIX 105
TABLE 34: PER-CLASS WEAPON DETECTION PERFORMANCE EVALUATION 106
TABLE 35: ACCIDENT PREDICTION SUB-SYSTEM PARAMETER CONFIGURATION 106
TABLE 36: ACCIDENT PREDICTION SYSTEM INCIDENT CLASSIFICATION ANALYSIS ... 107
TABLE 37: FIRE DETECTION MODULE CONFIGURATION MATRIX 108
TABLE 38: ACCESS METHODOLOGY INFRASTRUCTURE COMPARISON MATRIX 110
TABLE 39: BUS LANE MONITORING SYSTEM CONFIGURATION PROFILE 112
TABLE 40: OCR ENGINE ARCHITECTURE PERFORMANCE BENCHMARKING 113
TABLE 41: COMPARATIVE PERFORMANCE — SMART BRT SYSTEM VS. LITERATURE BENCHMARKS 114
TABLE 42: SMART BRT SYSTEM LIMITATIONS AND PROPOSED ENGINEERING MITIGATIONS 149
TABLE 43: QUANTITATIVE SUMMARY OF KEY SYSTEM FINDINGS VS. LITERATURE BENCHMARKS 145

Chapter 1: Introduction

This chapter introduces the Smart Bus Rapid Transit (BRT) initiative that motivates the project. It establishes the operational context of Egypt's BRT corridors, articulates the limitations of conventional manual operation, and frames the central problem the work addresses. It then states the project's aim, objectives, and scope, and outlines the structure of the remainder of this thesis.

1.1 Overview of Smart Bus Rapid Transit

Bus Rapid Transit (BRT) systems represent a cost-effective, high-capacity urban transport solution that blends the throughput of rail with the flexibility of bus networks. Advanced BRT deployments utilize dedicated corridors, off-board fare collection, and transit signal priority to deliver reliable service. According to the Institute for Transportation and Development Policy [8], modern BRT systems can handle up to 37,700 passengers per hour per direction, rivaling light rail performance at a fraction of the construction cost.

Egypt's Cairo Ring Road BRT corridor represents a strategic national investment in public mobility. Realizing its full potential requires transitioning from static, manual operations to AI-powered, data-driven automation. This project addresses that need through the design and implementation of a Smart BRT prototype that integrates computer vision, edge computing, and digital service platforms.

Globally, Bus Rapid Transit has become one of the most successful interventions in sustainable urban mobility. More than 180 cities now operate BRT corridors that together move tens of millions of passengers each day, and comparative analyses have shown that well-designed systems deliver rail-like capacity and reliability at a fraction of the capital cost and construction time of light rail or metro [7]. Transit authorities increasingly treat BRT not merely as a bus service but as the backbone of integrated, low-carbon transport networks that reduce congestion, greenhouse-gas emissions, and road-traffic fatalities [2].

What distinguishes a conventional BRT corridor from a smart one is the layer of sensing and automation built on top of the physical infrastructure. Advances in deep learning, and in computer vision in particular, have made it possible to extract rich operational intelligence directly from the video streams that operators already capture [13]. Modern single-stage object detectors can count passengers, gauge platform crowding, and recognise safety threats in real time, turning passive surveillance cameras into active decision-support sensors. Intelligent transportation systems that fuse these signals with fleet telemetry have been shown to improve headway regularity, shorten dwell times, and accelerate emergency response [9].

A central design consideration for such systems is where the intelligence runs. Streaming every camera feed to a central server for analysis is bandwidth-intensive, introduces latency, and raises privacy concerns. The edge-computing paradigm instead places inference close to the source — on compact GPU or accelerator

hardware mounted on the bus or at the station — so that only structured events and alerts, rather than raw video, traverse the network. This project adopts an edge-first architecture for precisely these reasons: it bounds end-to-end latency for safety-critical detections, reduces backhaul cost, and keeps personally identifiable imagery on the device wherever possible.

The operational characteristics that give BRT its advantage also make it especially amenable to automation. Dedicated, physically segregated lanes, off-board fare collection, level boarding, and intersection signal priority all serve to make the movement of buses and passengers more regular, and therefore more predictable, than in mixed traffic. This regularity is precisely the property that data-driven control exploits: when vehicles follow defined corridors and stop at known platforms, the problem of sensing occupancy, headway, and safety conditions becomes tractable, and the resulting measurements can be fed back into scheduling and enforcement with high confidence.

In the Egyptian context the case for intelligent public transport is particularly compelling. Greater Cairo is one of the most densely populated metropolitan regions in the Middle East and North Africa, and its road network carries a volume of daily trips that places sustained strain on travel times, air quality, and road safety. The national programme to develop dedicated Bus Rapid Transit corridors along the Cairo Ring Road is a direct response to this pressure, offering a high-capacity spine capable of absorbing a significant share of demand if it is operated efficiently. Yet the value of such infrastructure is realised only when the corridor is managed as a responsive, information-rich system rather than a fixed set of buses and stations.

This project situates itself within the broader global movement towards smart, sustainable cities, in which transport is treated as a cyber-physical system instrumented with sensing, connectivity, and automation. Public transit is consistently identified as one of the highest-leverage points for reducing urban greenhouse-gas emissions and congestion [2], and adding an intelligence layer multiplies that leverage by improving the efficiency, safety, and attractiveness of the service. The Smart BRT system described in this thesis is conceived as a transferable blueprint for that intelligence layer, designed around affordable hardware so that it remains realistic for transit authorities operating under the budget constraints common in developing economies.

The economic and environmental stakes of urban mobility lend further weight to this agenda. Traffic congestion imposes a substantial and recurring cost on metropolitan economies through lost productive hours, wasted fuel, and elevated logistics expense, while the associated tail-pipe emissions are a leading contributor to poor urban air quality and to the transport sector's carbon footprint. High-quality public transport is one of the few interventions that simultaneously relieves congestion and reduces emissions, but only to the extent that it persuades private-vehicle users to switch — a decision that hinges on the service being perceived as fast, reliable, safe, and convenient. The intelligence layer developed in this thesis targets exactly those perceived attributes: by smoothing flow, shortening waits, improving on-board security, and removing ticketing friction, it strengthens the value proposition that

drives the modal shift on which the wider economic and environmental benefits depend.

The benefits of such a platform accrue to several distinct groups of stakeholders, and the system is deliberately designed to serve each. For passengers, it promises shorter and more predictable waits, a safer on-board environment, and a frictionless digital experience that removes the queues associated with physical ticketing. For the operators and the authority that run the corridor, it converts a previously inert stream of surveillance video into a continuous source of operational intelligence, supporting demand-responsive scheduling, faster incident response, and automated enforcement of the dedicated right-of-way. For the security personnel charged with safeguarding the network, it provides an attentive second set of eyes that never tires and that surfaces only the events meriting intervention. Designing explicitly around these complementary perspectives is what allows a single technical platform to deliver value simultaneously to the people who ride, run, and protect the system.

The work also aligns with the broader national agenda for digital transformation and smart-city development. Initiatives of this kind position technology-enabled public services as a pillar of sustainable urban growth, and intelligent transport sits squarely within that vision as one of the services with the most direct and visible impact on citizens' daily lives. A locally developed, affordable Smart BRT platform contributes to this agenda not only as a deployable system but as evidence that the requisite expertise and tooling can be marshalled domestically, reducing dependence on imported, proprietary solutions and building engineering capacity that can later be redirected to adjacent problems.

1.2 Problem Statement and Motivation

Problem Statement

Contemporary BRT operations face several critical challenges that limit service quality and passenger safety:

- **Inefficient Capacity Management:** Fleet operators lack real-time visual telemetry on bus occupancy, making demand-responsive scheduling impossible.
- **Passenger Safety Deficiencies:** Passive CCTV monitoring cannot detect security incidents such as violence or weapon possession in real time.
- **Reactive Emergency Response:** Fire and accident detection relies on human reporting, increasing response latency.
- **Platform Overcrowding:** No automated feedback loop exists to alert dispatch when station platforms reach critical density.

- **Ticketing Bottlenecks:** Physical ticketing and manual validation create congestion at entry turnstiles.
- **Driver Non-Compliance:** No automated mechanism monitors operator fatigue, distraction, or unsafe behavior.

Taken together, these deficiencies describe a transit operation that is, in effect, operationally blind between control decisions: rich visual data is continuously captured by the existing camera network yet is never converted into the structured, real-time signals that scheduling, security, and safety responses require. The common thread is the absence of an automated perception layer. Each challenge — capacity, safety, emergency response, crowding, ticketing, and driver compliance — could be mitigated if the video already flowing through the system were interpreted automatically and turned into timely, actionable events. This observation is the central premise of the present work.

The cost of leaving these gaps unaddressed is measured in both operational and human terms. Without real-time occupancy data, buses run to fixed timetables that are frequently mismatched to demand, producing overcrowded vehicles at peak times and near-empty ones at others, eroding both efficiency and rider confidence. Without automated incident detection, security and safety events depend on a human operator noticing the right camera at the right moment among dozens of simultaneous feeds, an expectation that is unrealistic to sustain across a full shift. And without automated enforcement and access control, the very infrastructure that gives BRT its speed advantage is left vulnerable to lane violations and fare evasion. Each gap therefore translates directly into degraded service, elevated risk, or lost revenue.

Among the challenges enumerated above, passenger safety occupies a position of particular urgency. Conventional closed-circuit television was conceived as a forensic and deterrent technology: it records what happens so that footage can be reviewed afterwards, but it does little to prevent or interrupt an incident as it unfolds. In a busy corridor a single operator may be responsible for monitoring dozens of simultaneous video feeds, and human attention simply cannot be distributed across so many streams without lapses, particularly over the course of a long shift. The consequence is a structural gap between the moment an incident becomes visible to a camera and the moment a human notices it and responds. Automated, always-on analysis closes that gap by watching every feed continuously and escalating only the frames that warrant human attention, transforming the camera estate from a passive archive into an active early-warning system.

The capacity-management challenge is equally consequential, if less immediately dramatic. When a fleet runs to a fixed timetable that is blind to actual demand, the mismatch manifests as overcrowding during unanticipated peaks and as wasteful, near-empty running during troughs. Both outcomes are costly: the former degrades the passenger experience and can leave travellers stranded on full vehicles, while the latter squanders fuel, driver hours, and rolling-stock availability. Demand-responsive scheduling — in which dispatch decisions are informed by live measurements of

platform crowding and on-board occupancy — directly addresses this mismatch, but it is feasible only if those measurements exist in the first place. Supplying them accurately and continuously is one of the primary motivations for the sensing capability built in this project.

Project Motivation

The driving force behind this research is the urgent need to align Egypt's public transit infrastructure with modern smart city frameworks, specifically supporting sustainable development goals and Egypt's Vision 2030. By replacing reactive security and manual fleet management with proactive Edge AI solutions, this project aims to minimize economic losses caused by urban congestion, optimize resource allocation for transit authorities, and fundamentally ensure a safer, smarter, and more reliable daily commute for millions of citizens.

Beyond these technical drivers, there is a strong socio-economic motivation. Public transport in Egypt carries a large share of daily commuters, many of whom are highly sensitive to fare, safety, and reliability. A system that can demonstrably reduce crowding, deter and rapidly surface security incidents, and replace queue-bound paper ticketing with a seamless digital experience addresses the very concerns that shape ridership. Demonstrating that this can be achieved with affordable, commercially available hardware — rather than bespoke industrial equipment — is itself a contribution, because it lowers the barrier for transit authorities in developing economies to adopt smart-mobility technology.

The approach taken in this thesis is therefore to build that missing perception layer and to wire it directly into the operational and passenger-facing systems of the corridor. In concrete terms this means deploying a family of computer-vision models at the edge, close to the cameras, so that detections are produced with minimal latency; consolidating their outputs in a central backend that maintains the authoritative state of the fleet, stations, and passengers; and exposing the resulting intelligence through a passenger mobile application and an operator dashboard. The remainder of this introduction sets out the specific objectives, scope, and tools through which this approach is realised.

Two developments make this an opportune moment to address these problems. First, deep-learning object detection has matured to the point where a single model can run accurately and in real time on inexpensive, power-efficient hardware, removing the historical dependence on costly specialised sensors. Second, the cameras required as the primary sensing modality are, in most modern transit deployments, already installed for conventional surveillance. The marginal cost of adding intelligence is therefore largely a matter of software and modest edge-compute hardware, which is exactly the gap this project sets out to fill.

The convergence of several technological trends makes the present approach feasible where comparable attempts in earlier years were not. Detection models that once demanded server-class accelerators can now be compressed and optimised to run on palm-sized edge devices while retaining usable accuracy; transfer learning allows

these models to be specialised to a new domain from comparatively modest datasets; and mature runtimes permit a model trained in one framework to be exported and executed efficiently in another, decoupling research tooling from deployment. Crucially, the sensing substrate — networked cameras — is already present in most modern transit environments for conventional surveillance. The incremental investment required to add intelligence is therefore dominated by software and a modest amount of edge-compute hardware, which places the capability within reach of the budgets that transit authorities in the region actually operate under.

The digital-service dimension deserves emphasis in its own right, because it is the part of the system with which the ordinary passenger interacts most directly. Traditional fare collection — physical tickets, manual validation, and cash handling — is a persistent source of delay at entry points, a vector for revenue leakage, and an obstacle to the kind of integrated, account-based travel that modern riders expect. Replacing it with a cross-platform mobile application that supports digital ticketing, season passes, wallet top-ups, and live trip planning removes that friction at the point of use while simultaneously generating the structured ridership data on which planning and scheduling depend. The application is therefore not a peripheral convenience but an integral component of the operational system.

1.3 Project Objectives

This project is structured around the following engineering objectives:

- Develop and optimize specialized deep learning vision pipelines for passenger counting, crowd density estimation, weapon detection, fire localization, and anomaly behaviour detection.
- Design and deploy a cross-platform mobile application and administrative web portal for live tracking, occupancy data, and electronic ticketing.
- Implement a high-speed biometric facial recognition gate as a blueprint for touchless passenger authentication at station turnstiles.
- Evaluate all AI models under real-world operational constraints using standard metrics: precision, recall, F1-score, and inference latency.
- Integrate all modules into a cohesive edge-cloud architecture suitable for deployment in urban transit environments.

Collectively, these objectives trace a single path from raw sensor data to actionable intelligence and, ultimately, to an improved passenger experience. They are deliberately framed to be measurable: each vision pipeline is evaluated against standard detection metrics, the applications are assessed for responsiveness and usability, and the integrated system is judged by its end-to-end latency and reliability.

Meeting them in combination — rather than demonstrating any single capability in isolation — is what distinguishes an integrated smart-transit platform from a collection of independent experiments.

These objectives also define the research methodology that the remainder of the thesis follows. The vision pipelines are developed and tuned against curated datasets and then evaluated quantitatively; the backend and applications are engineered using established software-engineering practice and assessed against functional and non-functional requirements; and the integrated platform is validated end to end. This progression — from isolated model development, through subsystem engineering, to integrated evaluation — mirrors the structure of the chapters that follow and ensures that every claim made about the system is supported by either an experimental result or a demonstrable implementation.

The first objective, developing the vision pipelines, constitutes the technical core of the project. It is not a single detection task but a family of them — crowd density, directional counting, weapon presence, abnormal behaviour, fire, accidents, and lane intrusion — each with its own data characteristics, failure modes, and tolerance for error. A weapon or fire detector must err on the side of recall, since a missed event is far costlier than a false alarm, whereas a passenger counter must maintain precise, stable identities across frames to avoid double-counting. Reconciling these differing requirements within a common, edge-deployable framework, and tuning each model to its operating point, is a central engineering contribution of the work.

Equally central is the objective of integration. A collection of accurate models is of little operational use until their outputs are fused into a coherent, queryable picture of the corridor and delivered to the people who act on it. This is the role of the backend, which maintains the authoritative state of buses, stations, passengers, and incidents, and of the real-time messaging fabric that pushes events to operators and passengers as they occur. Treating the platform as a single system — rather than a set of disconnected demonstrations — is what allows upstream detections, such as a congestion estimate, to inform downstream decisions, such as a dispatch recommendation, and it is the property that most clearly distinguishes this work from a portfolio of isolated proofs of concept.

Finally, the significance of the project extends beyond the immediate prototype. By assembling a comprehensive, multi-capability transit-intelligence platform from openly available models and commodity hardware, the work demonstrates a path to smart-transit modernisation that does not depend on proprietary, capital-intensive systems. The resulting architecture is intended to serve as a reusable reference: its layering, its separation of the vision engine from the application backend, and its privacy-preserving treatment of biometric data are all transferable design choices that other transit authorities, and subsequent research, can adopt and extend. In a regional context where modernisation budgets are constrained, demonstrating that capability of this breadth is attainable affordably is, in itself, a meaningful outcome.

1.4 Project Scope

The Smart BRT prototype operates across three organizational domains, as summarized in Table 1:

Table 1 :Project Operational Domains

Domain	Core Components	Technical Focus
On-Bus Systems	In-cabin CCTV arrays, door-mounted counters, driver monitors	Edge video analytics, spatial tracking, safety validation
Station Infrastructure	Platform cameras, gate sensors, access control hardware	Crowd profiling, density analysis, biometric verification
Digital Interaction Layer	Database, web dashboards, Flutter mobile application	Ticketing, live tracking, transit telemetry

Note: Physical hardware components are evaluated at the prototype level, simulating real-world constraints within a controlled laboratory environment.

In summary, the principal contributions of this work are fourfold: first, the design and integration of eight complementary computer-vision modules into a single, coordinated transit-intelligence pipeline; second, a production-grade backend and data model that turn raw detections into actionable operational events; third, cross-platform passenger and operator applications that expose this intelligence through usable, fully localised interfaces; and fourth, an empirical evaluation of every model under realistic operating conditions. The scope is deliberately framed at the prototype level so that the architectural blueprint, rather than any single piece of hardware, constitutes the transferable outcome of the research.

It is important to delineate the boundaries of this scope precisely. The work targets the design, implementation, and evaluation of the software intelligence layer and its supporting applications; the physical procurement, installation, and certification of station and on-board hardware lie outside its remit and are addressed at the prototype level only. Likewise, the emphasis is on demonstrating that each capability is technically achievable and that the modules cooperate as a coherent system, rather than on a full-scale field deployment across an operational corridor, which would require institutional partnerships beyond the span of this research.

Realising even this prototype scope raises several cross-cutting considerations that recur throughout the thesis. Chief among them are latency, since safety-critical detections must be produced quickly enough to be acted upon; privacy, since the system processes imagery of members of the public and, in the access-control case, biometric data; and integration, since eight independently trained models, a transactional backend, and two client applications must behave as a single system. The design decisions taken in response — edge inference, on-device processing with

embedding-only biometric storage, and a clean layered architecture — are introduced here and revisited in detail in the chapters that follow.

The anticipated outcome of the project is twofold. Practically, it is a working prototype that demonstrably ingests live video, detects the targeted operational and safety events, and surfaces them to passengers and operators through usable applications. More broadly, its contribution is to show that a comprehensive, multi-capability transit-intelligence platform can be assembled from open models and commodity hardware, lowering the barrier to adoption and providing a reference architecture that future work, and other transit authorities, can build upon.

1.5 Hardware and Software Tools

Hardware

The Smart BRT platform was developed and evaluated on a hardware configuration that balances model-training throughput with realistic edge-deployment constraints. The principal components are summarized below.

- **NVIDIA GeForce RTX 3060 (6 GB VRAM):** Used for model training, hyperparameter tuning, and optimization.
- **NVIDIA Jetson Nano / Raspberry Pi 4:** Deployed as edge inference nodes to evaluate lightweight model performance.
- **Intel Core i7 / AMD Ryzen 7:** Utilized for data preprocessing, dataset management, and local web service hosting.
- **Peripherals:** Wide-angle zenithal cameras, HD IP/CCTV arrays, and Near-Infrared (NIR) biometric gate sensors.

Software Stack

The software environment integrates a Python-based computer-vision research stack with a .NET 8 production backend and cross-platform client applications. The core tools and frameworks are listed below.

- **AI/CV Core:** Python 3.10+, PyTorch, Ultralytics YOLOv8, InsightFace (ArcFace), FAISS, ONNX Runtime, and OpenCV.
- **Data & Analytics:** NumPy, Pandas, Roboflow, Matplotlib, Seaborn, and cvzone.

- **Mobile Application:** Flutter & Dart framework (cross-platform iOS/Android development), Google Maps API.
- **Web Platform:** HTML5, CSS3, JavaScript, and Angular framework.
- **Backend:** ASP.NET Core (.NET 8) Web API, SignalR (for real-time duplex communication) and TLS encryption.

1.6 Thesis Organization

The remainder of this thesis is organised into six chapters. Chapter 2 (Background) reviews the core technologies that underpin the system, including convolutional neural networks, the YOLOv8 detection architecture, object tracking, and ArcFace/FAISS biometric matching. Chapter 3 (Literature Review) surveys related work in intelligent transportation and computer-vision-based monitoring and identifies the research gap this project addresses. Chapter 4 (Proposed Work) presents the system design and methodology, the eight computer-vision modules, the use-case model, and the database schema. Chapter 5 (Results and Discussion) reports the experimental evaluation of each module together with the mobile and web applications. Chapter 6 critically discusses the system's strengths, limitations, and future roadmap, and Chapter 7 concludes the thesis.

Chapter 2: Background

This chapter establishes the technical foundations on which the Smart BRT system is built. It introduces the deep-learning and computer-vision concepts that the eight detection modules depend upon — convolutional neural networks, the YOLOv8 single-stage detection architecture, multi-object tracking, and the ArcFace embedding and FAISS indexing pipeline used for biometric access control — and explains how each technology contributes to accurate, low-latency inference on edge hardware.

2.1 Convolutional Neural Networks and Computer Vision

Convolutional Neural Networks (CNNs) are the foundational architecture behind modern computer vision systems. A CNN processes an input image through a sequence of learnable convolutional filters that extract hierarchical spatial features — edges and textures in early layers, and high-level semantic concepts such as objects and scene context in deeper layers [13].

The key components of a standard CNN architecture include:

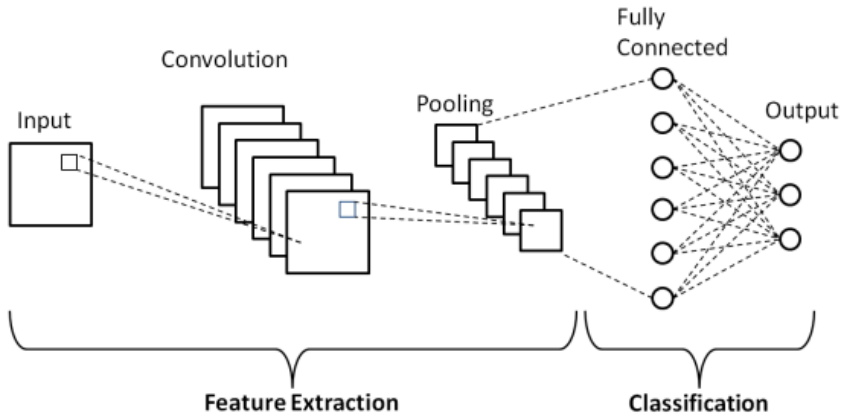


Figure 1: The key components of a standard CNN architecture

- **Convolutional Layers:** Apply a set of learnable filter kernels to the input tensor to generate specialized feature maps.
- **Activation Functions:** Typically the Rectified Linear Unit (ReLU) or its variants, which introduce non-linearity, allowing the network to learn complex bounding representations.
- **Pooling Layers:** Downsample the feature maps to reduce spatial dimensions, minimizing computational cost and improving translational invariance.
- **Fully Connected Layers:** Flatten the final feature maps into dense vectors to perform high-level classification or regression tasks.

Modern architectures, such as ResNet and EfficientNet, build upon these fundamentals by incorporating residual skip-connections and compound scaling parameters to achieve unprecedented accuracy on dense visual tasks without suffering from vanishing gradients.

These operations make CNNs uniquely suited to the visual tasks that arise in a transit environment. Because convolutional filters are translation-invariant and learn a hierarchy of features — from edges and textures in the early layers to object parts and whole objects in the deeper layers — the same architectural principles underpin every module in this work, whether the target is a crowded platform, a raised weapon, or a human face. Crucially, the convolution operation is highly parallelisable, which allows these networks to be accelerated on the compact GPUs used at the edge and to meet the real-time throughput that on-vehicle and in-station analytics demand.

2.2 YOLOv8 Architecture

The **You Only Look Once (YOLO)** family represents a benchmark class of single-stage object detection architectures that process an entire image frame through a single forward pass, optimized heavily for real-time edge inference. YOLOv8, engineered by Ultralytics, introduces an anchor-free detection paradigm coupled with a decoupled prediction head, yielding superior speed-to-accuracy trade-offs compared to prior iterations.

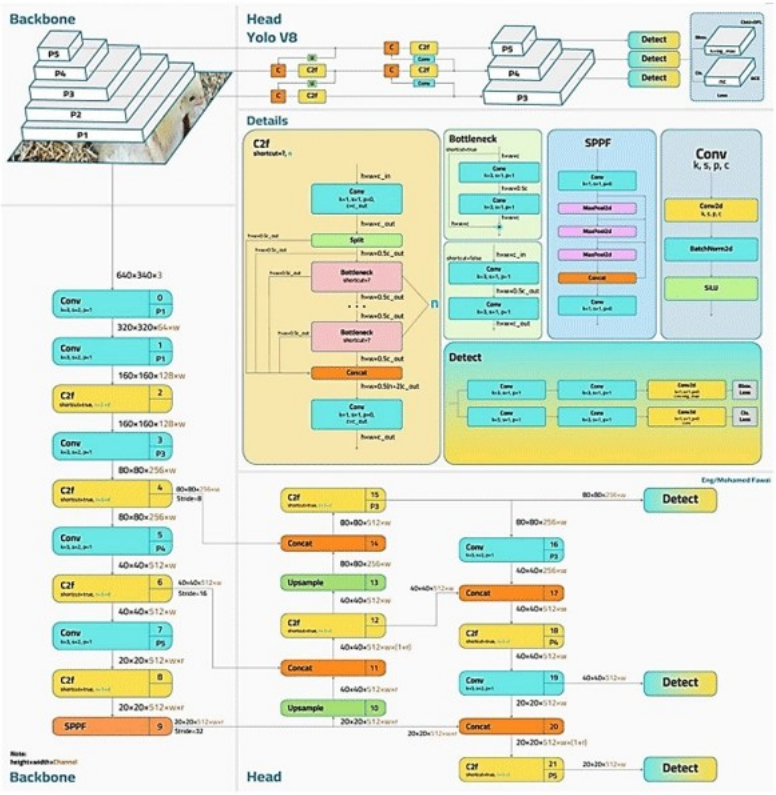


Figure 2 :YOLOv8 Architecture

2.2.1 Backbone

The YOLOv8 backbone leverages an optimized **Cross-Stage Partial (CSP)** network layout. By partitioning feature maps before and after convolution operations, this design limits redundant gradient computation steps and accelerates feature reuse across the network. This results in a highly compact, lightweight model footprint ideal for resource-constrained deployments.

Neck

The neck structure utilizes a hybrid **Path Aggregation Network (PANet)** alongside a **Feature Pyramid Network (FPN)**. The FPN component propagates rich semantic attributes top-down from deep, high-level feature maps to shallower layers. Concurrently, the PANet architecture passes precise localization boundaries bottom-

up. This bidirectional flow guarantees high spatial detection accuracy across multi-scale target objects.

Head

YOLOv8 transitions to a **decoupled detection head**, processing classification categories and bounding box regression variables in entirely separate computational branches. This structural separation prevents cross-task gradient conflicts, enhancing final localization targets. Furthermore, the head utilizes **Distribution Focal Loss (DFL)** for bounding box edge predictions, yielding sharp, stable bounding limits even under highly ambiguous visual conditions.

Two properties make the YOLOv8 family the natural choice for this project. First, as a single-stage detector it performs localisation and classification in a single forward pass, giving it a decisive speed advantage over two-stage detectors such as Faster R-CNN while remaining competitive on accuracy [23]. Second, it is distributed in several scales (n, s, m, l, and x) that trade accuracy against latency, so each Smart BRT module can be matched to the compute budget of its host device — a lightweight variant on an on-board edge node, a larger variant where a station server is available. After training in PyTorch, the models are exported to the ONNX format and executed through ONNX Runtime inside the C# computer-vision engine, decoupling deployment from the training framework.

2.3 Object Tracking: BoT-SORT and Kalman Filter

Object tracking extends static spatial detection by maintaining persistent, distinct identity vectors (IDs) for targets tracked across continuous temporal frames. This project implements **BoT-SORT** (Bootstrapped Object Tracking via SORT), an advanced tracker that unifies localized motion state prediction with robust visual appearance matching.

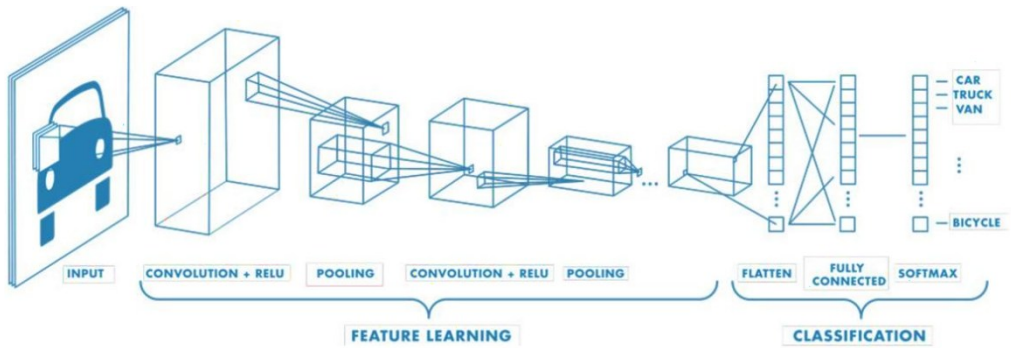


Figure 3 :BoT-SORT and Kalman Filter Tracking Pipeline

The core motion estimation component relies on the **Kalman Filter**. It maintains a real-time probabilistic state vector tracking the estimated position and relative

velocity of each object. Before a new visual frame updates the architecture, the filter projects the target's position using a sequence of specialized state equations.

The mathematical state sequence is defined as follows:

$$\hat{x}_{k|k-1} = F\hat{x}_{k-1|k-1}$$

$$P_{k|k-1} = FP_{k-1|k-1}F^T + Q$$

$$K_k = P_{k|k-1}H^T(HP_{k|k-1}H^T + R)^{-1}$$

$$\hat{x}_{k|k} = \hat{x}_{k|k-1} + K_k(z_k - H\hat{x}_{k|k-1})$$

When a new frame detection is registered, the tracker computes the **Kalman Gain (K)** to balance prediction certainty against measurement noise, executing the following correction step:

Where:

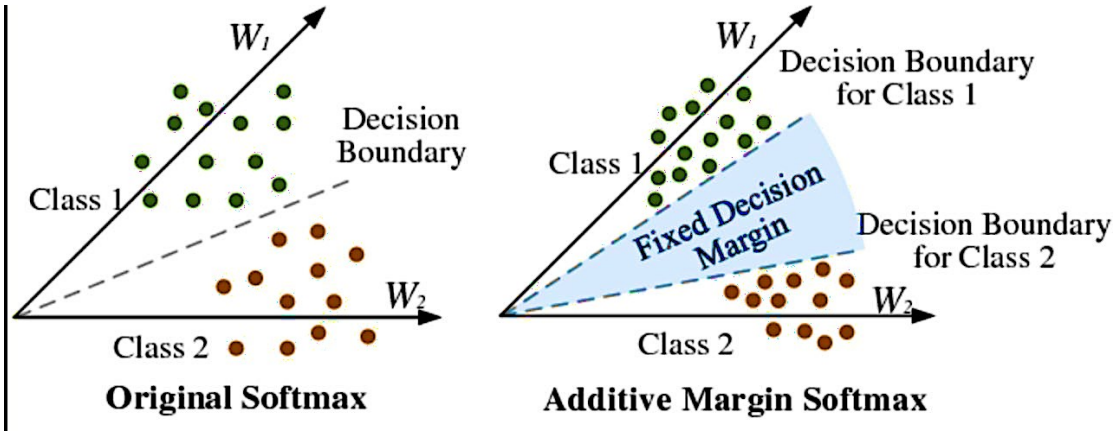
- F represents the state transition matrix,
- P denotes the state error covariance matrix,
- Q and R represent the process and measurement noise covariance parameters respectively,
- H represents the observation mapping matrix,
- 2k represents the actual measured vector at frame k.

To mitigate tracking failures caused by severe occlusions—such as passenger clustering around bus doors—BoT-SORT augments this spatial prediction by extracting visual appearance embeddings, verifying identity consistency over extended time horizons.

Reliable multi-object tracking is what turns frame-by-frame detections into the persistent identities on which the downstream logic depends. Passenger counting, for example, is only meaningful if the same individual is followed across the door threshold rather than recounted in every frame, and behaviour analysis requires a continuous motion history. BoT-SORT augments the classical detect-then-associate pipeline with camera-motion compensation and an appearance-embedding cue, which together reduce identity switches under the crowding and occlusion that are endemic to transit scenes [1].

2.4 ArcFace and Biometric Recognition

ArcFace is a state-of-the-art deep facial recognition framework designed to extract highly discriminative, low-dimensional facial feature embeddings. It accomplishes this by enforcing an **Additive Angular Margin (AAM)** boundary directly within the target feature space during training. Unlike traditional cross-entropy or standard softmax functions, AAM maps features onto a hypersphere, maximizing inter-class angular distance while minimizing intra-class variance.



The optimization target is formulated through the ArcFace loss function:

$$L = -\log \frac{e^{s(\cos(\theta_{y_i}+m))}}{e^{s(\cos(\theta_{y_i}+m))} + \sum_{j \neq y_i} e^{s\cos\theta_j}}$$

Where:

- m denotes the additive angular margin parameter (standardized at 0.5 radians),
- s represents the constant feature scaling radius,
- θ_y is the targeted angle separating the extracted embedding from the ground-truth weight vector.

This project utilizes the **InsightFace** library implementation (specifically the *buffalo_l* deep model backbone) to project detected facial crops into static, 512-dimensional vector spaces. Identity validation is performed by evaluating the exact cosine similarity metrics between live probe vectors and the localized database gallery.

Within the Smart BRT platform, ArcFace is the engine of the biometric access-control gate. Each enrolled passenger is represented by a single 512-dimensional embedding produced by the network; verification then reduces to measuring the angular distance between a live capture and the stored template. Because the additive angular margin

enforces large inter-class separation during training, these embeddings remain discriminative even under the pose, expression, and illumination variation typical of a station gate [3]. Storing only the abstract embedding rather than the source image is also central to the system's privacy posture.

2.5 FAISS: Fast Approximate Nearest Neighbor Search

FAISS (Facebook AI Similarity Search) is a highly efficient library optimized for executing rapid similarity searches and vector clustering within high-dimensional datasets. Within the automated biometric access gate subsystem, FAISS handles the k-nearest neighbor (k-NN) matching requests across the registered 512-dimensional passenger database.

While FAISS offers various complex structural index options—such as Inverted File structures (IVF) or Hierarchical Navigable Small World graphs (HNSW) to optimize large scales—this implementation utilizes a localized **Flat Inner-Product (IndexFlatIP)** configuration. Given the specific database constraints of registered transit commuters, the flat index structure ensures 100% search accuracy while maintaining sub-millisecond querying latencies.

FAISS is what allows biometric verification to scale. As the enrolled population grows, a brute-force comparison of a probe embedding against every stored template becomes prohibitively slow; FAISS instead organises the 512-dimensional vectors into an index that answers nearest-neighbour queries in sub-millisecond time [12]. For the gate-control use case, in which a passenger must be authenticated in the time it takes to approach the turnstile, this indexed search is the difference between a fluid and an unusable experience.

2.6 Anomaly Detection in Video Surveillance

Behavioural anomaly detection departs from the supervised, class-specific paradigm of conventional object detectors. Because abnormal events on public transport are rare, diverse, and difficult to enumerate exhaustively, the prevailing approach is semi-supervised reconstruction learning: a model is trained only on normal footage and learns to reconstruct it with low error, so that frames containing unseen or abnormal patterns yield a high reconstruction error that serves as the anomaly signal. Convolutional autoencoders, and more recently encoder-decoder networks built on pretrained backbones such as EfficientNet, are widely used for this spatial reconstruction task, with the mean squared error (MSE) between input and reconstruction acting as a per-region anomaly score. Purely spatial models, however, cannot capture motion dynamics, so they are complemented by spatiotemporal networks. Three-dimensional convolutional architectures such as ResNet3D extend residual learning across the temporal axis, classifying short clips of consecutive frames as normal or abnormal and thereby capturing aggressive or unusual actions that unfold over time. The anomaly subsystem proposed in this work adopts exactly this hybrid philosophy, pairing a reconstruction autoencoder with a three-dimensional convolutional classifier.

2.7 Software Engineering Foundations

Beyond the vision models, the platform rests on a set of established software-engineering foundations that are introduced here at a conceptual level; their concrete realisation is detailed in Chapter 4. The backend follows the Clean Architecture (onion) paradigm, which isolates domain logic from frameworks and infrastructure through concentric dependency layers, improving testability and long-term maintainability. Real-time delivery of detections, vehicle positions, and alerts relies on a persistent, bidirectional channel rather than request-response polling; the WebSocket protocol, abstracted by the SignalR library on the .NET platform, provides this low-latency push capability. Persistent state is managed through an object-relational mapper (Entity Framework Core) over a relational database, giving strongly typed, transactional access to the data model later presented as an entity-relationship diagram. On the client side, a single cross-platform toolkit (Flutter) targets both Android and iOS from one codebase, while a component-based single-page-application framework (Angular) powers the operator dashboard. These paradigms, namely layered architecture, real-time messaging, ORM-based persistence, and cross-platform user interfaces, constitute the engineering backdrop against which the proposed system is built.

Chapter 3: Literature Review

This chapter reviews prior research and existing systems relevant to the Smart BRT platform. It surveys the state of the art in Bus Rapid Transit operations, the application of artificial intelligence and computer vision to public transportation, and the specific detection problems addressed in this work — passenger counting, weapon and threat detection, and biometric access control — before summarising the research gaps that motivate the proposed system.

3.1 Bus Rapid Transit Systems

Bus Rapid Transit (BRT) systems have attracted growing research attention as a highly scalable and resilient urban transport solution.

- **Cost-Benefit and Throughput Analysis:** According to the Institute for Transportation and Development Policy [8], advanced BRT corridors can achieve massive passenger throughputs of up to 37,700 passengers per hour per direction (PPHPD). This efficiency is delivered at infrastructure construction costs ranging from $7 million to $15 million per kilometer—approximately one-tenth the capital expenditure required for underground heavy metro lines.
- **Socio-Economic Impacts:** Hidalgo and King [7] evaluate the long-term post-implementation outcomes of major Latin American BRT networks, reporting

measurable, systemic reductions in commuter travel times and localized traffic accident rates. Similarly, Guthrie [5] examines broader economic development effects, demonstrating that commercial and residential real estate construction tends to concentrate heavily within a half-mile radius of active BRT corridors, and that transit investments correlate directly with higher-wage job growth in surrounding municipal areas.

- **Environmental Sustainability:** The American Public Transportation Association [2] documents the modern environmental case for public transit networks. Their findings indicate that a targeted modal shift from private vehicles to organized, high-frequency bus networks significantly mitigates urban greenhouse gas emissions and minimizes the severe, stop-and-go congestion patterns that generate high localized air pollution.

3.2 AI and Computer Vision in Public Transportation

Recent advancements in deep learning have catalyzed a shift from legacy transit monitoring to proactive, automated environments.

- **Real-Time Incident Threat Response:** Liu, Zhang, and Wang [14] propose specialized deep learning-based detection algorithms tailored for identifying anomalous passenger behavior and locating prohibited items inside transit vehicles. Their framework demonstrates that modern computer vision pipelines can successfully transition surveillance operations from passive recording to active, real-time threat response. While their framework achieves high detection accuracy under variable cabin lighting conditions, it remains limited to localized, single-camera deployments.
- **Scalability Over Physical Sensors:** A comprehensive survey by MDPI [17] focusing on AI-driven approaches to urban transit sustainability reviews multiple deep learning edge applications—including automated passenger counting, proactive fleet scheduling, and safety incident detection. The survey concludes that Convolutional Neural Network (CNN)-based visual systems consistently outperform legacy physical sensor-based alternatives in both overall measurement accuracy and spatial scalability.

3.3 Passenger Counting Technologies

Automatic Passenger Counting (APC) technologies span several distinct methodological approaches within the literature.

- **Video Analytics vs. Overcrowding:** An evaluation by PMC/NCBI [19] analyzes video-based counting architectures utilizing standard CCTV cameras on metropolitan commuter trains. Their system reports localized accuracy levels above 90% under controlled settings but notes a significant performance degradation when subjected to dense physical occlusion and heavy crowding.
- **The Shift to Deep Learning:** The Rochester Institute of Technology [21] provides a critical review of emerging APC concepts, concluding that AI-based video analytics represent the definitive state-of-the-art methodology for high-density public transit due to their structural robustness and environmental generalizability. This industrial shift is further validated by Retail Sensing [20], confirming a widespread commercial adoption of AI-powered video counting frameworks over legacy infrared barriers and weight-based step sensors due to video's superior accuracy in dense, volatile passenger environments.
- **Targeted Implementations:** The International Journal of Information Engineering and Science [10] presents a specialized CNN-based YOLOv8 tracking system configured explicitly for passenger counting on public municipal buses, achieving precision and recall metrics exceeding 93%.
- **Methodological Optimizations:** Further comparative reviews by MDPI Automation [15] validate that Region-Based and Object Tracking methods vastly outperform simple, legacy virtual-line crossing approaches, particularly during peak boarding and alighting events. To address the problem of dense spatial clustering, an arXiv preprint by Head Anchor [6] proposes a specialized head-enhanced detection paradigm for crowded pedestrian tracking. This directly informs the dual Head/Person class selection strategy adopted within this project's vision architecture.

3.4 Weapon and Threat Detection

Ensuring onboard and station security demands high-precision visual object detection backbones capable of running under low-latency constraints.

- **Object Detection Backbones:** Research published in MDPI Sensors [16] presents a customized YOLOv8-based vehicle and pedestrian detection framework that achieves a high mean Average Precision (mAP) across diverse, unconstrained urban scenarios. This validates the YOLOv8 architecture as an exceptionally effective, stable backbone for safety-critical transit applications.
- **Deployment Protocols:** To ensure these architectures integrate safely into public spaces, the Intelligent Transportation Society of America [9] establishes deployment standards for automated visual surveillance in transit. Their guidelines define strict best practices for confidence threshold configurations and real-time alert escalation logic to minimize false alarms while ensuring passenger safety.

3.5 Biometric Access Control

Transitioning transit stations into smart hubs requires frictionless, secure, and rapid passenger authentication mechanisms at entry boundaries.

- **Throughput and Latency Optimization:** Meegle [18] reviews automated facial recognition deployments within high-throughput subway turnstile environments. Their documentation proves that deep embedding-based approaches utilizing the ArcFace architecture achieve an authentication throughput exceeding 30 passengers per minute in high-capacity rail hubs.
- **Hardware-Software Integration:** Industry-level documentation from both Turbo Turnstile [22] and Iron Man Turnstile [11] confirms an accelerating municipal adoption of facial recognition swing gates in transit infrastructure. Their technical reviews emphasize the necessity of pairing deep embedding matchers with Near-Infrared (NIR) physical sensors to maintain reliable verification under highly variable station lighting conditions.
- **Privacy-by-Design Compliance:** Addressing data governance concerns, Facia.ai [4] outlines the regulatory and privacy frameworks governing facial recognition in modern smart cities. They demonstrate that edge-processing

architectures—which immediately delete raw video frames and store exclusively mathematical, non-invertible vector embeddings—provide a legally defensible, "privacy-by-design" posture that satisfies strict compliance mandates.

3.6 Summary of Related Work

To provide a structured evaluation of the state-of-the-art methods found in the literature, Table 2 summarizes the core focuses, methodologies, performance benchmarks, advantages, and limitations of the reviewed studies. This matrix serves as the baseline for determining structural optimizations in this project.

Table 2: Comparative Summary of Literature Review Matrix

Reference / Study	Focus / Domain	Model Used	Accuracy / Performance	Pros	Cons
Liu et al. (2025)	Behavior & Threat Detection	Custom Deep Learning	High performance under variable light	Enables real-time threat response rather than passive recording	Limited to single-camera deployments; lacks multi-node scalability
MDPI Survey (2026)	Urban Transit Sustainability	CNN-based Systems	Outperforms legacy physical sensors	Highly scalable approach; addresses long-term transit scheduling	Comprehensive survey only; does not provide a custom edge prototype
PMC/NCBI (2020)	Video-based Passenger Counting	Standard CCTV Tracking	> 90% in controlled environments	Low-cost hardware integration using existing train infrastructure	Suffers severe degradation under high crowding and physical occlusions
Int. Journal of Info. Eng. (2026)	Passenger Counting on Buses	YOLOv8 CNN Pipeline	> 93% Precision and Recall	Highly optimized for dynamic bus cabin constraints	Functional silo; focuses exclusively on counting without security features

Meegle (2026)	Subway Turnstile Face Recognition	ArcFace Embeddings	Throughput > 30 passengers/minute	Extremely high throughput; strictly follows privacy-by-design frameworks	Highly dependent on specialized Near-Infrared (NIR) physical sensors

3.7 Research Gaps

The reviewed body of literature establishes three clear consensus findings:

1. **Socio-Economic Viability:** BRT networks offer a highly cost-effective, scalable alternative to traditional rail systems, yielding immediate traffic safety and urban efficiency returns.
2. **Technical Maturity:** Deep learning and real-time computer vision models have matured sufficiently to allow reliable, edge-based deployment in public transit environments.
3. **Module Isolation:** Specialized individual AI modules—such as automatic passenger counting, weapon threat localization, and biometric access validation—have been successfully demonstrated in isolation.

Identified Research Gap:

Despite these independent advancements, a major integration gap remains within the existing literature: **no current implementation orchestrates these disparate vision pipelines and digital layers into a single, unified edge-cloud ecosystem** tailored specifically to the operational complexities of a full-scale BRT network.

Most existing architectures treat these problems as isolated functional silos. This fragmentation significantly inflates physical hardware overhead on vehicles and prevents real-time, cross-modular data fusion. This project directly addresses this gap by presenting a unified, production-grade framework that cross-communicates telemetry across all subsystems.

Chapter 4: Proposed Work

This chapter presents the design of the proposed Smart BRT platform in full. It describes the overall system architecture and data flow, specifies each of the eight AI modules together with their detection pipelines, and documents the system's actors, use cases, data model, and the mobile and web application surfaces that expose the platform to its users.

4.1 System Overview

The Smart BRT system integrates eight autonomous Artificial Intelligence (AI) modules, each targeting a distinct operational domain within the Bus Rapid Transit ecosystem. Rather than acting as independent silos, these modules form a highly coordinated intelligence pipeline where the output of upstream subsystems directly feeds or informs downstream components.

At a high level, the platform is organised into four cooperating layers: an edge-capture layer of station and on-board cameras; a computer-vision engine that executes the YOLOv8 detectors and the ArcFace facial-recognition pipeline; a .NET 8 application backend with its SQL Server data store and SignalR real-time hub; and a client layer comprising the Flutter passenger application and the Angular operations dashboard. The basic elements of the system and the flow of data between them are shown in the figure below.

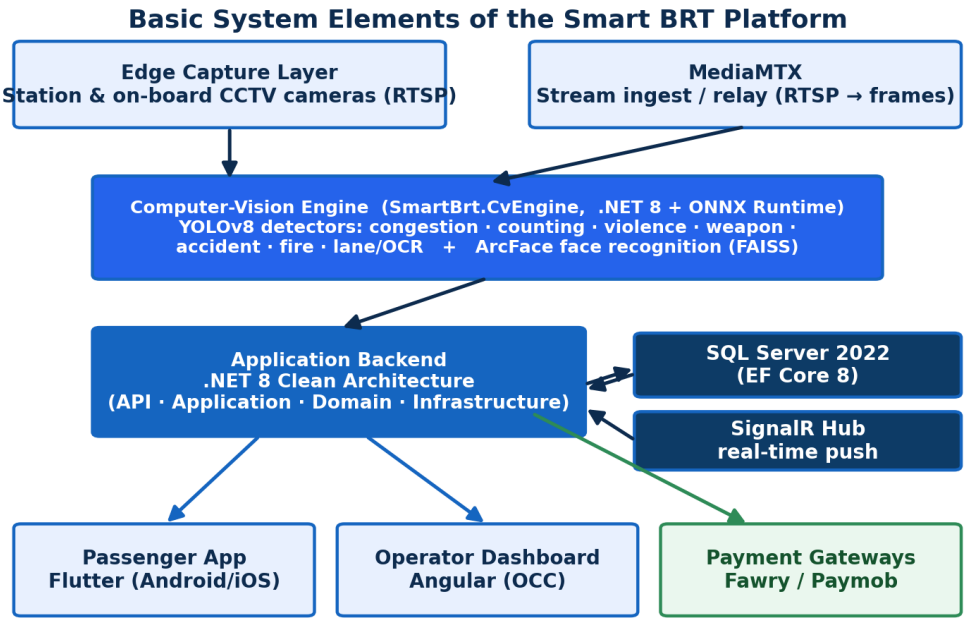


Figure 4: Basic system elements of the Smart BRT platform and the data flow between them

The structural flow of the proposed ecosystem operates across a sequential logical progression:

1. **Station Awareness:** Initiated at the platform level to profile passenger accumulation (**Module I**).
2. **On-Vehicle Telemetry:** Tracks real-time cabin occupancy and boarding dynamics (**Module II**).
3. **Interior Safety & Security:** Monitors passenger behaviors and detects active threats (**Modules III & IV**).
4. **Corridor Enforcement:** Extends intelligence to the transit highway infrastructure for accident prevention, fire safety, and lane protection (**Modules V, VI, & VIII**).
5. **Access Control:** Facilitates touchless, rapid station entry via biometric verification (**Module VII**).

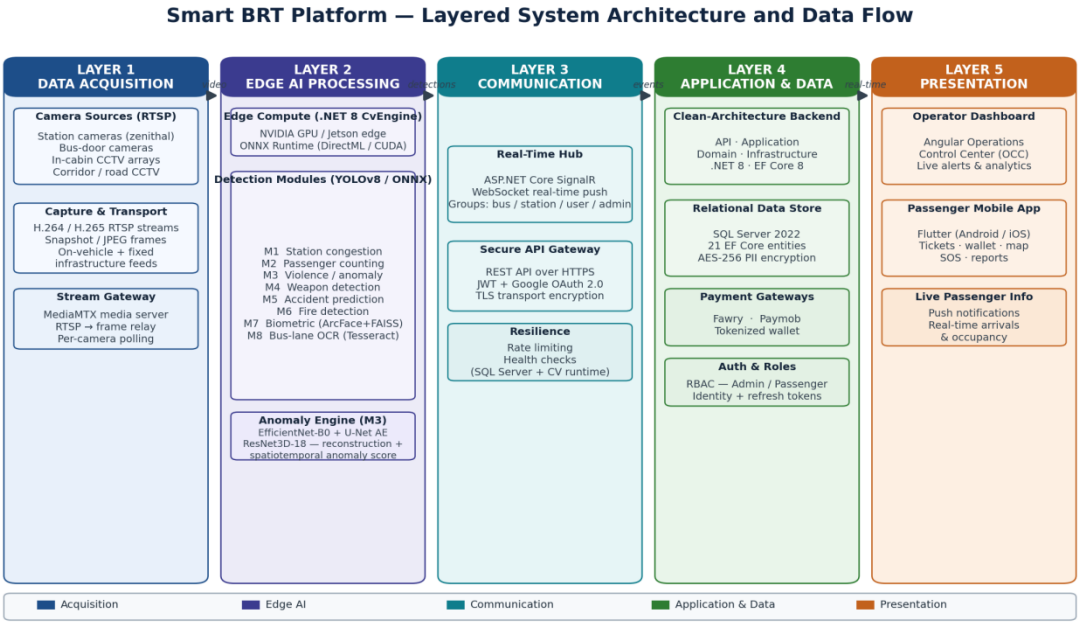


Figure 5: The structural flow of the proposed ecosystem operates across a sequential logical progression

System Development Methodology

The Smart BRT platform was engineered following an iterative, Agile-oriented Software Development Life Cycle (SDLC). Rather than a single linear pass, development proceeded through repeated cycles of requirements analysis, system design, implementation, testing, and deployment, with each iteration refining the previous one. This methodology was essential for a research-driven system in which the eight computer-vision modules, the .NET backend, and the client applications evolved in parallel and had to be integrated and validated incrementally.

The importance of this approach lies in its support for continuous feedback and early risk reduction. The first iterations delivered a minimal working skeleton — camera ingestion, a single detector, and a thin API — which was then progressively extended with additional detectors, the SQL Server data layer, real-time SignalR messaging, and the Flutter and Angular front-ends. Evaluating accuracy and latency at the end of every cycle exposed defects while they were still inexpensive to correct, and the maintenance-and-iteration stage fed the lessons learned back into the requirements of the next cycle. The stages of the adopted life cycle are illustrated in the figure below.

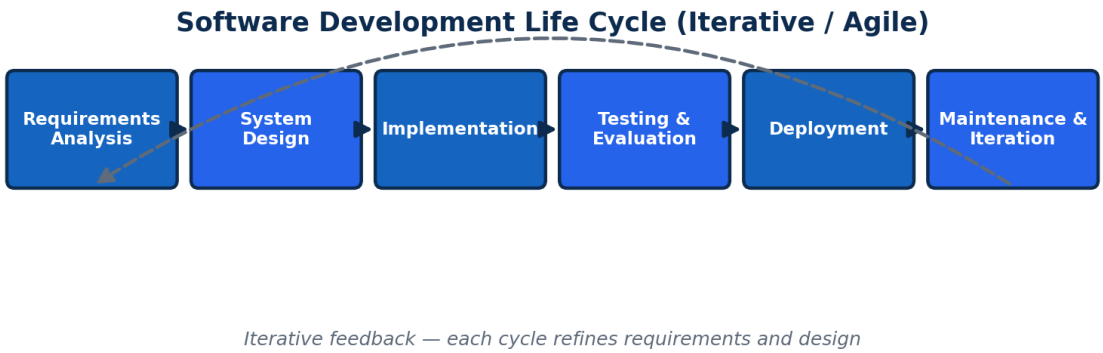


Figure 6: Software Development Life Cycle (iterative / Agile) adopted for the Smart BRT platform

System Requirements

The functional requirements define the specific behaviours, features, and operations that the Smart BRT platform must perform to meet passenger and operator needs. Each requirement is described separately below.

FR-1 — Station congestion monitoring. Continuously estimate platform crowd density from station cameras and feed the result into demand-responsive fleet scheduling.

FR-2 — Bus occupancy counting. Provide real-time, bidirectional passenger counts and seat-occupancy data for each bus from on-board cameras.

FR-3 — Safety incident detection. Automatically detect violence, weapons, fire, and accidents and raise operator alerts with a full evidence chain (video clips, GPS, and timestamps).

FR-4 — Biometric access control. Authenticate passengers at station gates using face recognition, with privacy-preserving storage of biometric templates.

FR-5 — Bus-lane enforcement. Detect unauthorised vehicles in the dedicated bus lane and recognise their license plates for violation recording.

FR-6 — Digital ticketing and passes. Allow passengers to purchase single tickets and season passes through the mobile application.

FR-7 — Wallet and payments. Support wallet top-up and payment through saved cards and mobile wallets, settled through the Fawry and Paymob gateways, with atomic transaction processing.

FR-8 — Live tracking and trip planning. Present passengers with real-time bus locations, estimated arrival times, and trip planning.

FR-9 — Reporting and operations. Let passengers file incident and complaint reports with status tracking, and give operators a dashboard for fleet, alert, and dynamic-scheduling management.

FR-10 — Emergency SOS. Allow a passenger to raise an emergency SOS alert from the mobile application that immediately notifies the operations centre with the passenger's live GPS location and continues to track that location until the alert is resolved.

FR-11 — Account registration and identity verification. Let a passenger create an account and complete identity verification by capturing the national-ID card, which is read automatically by optical character recognition, together with a live selfie subjected to a liveness check; verified accounts may additionally claim concessionary student status.

The non-functional requirements specify the quality attributes the system must satisfy. Each system aspect is described separately below.

Performance. Vision inference runs in real time (tens of milliseconds per frame) on edge GPU hardware, and alerts are delivered to the operations centre with low latency through the SignalR real-time hub.

Security. All traffic is served over HTTPS, and access is governed by role-based authorisation with distinct passenger, operator, and administrator roles. Sensitive data — including saved card details and national-identity numbers — is encrypted at rest using AES-256 through Entity Framework Core value converters and interceptors, and refresh tokens are persisted server-side so that sessions can be revoked. In keeping with the system's privacy-by-design principle, the biometric subsystem retains only the abstract face embedding rather than the source image.

Usability. The mobile and web interfaces are fully localised in Arabic and English with right-to-left layout support, following the BRT brand identity.

Reliability. The platform targets high availability through health checks and durable storage, guaranteeing that financial transactions are never lost.

Maintainability. A clean, layered .NET architecture separates the API, application, domain, and infrastructure concerns from the computer-vision engine, keeping module boundaries clear and testable.

Scalability. The containerised architecture supports horizontal expansion to additional buses, stations, and cameras without structural change.

4.2 Module I: Station Congestion Detection

Purpose and Role in the Pipeline

Before a bus arrives at a designated platform, the Operations Control Center (OCC) must evaluate passenger accumulation. Module I provides continuous, real-time density classification of station platforms. It answers a fundamental operational question: *How crowded is this platform right now?*

The output is expressed as one of three distinct congestion tiers—**Low, Medium, or High**—and is transmitted instantly to the Central Scheduling Engine and passenger-facing mobile applications, making it a critical upstream data source driving dynamic fleet synchronization.

Technical Implementation

The technical realization of this module is organized around the physical sensing layout and the processing logic described below.

- **Camera Layout:** Employs overhead cameras mounted at a zenithal (top-down) angle above the platform corridors.
- **Dual-Class Strategy:** Uses a customized YOLOv8 model fine-tuned to detect two specific classes: full-body *persons* and *human heads*. This dual-class approach compensates for heavy torso-level occlusions in dense crowds by relying on head counts, which remain visually distinct.
- **Frame-by-Frame Processing:** Processes frames independently without temporal tracking, focusing exclusively on instantaneous spatial density.

4.2.3 Decision Flow and Pipeline

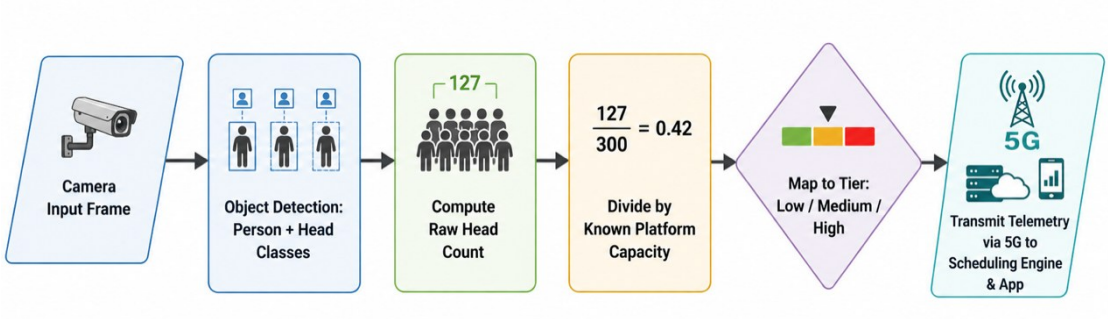


Figure 7 :Operational pipeline and decision flow for Module I (Station Congestion Detection) .

As illustrated in the corresponding figure, the processing pipeline operates in a deterministic linear sequence. The raw frame captured by the zenithal camera is

passed to the fine-tuned YOLOv8 model for head and person detection. To demonstrate the classification logic, a practical scenario is shown where the model detects **127** human heads. This raw count is dynamically divided by the known fixed platform capacity (e.g., **300** passengers), yielding a spatial density ratio of **0.42**. This ratio is then evaluated against the predefined classification thresholds to map the platform to its respective congestion tier (e.g., Medium), before broadcasting the final telemetry packets via the 5G network interface.

Integration with the Scheduling Engine

When Module I issues a *High* congestion alert, the Central Scheduling Engine triggers automated operational adjustments: reducing route headways, dispatching auxiliary shuttle vehicles, or pushing high-density advisories to passenger mobile apps.

4.3 Module II: Real-Time Passenger Counting System

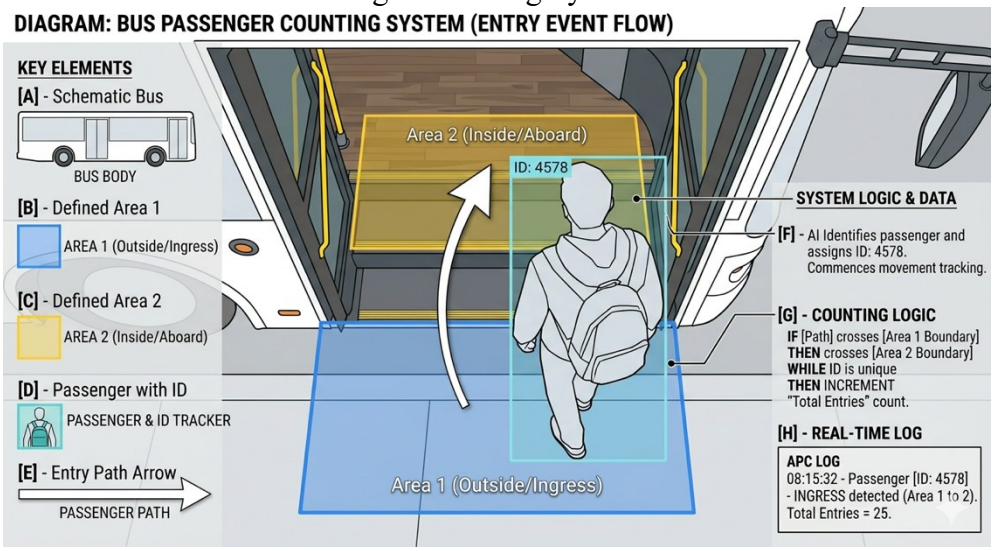


Figure 8 :Diagram of the bus passenger counting system (entry event flow)

Purpose and Role in the Pipeline

While Module I monitors static platform demand, Module II tracks dynamic transit supply at the vehicle boundaries. It records every passenger boarding or alighting, maintains continuous onboard vehicle occupancy telemetry, and computes localized remaining seat availability.

Technical Implementation

This module is implemented through the door-level tracking and directional-counting pipeline detailed below.

- **Tracking Framework:** Door-mounted cameras feed continuous video into a YOLOv8 detector coupled with a Kalman Filter-based motion predictor (**BoT-SORT**) to maintain persistent passenger tracking IDs across sequential frames.
- **Dual-Zone Logic:** Employs two virtual rectangular polygons drawn across the doorway threshold ($Area_1$ and $Area_2$). This spatial transition logic eliminates false counts caused by passengers lingering, pausing, or reversing direction at the bus doors.

Formulation

The real-time available seating capacity is mathematically derived at frame κ using the following relation:

$$\text{Available Seats}_k = C_{\text{total}} - (E_{\text{total}} - X_{\text{total}})$$

Where:

- C_{total} represents the maximum physical seating capacity of the bus asset,
- E_{total} is the cumulative sum of passenger entry events,
- X_{total} is the cumulative sum of passenger exit events.

4.3.4 Decision Flow and Pipeline

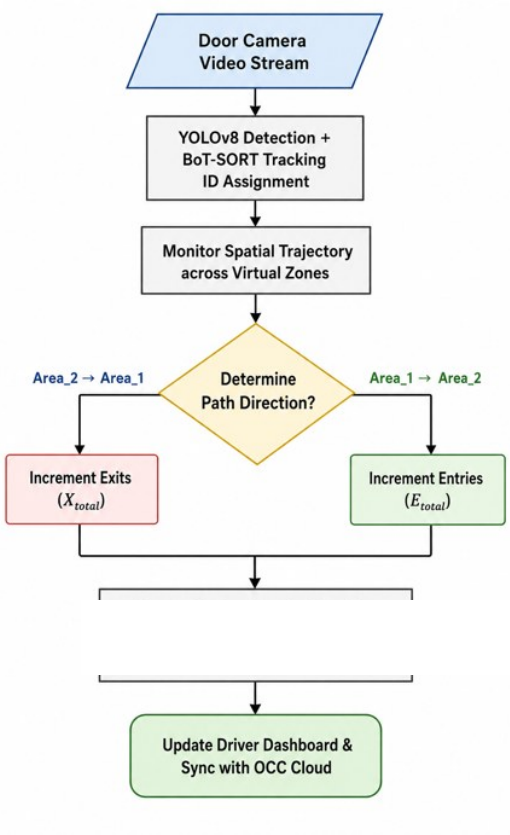


Figure 9: Decision logic and computational pipeline for Module II (Real-Time Passenger Counting System).

The flowchart details the programmatic validation loop where continuous video frames are evaluated. Tracked targets are monitored as they cross the virtual thresholds. Depending on the directional vector of the trajectory, the system dynamically branches to update either the entry or exit registries, subsequently recalculating the net seating availability before pushing telemetry updates to the cloud infrastructure.

Complementary Interior Density System

To validate the door-counter accuracy, a secondary, lightweight frame-by-frame model samples the interior bus cabin cabin. Operating independently of tracking loops, it applies the dual-class Head/Person strategy to estimate interior density, ensuring low-overhead structural redundancy directly on edge devices.

4.4 Module III: Violence and Abnormal Behavior Detection

Purpose and Role in the Pipeline

Module III shifts transit surveillance from passive, human-dependent CCTV auditing to automated, low-latency threat flagging. It continuously scans interior bus cabins and station environments for physical altercations, ensuring immediate passenger safety escalation.

Technical Implementation

The module implements a hybrid architecture combining a deep neural network anomaly estimator with a rule-based temporal validation layer to filter out edge-case false positives (e.g., sudden braking or stumbling).

The anomaly engine scores every 16-frame video clip based on three structural features:

- **Motion Vectors:** Magnitude of pixel-level displacement between consecutive frames.
- **Bounding Box Variance:** Rapid, irregular expansions indicating violent limb extension.
- **Inter-Person Proximity:** Dense bounding box overlaps combined with high-velocity motion indicators.

4.4.3 Decision Flow and Pipeline

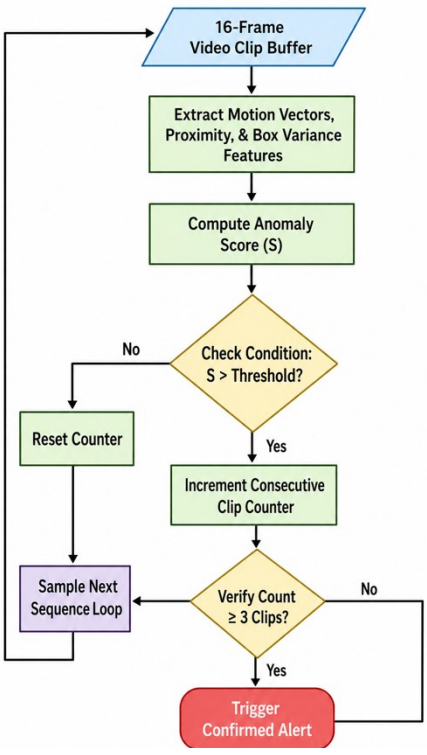


Figure 10 : Decision logic and computational pipeline for Module III (Violence and Abnormal Behavior Detection System)

dual-stage filtering mechanism to eliminate environmental false positives. After features are extracted and scored, the initial threshold filter determines if the anomaly metric S violates safe operating baselines. Rather than immediately triggering an alarm, a downstream temporal validation loop counts consecutive anomalous sequences. A critical alert is only dispatched to the Operations Control Center when the event behavior persists for a minimum threshold of 3 consecutive video segments, ensuring a highly reliable operational deployment.

Alert and Logging

When an anomaly is validated across 3 consecutive clips, an incident package is dispatched to the driver's HUD, local transit authorities, and security services. The package wraps an annotated image capture, specific asset ID, localized GPS coordinates, and a severe behavioral classification tag.

Anomaly-Detection Model: Architecture, Preprocessing, and Inference

Module III departs from the closed-set classifiers used elsewhere in the pipeline and instead employs a hybrid, reconstruction-plus-classification anomaly-detection model. The design rationale is that violent or abnormal passenger behaviour is rare and open-ended: rather than enumerating every abnormal class, the system learns a compact representation of normal behaviour and flags statistically significant departures from it. Two complementary networks are combined, a spatial reconstruction autoencoder and a spatiotemporal three-dimensional convolutional classifier.

The spatial branch is a convolutional autoencoder whose encoder is an ImageNet-pretrained EfficientNet-B0, extracted at five feature levels of 16, 24, 40, 112, and 320 channels, and whose decoder is a U-Net-style stack of transposed-convolution blocks with skip connections that reconstructs the input through a final sigmoid layer, for a total of approximately eight million parameters. Trained exclusively on normal passenger crops, it reconstructs routine appearances with very low error; an anomalous crop, never seen during training, produces a markedly higher mean-squared-error reconstruction loss that is used directly as the anomaly score. A decision threshold of 0.0054, the 97.5th percentile of the normal validation distribution, separates normal from abnormal, after which an exponential moving average with a smoothing factor of 0.3 stabilises the per-track score across frames and suppresses transient spikes.

The spatiotemporal branch is an eighteen-layer three-dimensional residual network (ResNet3D-18) that consumes clips of sixteen consecutive frames and performs a binary normal-versus-abnormal classification, capturing motion-based cues such as sudden lunges, falls, or struggles that a single-frame model cannot perceive. The two branches are deliberately complementary: the autoencoder localises appearance

anomalies on a per-person basis, while the three-dimensional network reasons over short temporal windows.

Preprocessing differs between the two branches. For the autoencoder, each tracked-person crop is converted from BGR to RGB, resized to 320 by 320 pixels, and scaled to the zero-to-one range before being arranged as a channels-first tensor. For the ResNet3D classifier, sixteen frames are resized to 112 by 112, scaled to the zero-to-one range, and standardised with the ImageNet statistics (channel means of 0.485, 0.456, 0.406 and standard deviations of 0.229, 0.224, 0.225); the frames are stacked into a tensor of shape three by sixteen by 112 by 112 and advanced with an eight-frame sliding stride so that successive windows overlap.

The autoencoder is trained for thirty epochs with the Adam optimiser at a learning rate of 0.0001 under a mean-squared-error objective with mixed-precision acceleration, followed by a contrastive fine-tuning stage in which abnormal samples are pushed towards higher reconstruction error, widening the separation between the normal and abnormal score distributions (a measured normal mean of approximately 0.0022 against an abnormal mean of approximately 0.0087). At inference time the subsystem operates as a cascade: a YOLOv8 detector localises each person, a DeepSORT tracker maintains a stable identity across frames, the autoencoder scores each per-person crop, the smoothed score is compared against the threshold, and a normal or abnormal label is emitted together with the bounding box and confidence. For production deployment the spatiotemporal model is exported to the ONNX format and served inside the .NET 8 CvEngine as the BehaviorAnalyzer detector, so that the model trained in PyTorch executes through ONNX Runtime alongside the platform's other detectors.

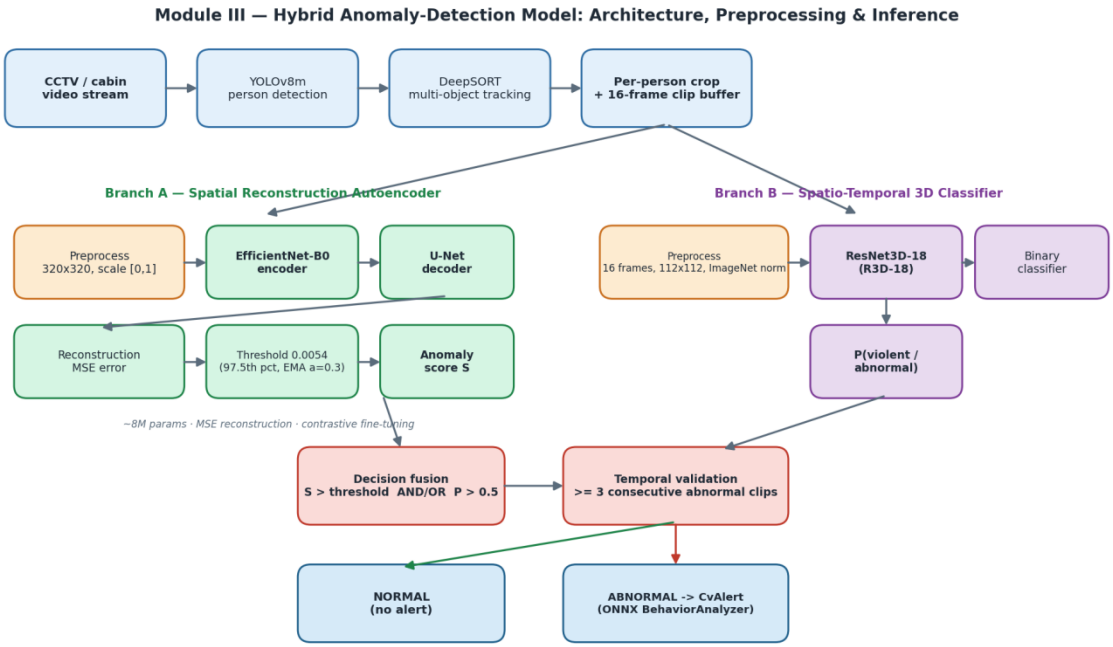


Figure 11: Module III hybrid anomaly-detection model — two-branch architecture (EfficientNet-B0/U-Net reconstruction autoencoder and ResNet3D-18 classifier), preprocessing, and the inference cascade.

4.5 Module IV: Weapon Detection System

Purpose and Role in the Pipeline

Operating in lockstep with behavioral analysis, Module IV secures station access corridors, turnstile channels, and vehicle interiors by deploying a vision pipeline focused on isolating visible weapon configurations before an incident escalates.

Technical Implementation

The detection capability of this module is delivered through the model selection and post-processing configuration described below.

- **Model Selection:** Uses a customized single-stage YOLOv8 architecture optimized for edge latency over heavier two-stage alternatives (such as Faster R-CNN), achieving an inference speed of ≈45 ms per frame ($> 22\ FPS$).
- **Dataset Target:** Trained across five critical threat classes: *Knife, Handgun, Scissors, Pliers,* and *Explosives*.
- **Architectural Mechanics:** The backbone uses CSPDarknet with C2f modules for feature extraction. The PANet neck handles multi-scale context aggregation, and the decoupled detection head isolates classification tasks from bounding box regression to distinguish visually similar objects (e.g., separating a knife from everyday tools).

4.5.3 Decision Flow and Pipeline

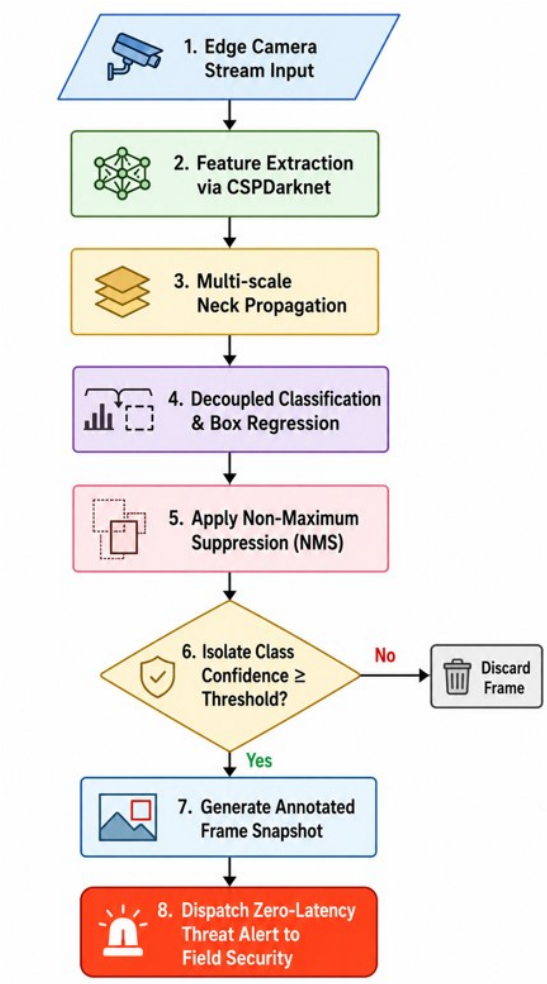


Figure 12:Deep learning architecture and post-processing pipeline for Module IV (Weapon

The flowchart in the corresponding figure details the single-stage convolutional execution pipeline optimized for real-time threat detection. Raw video streams are sequentially propagated through the CSPDarknet backbone for multi-scale feature extraction and the PANet neck for spatial context aggregation. Once the decoupled head generates bounding box candidates, Non-Maximum Suppression (NMS) is applied to prune redundant overlapping boxes. Finally, a deterministic confidence filter acts as a guardrail; instances exceeding the calibrated target probability threshold trigger an immediate system state escalation, dispatching annotated evidence snapshots to localized safety personnel without cloud routing overhead.

4.6 Module V: Accident Detection and Prediction System

Purpose and Role in the Pipeline

Module V expands system situational awareness from passenger hubs to the active transit corridor itself. By analyzing vehicle vector trajectories on the road network, it shifts incident management from a reactive post-crash model to a predictive collision-warning framework.

Technical Implementation

The analytical pipeline evaluates vehicle kinematics sequentially in under 30 ms:

1. **Detection:** YOLOv8n identifies all vehicle assets within the roadway frame boundaries.
2. **Trajectory Buffering:** Tracks vehicle vectors over a rolling 30-frame temporal buffer.
3. **Kinematic Metrics:** For every vehicle pair, the system calculates the Euclidean distance (D_E) between centroids and the approach speed (Δd), defined as the mean rate of change of distance across the last 6 frames.

4.6.3 Decision Flow and Pipeline

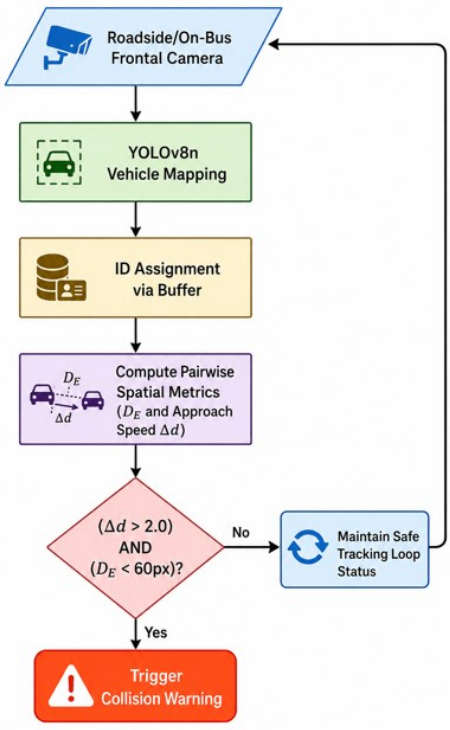


Figure 13: Kinematic hazard evaluation and predictive decision flow for Module V (Accident Prediction System).

The computational logic illustrated in the corresponding figure delineates the localized tracking and risk-assessment architecture. The localized deployment evaluates consecutive video frames via a lightweight

YOLOv8n network to map spatial coordinates. By analyzing the temporal buffer, the spatial logic continuously computes pairwise metrics, specifically tracking the Euclidean distance (D_E) between bounding box centroids and the rate of approach acceleration (Δd). If a vehicle breaches the dual-parameter safety thresholds—defined by high-velocity convergence within critical pixel proximity—the system bypasses nominal tracking states to execute an instantaneous edge-level collision escalation protocol.

Incident Classification Tiers

Verified telemetry signals are categorized into four distinct operational layers within the cloud backend:

- **High Probability Collision:** Confirmed centroid intersection or critical approach parameters.
- **Near-Miss Event:** Severe proximity convergence that successfully resolves without a crash impact.
- **Obstacle Alert:** Detection of stationary or slowly drifting hazards inside the active lane.
- **System False Alert:** Suppressed instances caused by environmental vibrations or lighting transitions.

4.7 Module VI: Fire Detection System

Purpose and Role in the Pipeline

Module VI provides fire safety across the BRT corridor, station boundaries, and engine compartments. It provides prompt optical flame identification at the point of ignition, bypassing the accumulation lag inherent in traditional smoke alarms.

Technical Implementation

The implementation of this module rests on the model configuration and operational parameters outlined below.

- **Model Configuration:** Employs a specialized YOLOv8n (Nano) framework trained exclusively on the *Flame* class.
- **False Positive Mitigation:** The model deliberately omits smoke classification. Smoke exhibits high visual ambiguity in outdoor environments due to vehicle

exhaust, dust, and industrial emissions. Flame features distinct, high-saturation luminous signatures and localized flickering dynamics that can be separated from brake lights or headlight reflections.

- **Spatial Filters:** Inputs are normalized to 640×480 pixels. Detections must satisfy a confidence threshold ($\tau \geq 0.50$) and a spatial filter requiring a minimum bounding box area (A_{min} pixels) to eliminate distant, non-threatening light sources.

4.7.3 Decision Flow and Pipeline

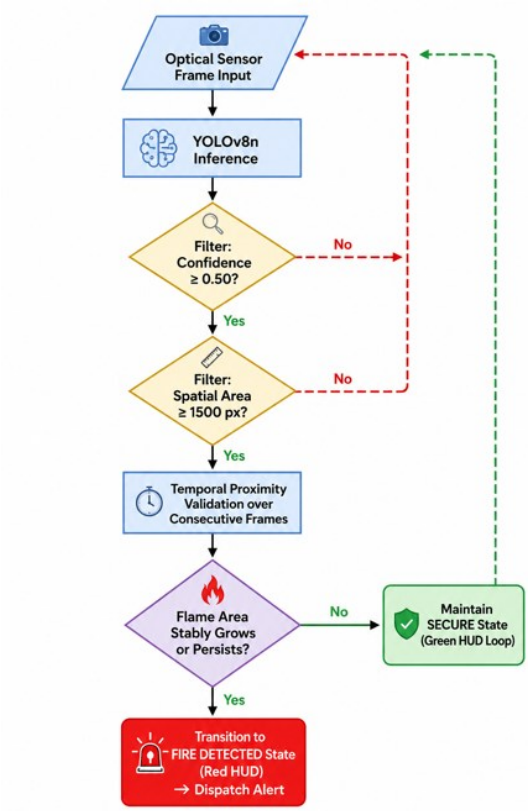


Figure 14: Deterministic verification pipeline and spatial-temporal filtering logic for Module VI (Fire Detection System).

the corresponding figure illustrates the layered safeguarding mechanisms implemented on the edge inference loop to eliminate environmental false positives in highway corridors. The pipeline operates as a deterministic cascaded filter: detections generated by the YOLOv8n network are subjected to a specific target probability threshold (≥ 0.50) and a spatial area filter (≥ 1500 pixels) to prune distant light sources or non-threatening small reflections. Detections passing these visual criteria undergo a localized temporal validation loop. Only when a detected fire object exhibits stable spatial persistence or growth across consecutive frames does the

system state transition to a critical alarm escalation, bypassing the nominal "SECURE" state loop.

4.8 Module VII: Biometric Access Control via Facial Recognition Gates

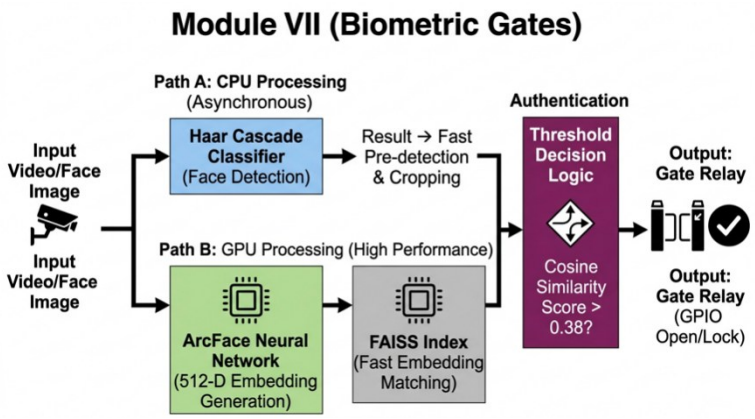


Figure 15 :Module VII (Biometric Gates)

Purpose and Role in the Pipeline

Module VII modernizes station entry management by replacing physical tokens, magnetic cards, and QR code scans with seamless biometric validation at entry gates. This eliminates ticketing queues, prevents fare evasion, and automates passenger check-in.

Technical Implementation

The edge system runs a two-tier parallel processing stack:

- **Tier 1 (CPU Thread):** A high-speed Haar Cascade classifier continuously scans wide-angle ingress fields to locate human faces at a distance, initiating the matching process early to reduce entry latency.
- **Tier 2 (GPU Thread):** Located facial crops are passed asynchronously to an ArcFace neural network (*buffalo_l* backbone), mapping facial features to a 512-dimensional vector embedding space.

4.8.3 Decision Flow and Pipeline

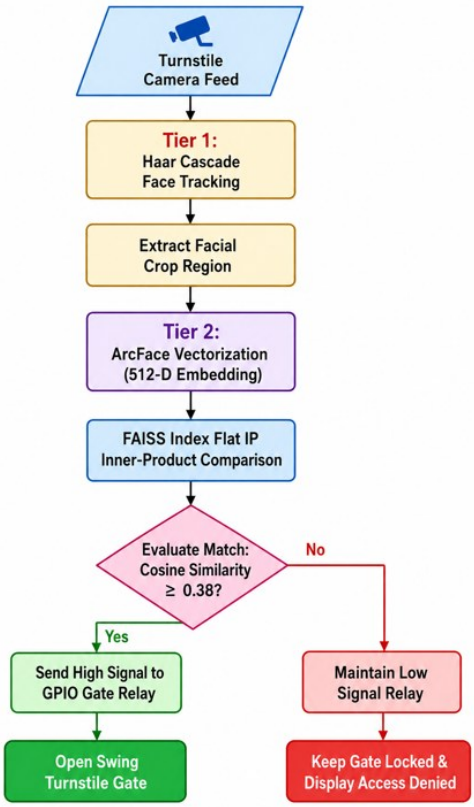


Figure 16: Multi-tier hierarchical verification and physical hardware control pipeline for Module VII (Biometric Access Gates).

The structural workflow shown in the corresponding figure details the decoupled authentication pipeline optimized for low-latency physical access control. The execution logic is segmented into a two-tier hierarchy: Tier 1 utilizes a lightweight Haar Cascade algorithm on the edge host to continuously monitor the camera stream and isolate candidate facial crop regions. Once a region is validated, Tier 2 runs an optimized ArcFace network to synthesize a high-dimensional 512-D feature vector. This embedding is computed against local registries via a FAISS Index using Inner-Product vector search. If the similarity metric satisfies the mathematical boundary condition (≥ 0.38), a high-level digital signal is written to the physical micro-controller GPIO pins to energize the relay and trigger the opening event; otherwise, the hardware remains locked by default.

Privacy Architecture

To ensure a robust privacy-by-design posture, raw facial images are immediately purged from volatile memory once transformed into 512-dimensional mathematical representations. Only non-invertible vector hashes are stored or cross-referenced, ensuring data security even in the event of an infrastructure breach.

4.9 Module VIII: Bus Lane Monitoring and Violation Detection System

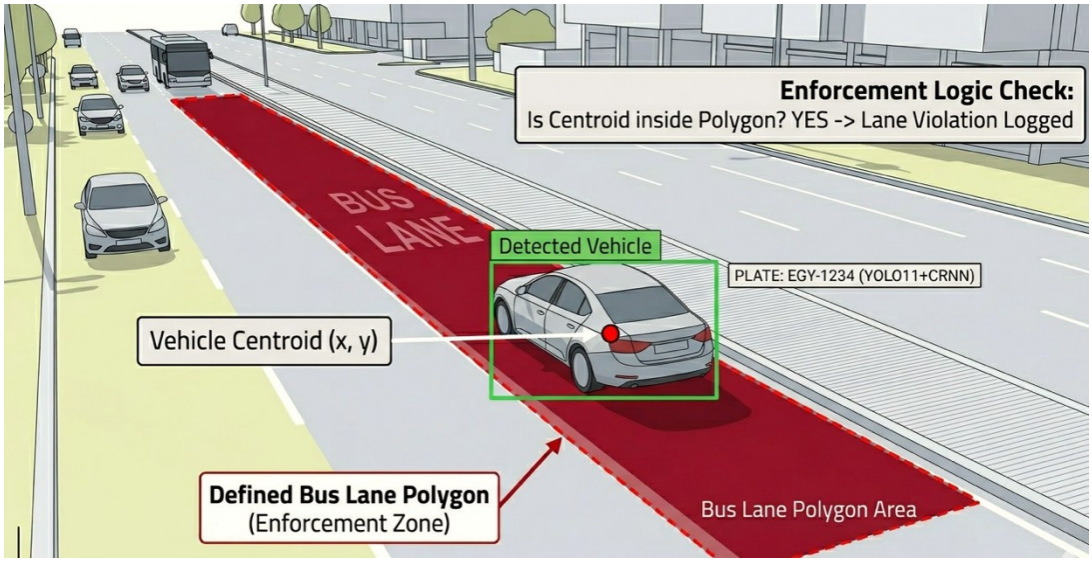


Figure 17: Bus Lane Monitoring and Violation Detection System

Purpose and Role in the Pipeline

Module VIII enforces infrastructure integrity by protecting the dedicated BRT lane. When unauthorized private vehicles encroach on the BRT corridor, transit reliability degrades, throwing off downstream fleet scheduling decisions. This module provides automated legal enforcement to protect lane exclusivity.

Technical Implementation

The enforcement engine operates as a two-stage sequential computer vision architecture:

- **Stage 1 (Vehicle & Plate Localization):** A YOLOv8 model detects vehicles within a defined BRT lane polygon. A minimum temporal dwell-time filter ($t_{min} = 0.5\ seconds$) ensures vehicles briefly merging at intersections do not trigger erroneous violations.
- **Stage 2 (Optical Character Recognition):** Cropped license plate regions are passed to the Tesseract OCR engine to read alphanumeric characters. Extracted text is validated against standard Egyptian plate formats, and authorized vehicle databases are cross-referenced to filter out emergency assets and permitted transit fleets.

4.9.3 Decision Flow and Pipeline

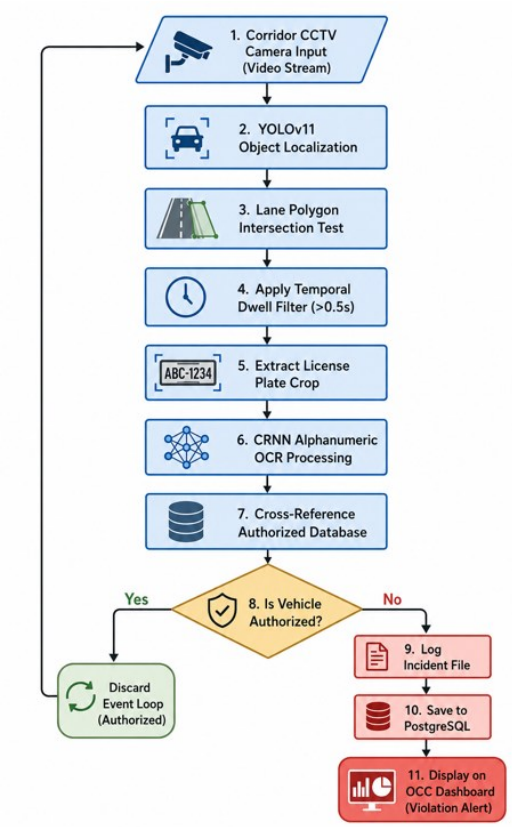


Figure 18: Multi-stage enforcement pipeline and alphanumeric OCR logic for Module VIII (Bus Lane Monitoring System).

The spatial-temporal enforcement workflow illustrated in the corresponding figure describes the sequential verification logic designed to detect and log transit lane violations. The pipeline processes high-angle corridor video streams using a YOLOv8 detector to identify vehicle bounding boxes, which are instantly verified via a polygonal intersection test mapping the enforced bus lane coordinates. To prevent transient false tracking, a temporal dwell filter checks if the target remains inside the polygon boundaries for more than 0.5 seconds. Once a violation is mathematically established, the region-of-interest containing the license plate is cropped and passed to the Tesseract OCR engine for alphanumeric character recognition. The extracted string is cross-referenced against the central authorized transit registry; unauthorized vehicles trigger an automated incident logging sequence that flushes structured violation metadata to the central SQL Server database and pushes real-time alert frames to the OCC dashboard.

Violation Recording and Security

Confirmed infractions are committed to the central SQL Server database. Each entry stores the extracted character text string, a localized timestamp, specific camera metadata, corridor coordinates, and an annotated high-resolution reference image. All

transport streams are secured using TLS encryption protocols, with access strictly restricted via role-based authentication logs.

4.10 System Use Case Analysis

System Overview and Scope

The Smart BRT (Bus Rapid Transit) platform constitutes an enterprise-grade, intelligent public transportation management ecosystem integrating edge-deployed Computer Vision (CV) inference, distributed real-time telemetry ingestion, and data-driven scheduling orchestration. The primary objective of the platform is to automate transit enforcement, optimize commuter throughput, and ensure structural security across corridors and terminal hubs.

This section presents the formal Use Case Analysis detailing the functional boundaries of the system. By defining all interacting actors (human operators, hardware assets, and autonomous processes), this analysis maps the operational workflows and transaction scopes of the software architecture in compliance with the Unified Modeling Language (UML 2.x) and IEEE 830 requirements documentation standards.

System Actors

System actors are classified as primary human users, autonomous computing agents, or external downstream enterprise platforms that interact across the system boundary.

Table 3: Smart BRT System Actor Specifications

Actor Name	Type	Interaction Protocol / Gateway	Architectural Description
Passenger	Primary (Human)	Mobile App REST APIs / SignalR WebSockets	The end-user of the mobile client. Authenticates to manage profiles, purchase tickets, evaluate route geometries, track assets, execute digital wallet top-ups, and trigger emergency SOS signals.
Admin Operator	Primary (Human)	Angular Web Dashboard / REST APIs	Advanced system administrator operating within the central Operations Control Center (OCC). Monitors real-time CV anomaly

			alerts, manages camera asset configurations, triages passenger issue reports, applies scheduling overrides, and reviews system analytics.
CV Runtime	Autonomous Agent	gRPC Channel / Edge Broker Pipeline	High-throughput background inference engine processing optical streams from stations and vehicle cabins. Automatically extracts behavioral characteristics, detects physical threats, calculates crowd metrics, and verifies facial identities.
Telemetry Hardware	Autonomous Agent	TCP Socket Gateway	On-bus IoT tracking devices and GPS units that continuously transmit geo-spatial coordinates, velocity vectors, and mechanical occupancy metrics over wireless transport layers.
Scheduling Engine	Autonomous Agent	Internal Background Worker Thread	An optimization service running on the backend infrastructure that continuously evaluates route load thresholds and automatically triggers dynamic transit scheduling recommendations.
Payment Gateway	External System	Secure HTTPS REST (e.g., Fawry API / Webhooks)	A PCI-DSS compliant third-party payment aggregator managing monetary transactions, digital wallet top-ups, and financial auditing logs.
Notification Service	External System	SignalR real-time push	External notification carrier responsible for routing real-time

			transaction alerts, scheduling delays, and high-priority emergency pushes to client terminals.
National ID API	External System	Government REST Web Service Gateway	Official external state identity lookup registry utilized during passenger onboarding to execute Know Your Customer (KYC) compliance checks.
GPS / Map Provider	External System	Google Maps / Mapbox Vector Tile API	External spatial data provider used to render interactive mapping surfaces, calculate routing distances, and display live estimated times of arrival (ETA).

High-Level Use Case Model

To preserve systemic hierarchy and maintain low visual complexity during software architectural reviews, the functional surface area of the Smart BRT Platform is split into a macro-level enterprise application architecture and a micro-level commuter service sub-boundary.

Macro Platform Enterprise Architecture Model

The master system scope mapping all system actors, behavioral dependencies, execution stereotypes, and foundational packages within the container boundary is modeled below.

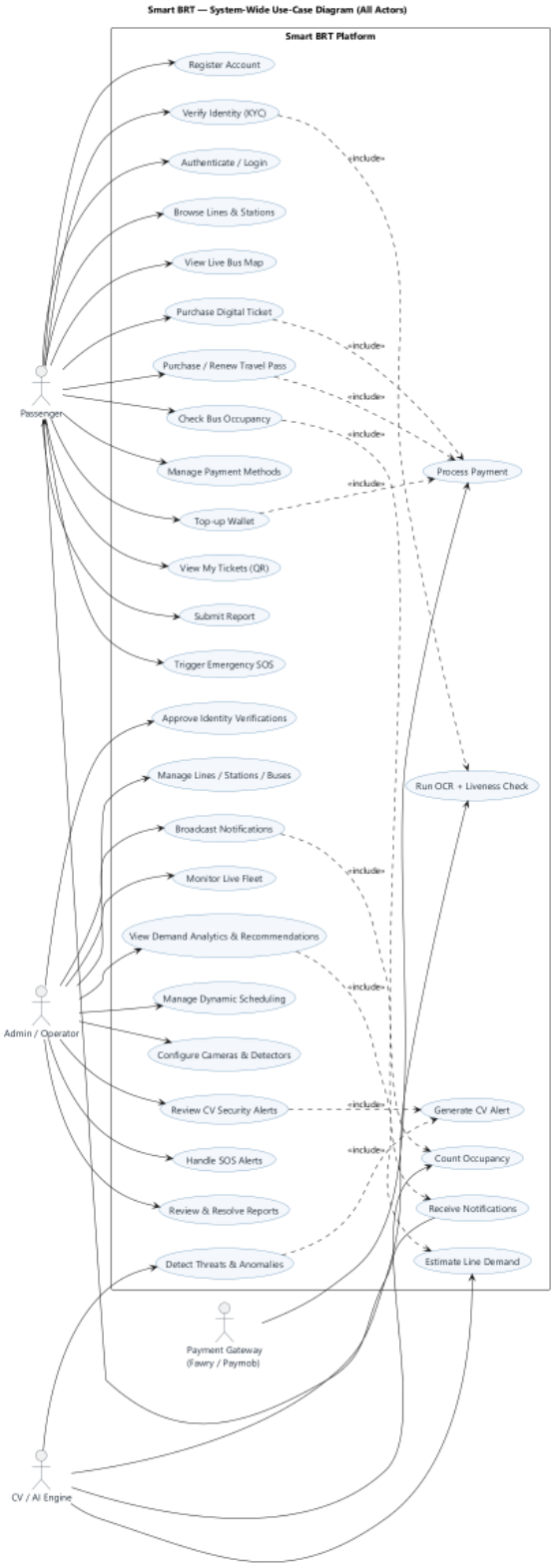


Figure 19: Unified Use-Case Diagram for the Smart BRT Platform

- **Description and Architectural Analysis:** the corresponding figure establishes the global enterprise software boundaries of the platform. It models a clean structural bifurcation within a single master application container, delineating the **Admin Operations** package from the **Passenger Services** package. The diagram visually highlights that while the Admin Operator and Passenger possess entirely isolated, non-overlapping interface privileges, their transactional operations converge on shared cross-cutting infrastructure services located outside the standard packages.
- Crucially, the diagram models complex structural stereotypes: financial transactions (Purchase Ticket, Wallet Top-Up, Buy Travel Pass) and critical safety mechanisms (Trigger SOS Alert) carry explicit, unidirectional <<include>> dependencies pointing to the shared Process Payment and Authenticate use cases. Furthermore, a behavioral variation point is mapped via an <<extend>> relationship running from View Ticket History to Purchase Ticket, documenting the downstream workflow extension generated immediately after a successful transaction lifecycle completion.

Isolated Passenger Services Subsystem Model

- **Description and Architectural Analysis:** the corresponding figure (isolates the behavioral layout of the commuter application interface. By removing administrative layers, it maps the precise execution requirements for the Passenger primary actor across 11 core use cases. This subsystem view details how daily commuter functions rely on shared security abstractions.
- The structural modeling demonstrates that data manipulation actions require verified authorization states; the <<include>> arrows enforce a programmatic dependency showing that a user cannot successfully finalize a ticket acquisition, modify an active transit pass subscription, top up electronic ledger balances, or dispatch an emergency SOS signal without forcefully running through the parent Authenticate state machine. This architectural constraint protects the backend API gateway against unauthorized access and unauthenticated request spam.

Master Use Case Registry

The system's functional surface area is mapped across 23 core use cases. Table 4 establishes the unique identifiers, actor mapping, and structural state guardrails for each lifecycle event.

Table 4: Master System Use Case Registry

ID	Use Case Name	Primary Actor(s)	Pre-conditions	Post-conditions
UC-01	Authenticate User	Passenger / Admin	Valid client registration credentials exist in the identity store.	Secure cryptographically signed JWT Access and Refresh tokens are issued.
UC-02	Register & Verify Identity (KYC)	Passenger	Valid civil National ID documents are physically available.	Account profile initialized; identity status marked as Pending Review.
UC-03	Manage Profile	Passenger	Session state verified via valid active Bearer token.	Database record updated with the mutated passenger profile fields.
UC-04	Browse Stations & Plan Trip	Passenger	Network layer connectivity active (Public Access).	Optimized multi-hop spatial route data and calculated fare matrix rendered.
UC-05	Track Bus on Live Map	Passenger	User device has initialized a location map viewer wrapper.	Continuous SignalR stream renders real-time vehicle spatial marker updates.
UC-06	View Bus Occupancy	Passenger	Targeted bus asset reference is active within the client view.	Live capacity tier indicators displayed on the passenger HUD interface.

UC-07	Purchase Digital Ticket	Passenger	Authenticated; client wallet balance exceeds targeted fare cost.	Financial ledger debited; encrypted unique boarding QR record generated.
UC-08	Buy Periodic Travel Pass	Passenger	Authenticated; ledger balance satisfies selected subscription rule.	Subscription sub-record activated within the passenger data table.
UC-09	View Ticket History	Passenger	Session state verified via valid active Bearer token.	Relational database historical ticket logs rendered on client interface.
UC-10	Execute Wallet Top-Up	Passenger	Authenticated client state; active payment instrument details provided.	Third-party validation confirmed; wallet balance incremented.
UC-11	Trigger Emergency SOS Alert	Passenger	Active authenticated session; GPS location hardware initialized.	Incident committed to persistence layer; real-time alerts pushed to OCC.
UC-12	Submit System Issue Report	Passenger	Session state verified via valid active Bearer token.	Unresolved structural report ticket injected into the admin triage table.
UC-13	View System Notifications	Passenger	Session state verified via valid active Bearer token.	Unread notification list pulled and rendered on the client application.

UC-14	View Live System Dashboard	Admin Operator	Administrator authenticated via valid privileged security role.	Real-time transit KPIs, system health vectors, and active maps loaded.
UC-15	Manage Camera Configurations	Admin Operator	Administrative session active with operational infrastructure roles.	Persistent storage schema execution of Camera CRUD transactions completed.
UC-16	Review Automated CV Security Alerts	Admin Operator	Computer Vision inference runtime detects active hazard events.	Hazard alert status updated, logged, and tracked within the triage history.
UC-17	Monitor and Escalate SOS Alerts	Admin Operator	Active emergency SOS signal is broadcast by an asset or user.	Critical response dispatch logged; state flag mutated to Resolved.
UC-18	Triage Inbound Issue Reports	Admin Operator	Unresolved user reports populate the system administrative queue.	Tickets processed; notification update dispatched to the reporting client.
UC-19	System User Administration	Admin Operator	User has master root administrative privileges allocated.	Target user account permissions, suspensions, or profiles modified.
UC-20	Manage Routes and Stations	Admin Operator	Administrative session active with structural engineering roles.	GIS spatial coordinates, line data, and ordered stops collection updated.

UC-21	Manage Fare Matrix and Pricing	Admin Operator	Administrative session active with financial controller roles.	Dynamic pricing models and baseline transit fare policies updated.
UC-22	Manage Fleet Dynamic Scheduling	Admin Operator	Privileged administrative session active within the OCC.	Dynamic dispatch recommendation accepted or manual override applied.
UC-23	Review System Auditing & Analytics	Admin Operator	Valid root administrative session active.	Tabular reports, financial summaries, and machine logs generated.

Detailed Use Case Specifications

To maintain total compliance with the ISO/IEC/IEEE 29148 Software Requirements Specification (SRS) guidelines, critical use cases representing human, technical, and automated agents are documented below in full tabular detail.

Passenger Specifications

Table 5:Use Case Specification - UC-07: Purchase Digital Ticket

Specification Field	**Technical & Structural Description**
Use Case Name	Purchase Digital Ticket
Unique Identifier	UC-07
Primary Actor	Passenger
Secondary / Supporting Systems	Internal Ledger DB, Symmetric Encryption Module
Pre-conditions	1. The user must possess an active, authenticated session verified via a valid JWT Bearer token. 2. The user's internal electronic wallet balance must exceed the calculated trip cost.
Post-conditions	1. The internal electronic ledger is debited, and an immutable log entry is appended. 2. A secure, high-entropy cryptographically verifiable boarding QR ticket is issued.

Main Success Flow	1. Passenger selects the origin station, destination station, and dynamic departure parameters. 2. The system calculates the absolute fare cost based on transit zone hops and cross-references user balances. 3. The backend instantiates an atomic transaction block and applies a **row-level database lock** on the user's ledger row to prevent multi-device race conditions (double-spending vulnerability). 4. Passenger confirms the transaction by submitting the purchase request payload. 5. The system deducts the calculated amount and inserts a transaction audit record into the ledger schema. 6. The backend triggers the encryption provider subroutine to generate a signed data string containing Ticket_ID, User_ID, and expiration timestamps. 7. The signed string is compiled into a high-density QR code matrix and dispatched to the client UI container, while simultaneously releasing the database row-level lock.
Alternative & Exception Flows	**Exception Flow A - Insufficient Ledger Balance:** 1. In Step 2, if the system balance check evaluates as lower than the calculated fare, the transaction execution halts. 2. The system rolls back the context, releases the row lock, and dispatches an HTTP 422 Unprocessable Entity payload to the mobile UI. 3. The mobile client intercepts the error block and pops up a balance-low notification directing the user to execute a wallet top-up (UC-10).

Table 6:Use Case Specification - UC-11: Trigger Emergency SOS Alert

Specification Field	**Technical & Structural Description**
Use Case Name	Trigger Emergency SOS Alert
Unique Identifier	UC-11

Primary Actor	Passenger
Secondary / Supporting Systems	SignalR Real-Time Hub Server
Pre-conditions	1. The mobile application container must possess active device hardware location permissions (GPS).
Post-conditions	1. An active crisis record is instantiated within the persistent database layer. 2. A zero-latency visual and auditory flashing notification is broadcasted across all active OCC admin screens.
Main Success Flow	1. Passenger initiates a long-press action on the UI SOS trigger for exactly two seconds (preventing accidental pocket activation). 2. The application bypasses standard low-priority API queues, fetching high-accuracy device coordinates (Latitude, Longitude). 3. The client executes an optimized, low-overhead HTTP POST request targeting the emergency gateway endpoint /api/v1/safety/sos. 4. The backend maps the coordinates using geospatial distance calculations to resolve the closest physical station corridor or active vehicle ID. 5. An immutable incident record is committed to the database with a default state of Open/Unresolved. 6. The safety module pushes the emergency payload to the active **SignalR alert hub** group, instantly popping up a flashing audio-visual terminal override on the Admin Operator dashboard.
Alternative & Exception Flows	**Alternative Flow A - Complete Cellular Data Loss:** 1. If the client detects a failure to connect to the HTTP gateway due to cell dead-zones, it fallback-routes via cellular telephony. 2. The application packages the spatial coordinate parameters into an encrypted, highly compressed SMS string format. 3. The application forces the mobile device modem to route the raw text packet directly to the municipal transit authority emergency short-code.

Admin & Automated Agent Specifications

Table 7: Use Case Specification - UC-16: Review Automated CV Security Alerts

Specification Field	Technical & Structural Description
Use Case Name	Review Automated CV Security Alerts
Unique Identifier	UC-16
Primary Actor	Admin Operator
Secondary / Supporting Systems	CV Runtime Edge Inference Engine, Central Media Broker Storage
Pre-conditions	1. The edge-deployed Computer Vision (CV) model detects a threat (e.g., weapon drawn, passenger assault) with a confidence factor scoring $\geq 75\%$.
Post-conditions	1. The alert state is mutated based on human operator validation (either escalated to dispatch or flagged as false positive).
Main Success Flow	1. The local CV Runtime captures a target anomaly event, flags a static evidence frame snapshot, and compiles metadata (Camera ID, Station Zone, Threat Class). 2. The edge device streams the binary payload over a low-latency gRPC channel directly to the central backend system. 3. The backend logs the event under the AutomatedAlerts schema and pushes a high-priority interrupt signal to the active Admin Operator web dashboard. 4. The Admin Operator selects the flashing alert node, displaying the telemetry data alongside the static frame capture and the live camera stream. 5. The Admin Operator visually audits the stream and clicks "Verify and Confirm Anomaly". 6. The system mutates the alert state database index to Verified and triggers an escalation workflow that alerts physical terminal security teams.
Alternative & Exception Flows	**Alternative Flow A - False Positive Flagging:**

	1. In Step 5, if the human operator determines the model misidentified an object (e.g., a long tool misidentified as a weapon), they select "Dismiss as False Positive". 2. The system changes the database record state flag to FalsePositive, updates the logging table, and routes the anonymized image to the model training pipeline folder for future active-learning optimization cycles.

Table 8: Use Case Specification - UC-22: Manage Fleet Dynamic Scheduling

Specification Field	**Technical & Structural Description**
Use Case Name	Manage Fleet Dynamic Scheduling
Unique Identifier	UC-22
Primary Actor	Admin Operator
Secondary / Supporting Systems	Autonomous Scheduling Engine, IoT Bus Display Gateway
Pre-conditions	1. Sudden passenger surge detected via CV station crowd-density estimation, or a vehicle breakdown occurs on an active corridor line.
Post-conditions	1. Modified dispatch arrays are written to the persistence cluster, mutating vehicle route tables and changing the public ETA calculations.
Main Success Flow	1. The background Autonomous Scheduling Engine continuously executes route optimization algorithms on the current transit matrix. 2. The engine generates a recommended optimization vector (e.g., "Re-route Bus 204 from Low-Demand Line A to High-Density Corridor Line B for the next 45 minutes"). 3. The recommendation is pushed as an interactive pop-up card to the Admin Operator's dispatch dashboard panel. 4. The Admin Operator audits the routing impact graph via an administrative simulator overlay.

	5. The operator clicks "Approve and Implement Fleet Optimization". 6. The system commits the new timetables to the database, publishes the updated route instructions to the vehicle's onboard driver console via an **real-time message packet**, and dynamically changes the ETA readouts on passenger mobile client displays.

Critical Technical Workflow Sequence Diagrams

To provide thorough validation of data flows and platform components during architectural design reviews, the sequence of messages for high-priority workflows is modeled chronologically below.

Sequence Diagram — Emergency SOS Communication Lifecycle (realising UC-11)

This diagram maps the low-latency message propagation path designed to ensure emergency signals immediately bypass standard network congestion layers to alert the

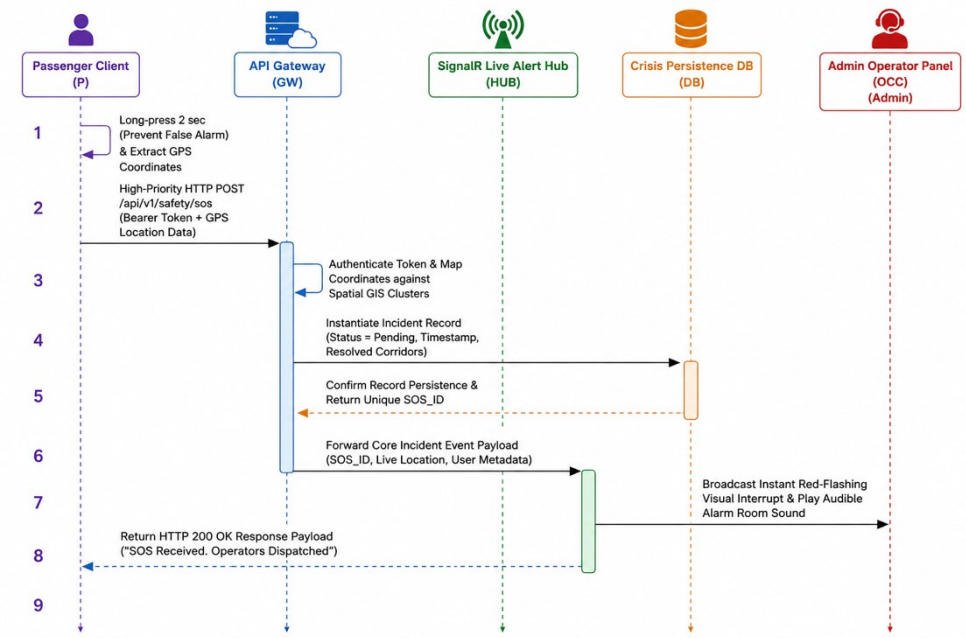


Figure 20: Technical UML Sequence Diagram mapping the low-latency messaging lifecycle of UC-11 (Trigger Emergency SOS Alert).

OCC.

Sequence Diagram — Automated Edge CV Threat Ingestion & Review (realising UC-16)

This diagram maps the architectural integration showing how local edge camera inference engines communicate threat intelligence up to the central platform stack for verification.

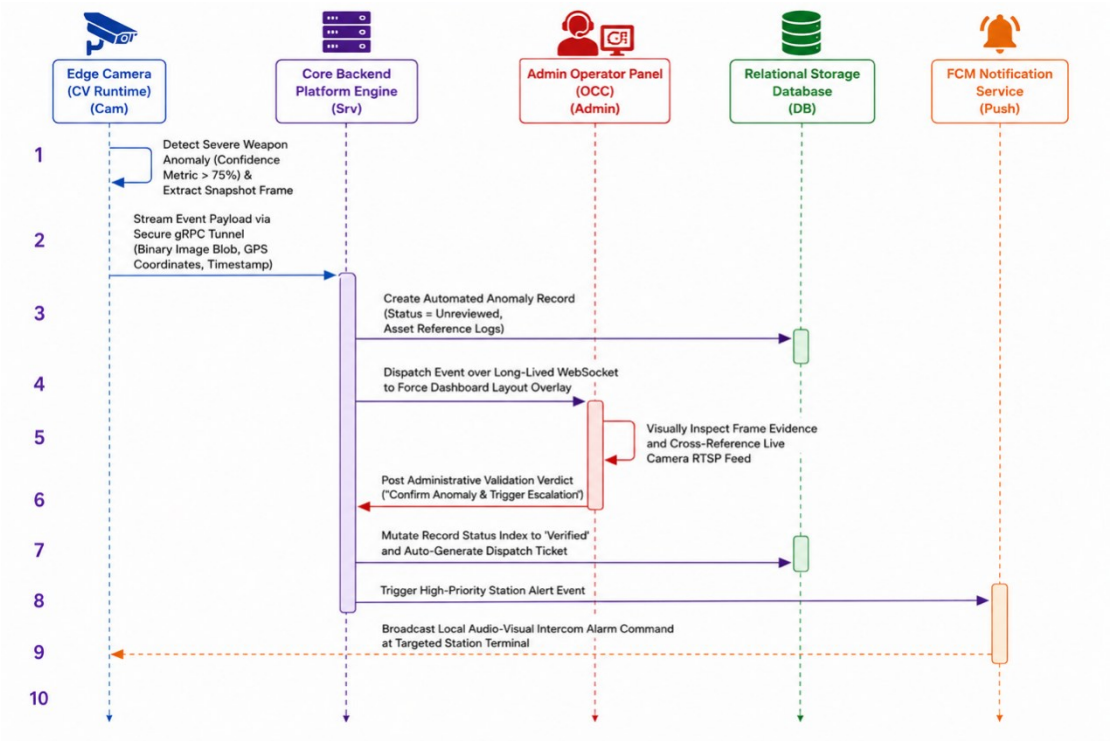


Figure 21: Technical UML Sequence Diagram mapping the automated threat detection and human-in-the-loop validation lifecycle of UC-16 (Review Automated CV Security Alerts).

Admin Operator Use Cases: Behavioral Specification

This chapter presents the complete behavioral specification for each Admin Operator use case executed via the Angular-based administrative dashboard. The admin actor exercises elevated system privileges, enabling real-time monitoring, security incident management, fleet scheduling oversight, and user lifecycle administration. Each use case is specified with its triggering context, preconditions, postconditions, and principal interaction sequence.

- **Actor:** Admin Operator
- **Pre-conditions:** Authenticated with admin role.
- **Post-conditions:** Real-time KPIs and status indicators displayed.
- **Description:** The admin logs into the dashboard and sees a summary of the platform state: active buses, total passengers, open SOS alerts, pending reports,

CV alert counts, line demand levels, and camera health. All data is updated in real-time via SignalR subscriptions.

UC-15 — Manage Cameras

- **Actor:** Admin Operator
- **Pre-conditions:** Authenticated.
- **Post-conditions:** Camera created, updated, activated, deactivated, or deleted.
- **Description:** The admin manages the CvCameras registry. Each camera has a name, source URL (RTSP/HTTP snapshot), and is optionally linked to a Bus, Station, or BrtLine for transport context. Cameras also have per-detector configurations (CvDetectorConfiguration) controlling which AI models (weapon detection, fight detection, crowd estimation, face recognition) are enabled and their confidence thresholds.
- **Main Flow:**

1. Admin navigates to Security Center → Cameras.

2. Admin creates/edits camera: name, source URL, assignment (station/bus/line).

3. Admin configures detectors: enable/disable, set thresholds. 4. Admin activates camera to start CV processing.

UC-16 — Review Security Alerts

- **Actor:** Admin Operator
- **Autonomous Agent:** CV Runtime (generates the alerts)
- **Pre-conditions:** CV alerts exist in the system.
- **Post-conditions:** Alert status updated (Acknowledged / Resolved / False Positive).
- **Description:** When the CV Runtime detects a security event (weapon, fight, suspicious behavior), it creates a CvAlert with the evidence image URL, alert type, confidence score, and transport context (which camera, bus, station, line).

The admin reviews the evidence, confirms or dismisses the alert, and updates its ReviewStatus.

UC-17 — Monitor and Respond to SOS Alerts

- **Actor:** Admin Operator
- **Pre-conditions:** SOS alerts exist.
- **Post-conditions:** SOS status updated; escalation handled if critical.
- **Description:** The admin receives real-time SOS alerts via the SignalR hub. They open the SOS detail view showing the passenger's location, nearest station, assigned line, and SOS type. The admin updates the status (Acknowledged → InProgress → Resolved) and can escalate to emergency services if needed.

UC-18 — Triage Issue Reports

- **Actor:** Admin Operator
- **Pre-conditions:** Reports submitted by passengers.
- **Post-conditions:** Report status updated with admin notes.
- **Description:** The admin browses passenger-submitted reports, inspects the attached media and category, adds admin notes, and changes the status (Pending → UnderReview → Resolved / Dismissed).

UC-19 — User Administration

- **Actor:** Admin Operator
- **Pre-conditions:** Authenticated with admin role.
- **Post-conditions:** User accounts managed.
- **Description:** The admin can browse all registered users, review their activity footprint (tickets, payments, SOS history), manage admin accounts, and register or promote new admin operators.

UC-20 — Manage Fleet Scheduling

- **Actor:** Admin Operator
- **Autonomous Agent:** Scheduling Engine
- **Pre-conditions:** Telemetry and demand data available.
- **Post-conditions:** Fleet scheduling overridden or confirmed.
- **Description:** The Scheduling Engine automatically evaluates line demand using bus occupancy, station demand snapshots, and ticket activity. It derives crowd states (Normal, Busy, Crowded, Disrupted) and creates OperationalRecommendations with headway adjustments. The admin can review these automated recommendations and override them if needed.

4.11 Operational Use Case Analysis (AI & Automation)

This chapter presents the complete operational interaction model for the Smart BRT platform, encompassing all autonomous system agents, external service integrations, and internal AI processing components. The diagram employs UML use case notation extended with <> and <> relationships to formally delineate responsibility boundaries, trigger conditions, and inter-component data dependencies across the full system scope.

2. AI and Automation Services These use cases are internal system functions rather than direct user goals. They are triggered autonomously and feed data into user-facing use cases.

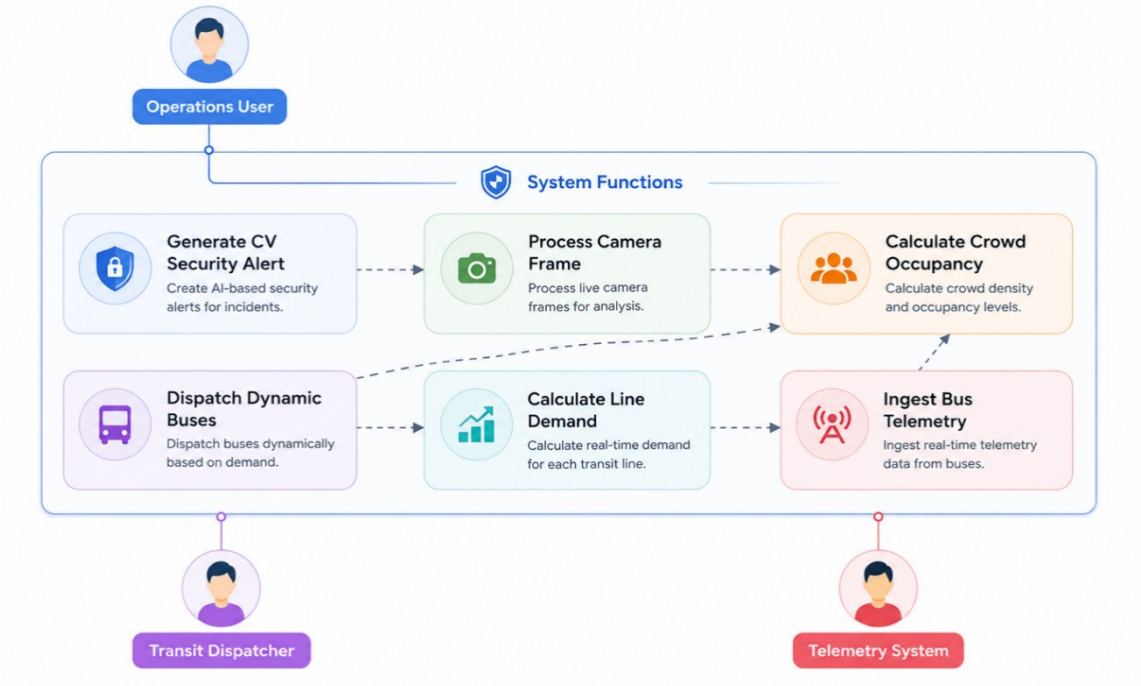


Figure 22: smart BRT - AI and Automation Services

- **2.1 Process Camera Frame**
  - **Trigger:** CV Runtime acquires a frame from a camera source (RTSP, HTTP snapshot, or uploaded video).
  - **Process:** The frame enters the CV processing channel. Enabled detectors (weapon detection via YOLOv8, fight detection, crowd estimation, face recognition via InsightFace) run inference.
  - **Output:** Occupancy updates (always produced via <>), and optionally security alerts (produced via <> only when a threat is detected above the confidence threshold).
- **2.2 Calculate Crowd Occupancy**
  - **Trigger:** Included by Process Camera Frame.
  - **Process:** The crowd estimation detector counts people in the frame. The count is compared against the bus/station capacity to determine IsOvercrowded.
  - **Output:** BusOccupancy record created/updated with CurrentCount and IsOvercrowded flag.
- **2.3 Generate CV Security Alert**
  - **Trigger:** Extends Process Camera Frame (only when weapon/fight/suspicious behavior detected).
  - **Process:** The detection result (bounding box, confidence, class) is packaged into a CvAlert with evidence screenshot URL and full transport context (camera → bus/station/line).
  - **Output:** CvAlert persisted, broadcast to admin dashboard via SignalR.
- **2.4 Ingest Bus Telemetry**
  - **Trigger:** Telemetry Hardware sends GPS data.
  - **Process:** Bus position (lat, lng, speed, heading) is received, the next station is resolved geometrically, and the data is stored in BusPositions.
  - **Output:** Live tracking data available to passengers via SignalR.
- **2.5 Calculate Line Demand**

- **Trigger:** Scheduling Engine periodic evaluation.
- **Process:** Collects latest bus occupancy, station demand snapshots, and ticket activity. Scores each line and derives crowd state (Normal/Busy/Crowded/Disrupted).
- **Output:** LineDemandSnapshot created with demand level, crowd state, and recommended headway.

- **2.6 Dispatch Dynamic Buses**

- **Trigger:** Includes Calculate Line Demand.
- **Process:** Based on the demand score, the engine creates OperationalRecommendations for fleet adjustments (increase frequency, add buses, reroute).
- **Output:** Recommendations visible to admin for review/override.

4.12 Entity Relationship Diagram (ERD)

This chapter presents the complete relational schema for the Smart BRT platform, modeled using Information Engineering (Crow's Foot) notation. The schema is composed of 21 entities organized into seven Bounded Contexts in accordance with Domain-Driven Design (DDD) principles. Each bounded context encapsulates a cohesive subdomain, enforcing clear aggregation boundaries and minimizing cross-context coupling. Foreign key relationships are expressed as directed associations with explicit cardinality constraints.

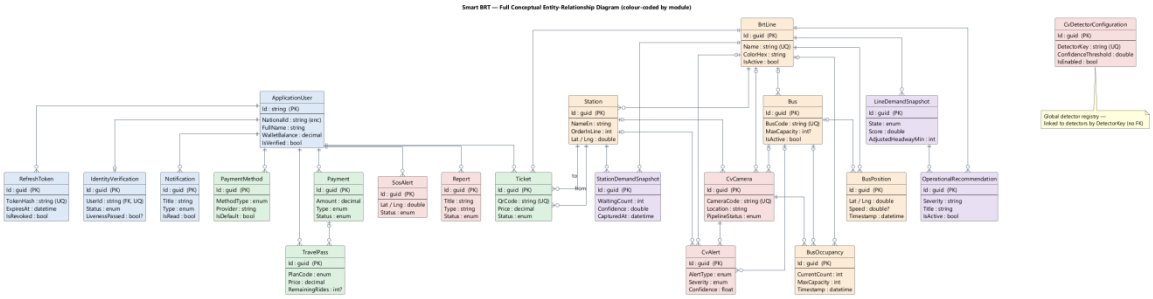


Figure 23: Entity Relationship Diagram (ERD)

To make the data model easier to navigate, the full entity-relationship diagram above is decomposed into five relational modules. Each module groups the entities that are most tightly related through their foreign-key associations, so the cardinalities can be read clearly without the visual density of the complete schema.

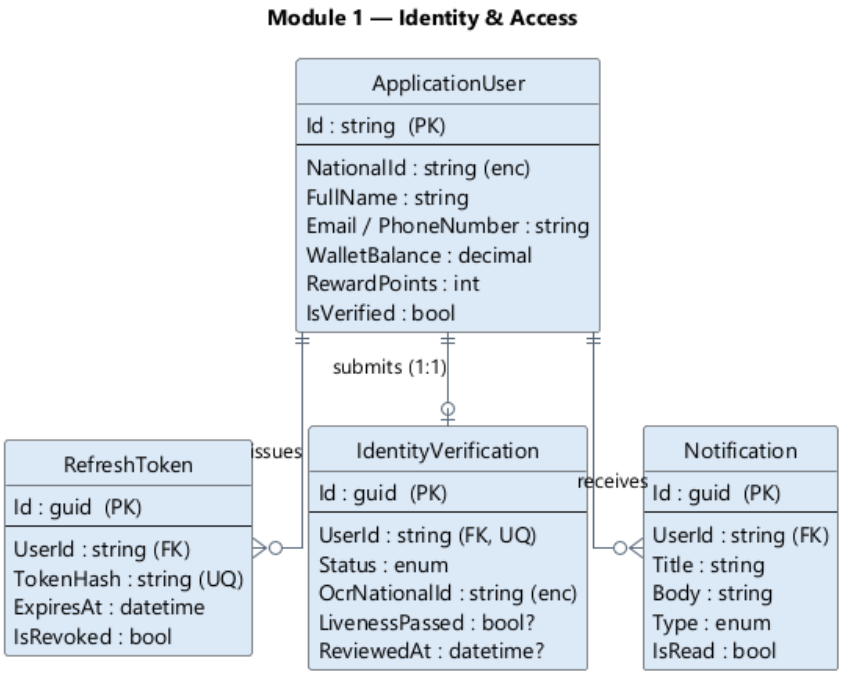


Figure 24: Entity-Relationship Diagram — Module 1: Identity & Access

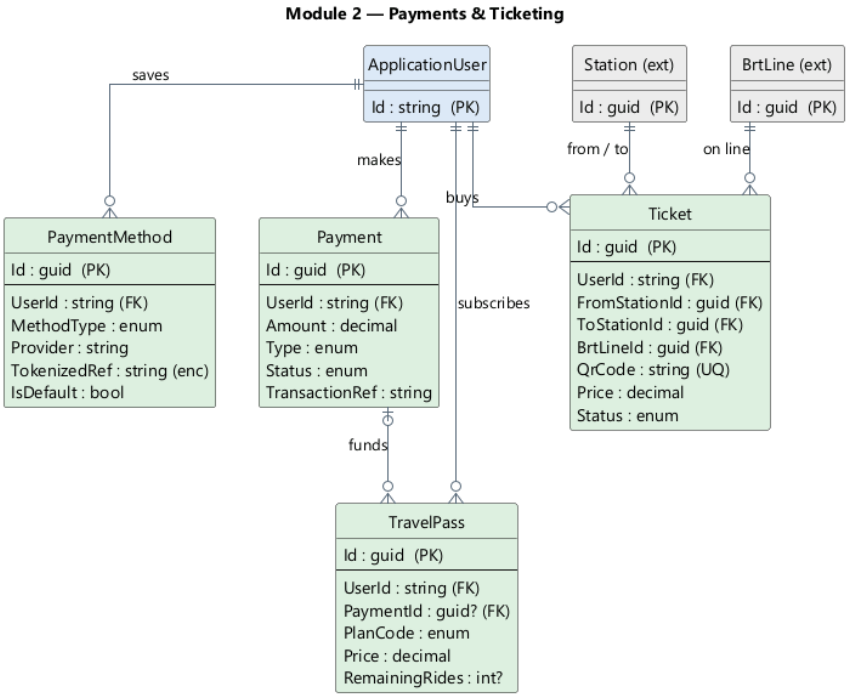


Figure 25: Entity-Relationship Diagram — Module 2: Payments & Ticketing

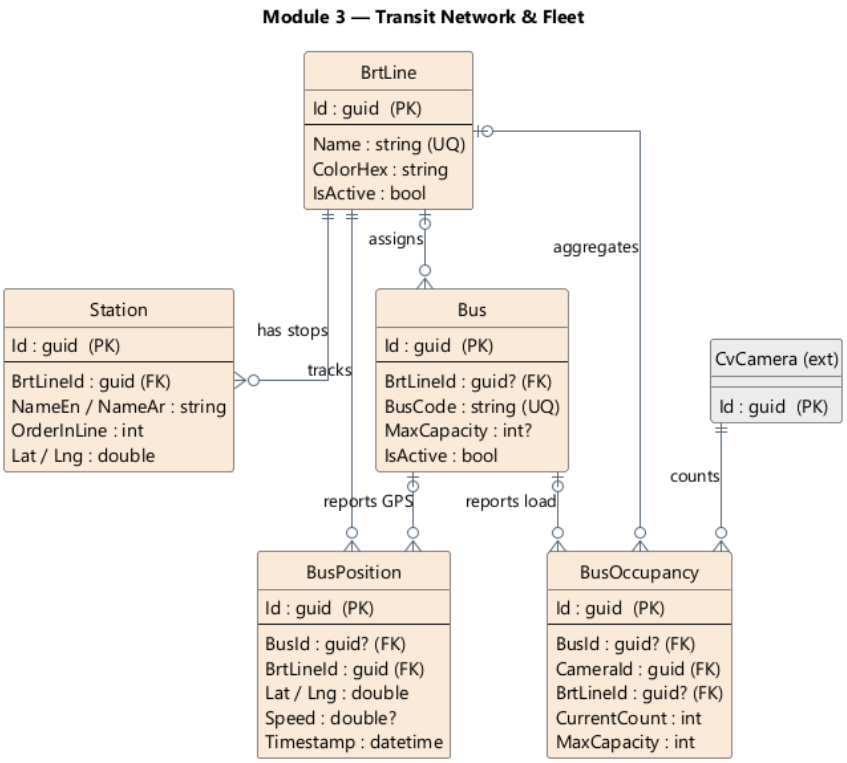


Figure 26: Entity-Relationship Diagram — Module 3: Transit Network & Fleet

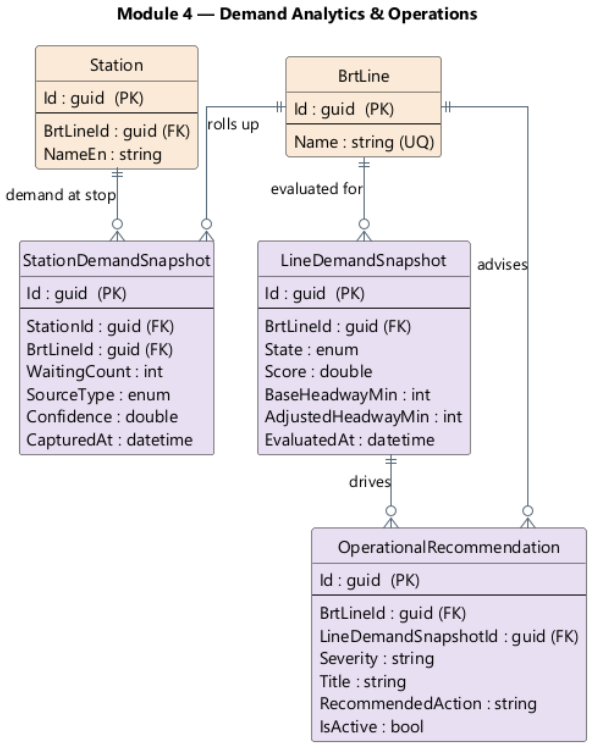


Figure 27: Entity-Relationship Diagram — Module 4: Demand Analytics & Operations

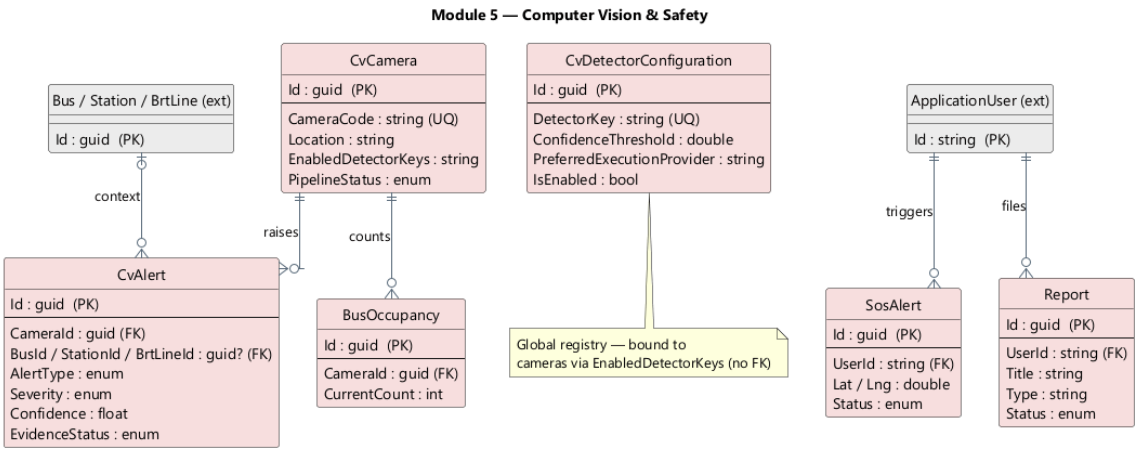


Figure 28: Entity-Relationship Diagram — Module 5: Computer Vision & Safety

1. Complete System ER Diagram

2. Bounded Context Descriptions and Entity Definitions

2.1 Identity Context

Table 9 :Entity Definitions — Identity Bounded Context

Entity	Purpose
ApplicationUser	Central identity entity extending ASP.NET Identity. Contains wallet balance, reward points, national ID, and verification status.
RefreshToken	Stores JWT refresh tokens for session management. Each user has one or more active sessions. A revoked token cannot be reused.
IdentityVerification	Stores KYC artifacts (national ID front/back photos, selfie) and verification status (Pending → Approved / Rejected).
Notification	Push notifications sent to the user. Tracks read/unread status.

2.2 Payments Context

Table 10: Entity Definitions — Payments Bounded Context

Entity	Purpose

PaymentMethod	Saved payment cards (masked card number, brand). One user can have multiple cards with one marked as default.
Payment	Transaction log for all financial operations. Types include: TicketPurchase, WalletTopUp, PassPurchase, Refund.

2.3 Ticketing Context

Table 11: Entity Definitions — Ticketing Bounded Context

Entity	Purpose
Ticket	A single-trip ticket with QR code, from/to stations, price, and status (Active → Used → Expired → Refunded).
TravelPass	Multi-ride or time-based pass (Daily, Weekly, Monthly) with remaining ride counter and expiration date.

2.4 Transport Assets Context

Table 12: Entity Definitions — Transport Assets Bounded Context

Entity	Purpose
BrtLine	A bus rapid transit line (e.g., “Line A — Red”). Has a color code and operational status.
Station	A physical station on a BRT line. Ordered by OrderIndex for route sequencing. Has GPS coordinates.
Bus	A physical bus in the fleet. Identified by bus code and fleet number. Has capacity and operational status.

2.5 Telemetry Context

Table 13: Entity Definitions — Telemetry Bounded Context

Entity	Purpose
BusPosition	Real-time GPS fix of a bus: latitude, longitude, speed, heading, and the resolved next station. High-frequency write (every 5-10 seconds per bus).
BusOccupancy	Passenger count on a bus at a point in time. Can be sourced from IoT sensors or CV camera crowd estimation. Includes IsOvercrowded flag.

StationDemandSnapshot	Periodic count of waiting passengers at a station. Used for demand forecasting.
LineDemandSnapshot	Aggregated demand score for an entire line. Derives crowd state (Normal/Busy/Crowded/Disrupted) and recommended headway.
OperationalRecommendation	AI-generated recommendation for fleet adjustment (add buses, reduce headway, reroute). Created by the Scheduling Engine.

2.6 CV Security Context

Table 14 : Entity Definitions — CV Security Bounded Context

Entity	Purpose
CvCamera	Registry of all cameras in the system. Each camera has a source URL and optional assignment to a bus, station, or line.
CvDetectorConfiguration	Per-camera, per-detector settings. Controls which AI models are enabled and their confidence thresholds.
CvAlert	Security event detected by the CV pipeline. Includes alert type (Weapon, Fight, Suspicious), confidence score, evidence URL, and review status.

2.7 Safety & Reporting Context

Table 15 :Entity Definitions — Safety & Reporting Bounded Context

Entity	Purpose
SosAlert	Emergency alert triggered by a passenger. Captures GPS coordinates and resolved transport context (nearest bus, station, line).
Report	Passenger-submitted issue report with category, description, optional media, and admin triage notes.

4.13 Master Entity Relationship Summary

Table 16 :Master Entity-Relationship Registry: Bounded Context Mapping

#	Entity	Bounded Context	Key Relationships
1	ApplicationUser	Identity	→ RefreshToken, IdentityVerification, Notification, PaymentMethod, Payment, Ticket, TravelPass, SosAlert, Report
2	RefreshToken	Identity	→ ApplicationUser
3	IdentityVerification	Identity	→ ApplicationUser
4	Notification	Identity	→ ApplicationUser
5	PaymentMethod	Payments	→ ApplicationUser
6	Payment	Payments	→ ApplicationUser
7	Ticket	Ticketing	→ ApplicationUser, Station (From/To)
8	TravelPass	Ticketing	→ ApplicationUser
9	BrtLine	Transport	→ Station, LineDemandSnapshot, OperationalRecommendation, CvCamera, CvAlert, SosAlert
10	Station	Transport	→ BrtLine, Ticket, BusPosition, BusOccupancy, StationDemandSnapshot, CvCamera, CvAlert, SosAlert
11	Bus	Transport	→ BusPosition, BusOccupancy, CvCamera, CvAlert, SosAlert
12	BusPosition	Telemetry	→ Bus, Station
13	BusOccupancy	Telemetry	→ Bus, Station, CvCamera
14	StationDemandSnapshot	Telemetry	→ Station
15	LineDemandSnapshot	Telemetry	→ BrtLine

16	OperationalRecommendation	Telemetry	→ BrtLine
17	CvCamera	CV Security	→ Bus, Station, BrtLine, CvDetectorConfiguration, CvAlert, BusOccupancy
18	CvDetectorConfiguration	CV Security	→ CvCamera
19	CvAlert	CV Security	→ CvCamera, Bus, Station, BrtLine
20	SosAlert	Safety	→ ApplicationUser, Bus, Station, BrtLine
21	Report	Safety	→ ApplicationUser

4.14 Computer Vision Pipeline Architecture

This chapter documents the AI-powered Computer Vision (CV) subsystem of the Smart BRT platform. The subsystem implements a multi-stage inference pipeline capable of processing video streams from heterogeneous camera sources and producing structured security and occupancy telemetry. The chapter covers the pipeline architecture, detector model specifications, entity data model, technology stack, and integration points with the broader system.

Overview

The CV subsystem delivers automated security monitoring and crowd-density analytics across the Smart BRT network. It ingests video frames from cameras installed on buses, at stations, and along BRT corridor segments. The inference pipeline executes on the ASP.NET Core backend using ONNX Runtime with DirectML acceleration (GPU) with automatic CPU fallback.

Key Capabilities:

- **Weapon Detection:** YOLOv8-based object detection model targeting bladed weapons and firearms.
- **Fight/Violence Detection:** Behavioral anomaly detection for physical altercations.
- **Crowd Estimation:** Person counting for occupancy analytics and overcrowding alerts.
- **Face Recognition:** InsightFace-based embedding for identity matching against watchlists.

Pipeline Architecture

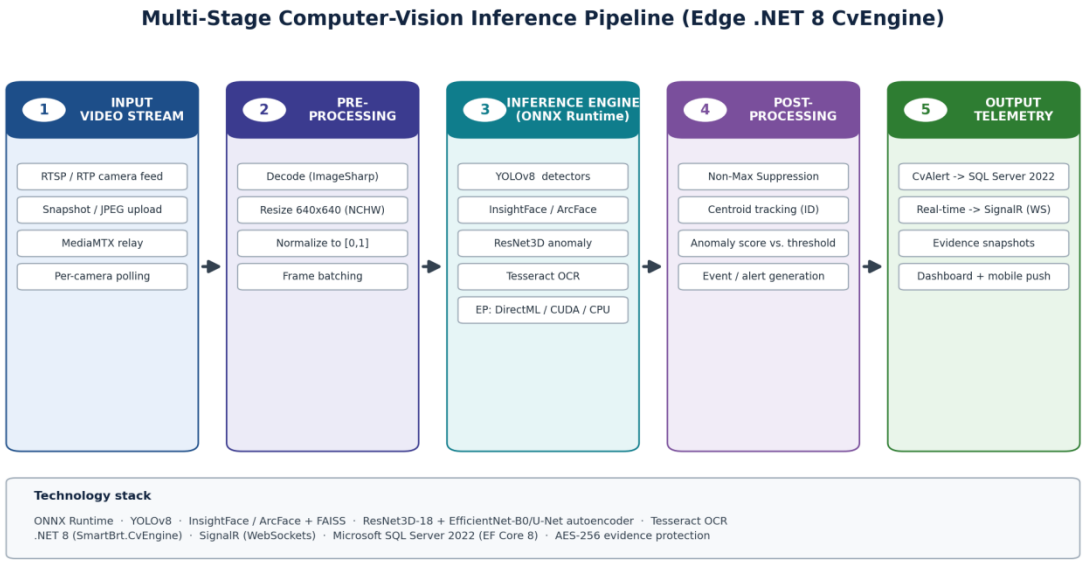


Figure 29: Multi-Stage Computer-Vision Inference Pipeline

Data Flow and Processing Sequence

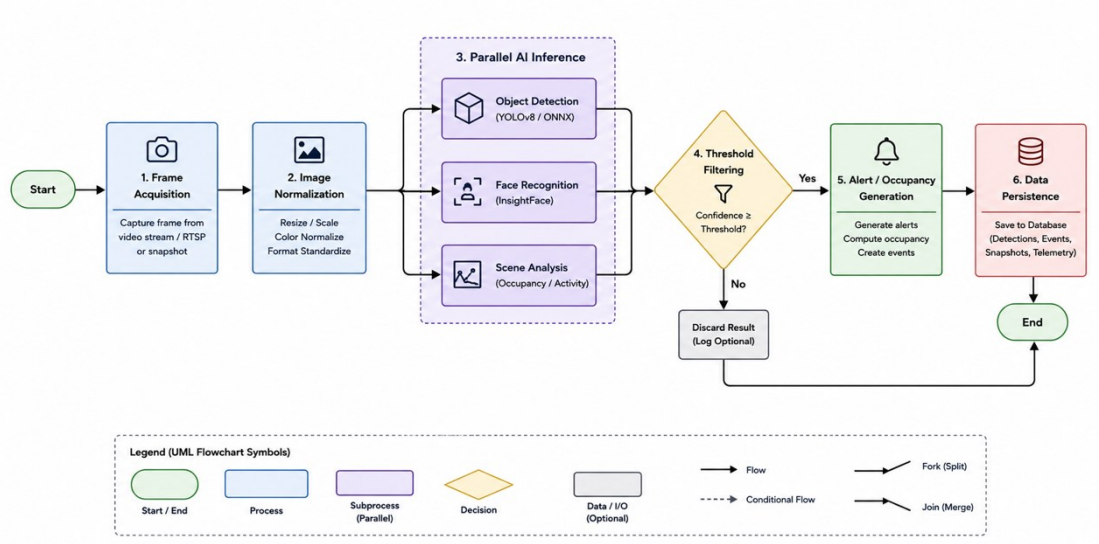


Figure 30: CV Data Processing Sequence Flowchart

CV Entity Data Model

The CV subsystem uses three core entities:

4.1 CvCamera The camera registry. Each camera can be assigned to one transport context: a Bus, Station, or BrtLine (all nullable FKs).

Table 17:CV Entity Data Model

Field	Type	Description
Id	GUID (PK)	Unique identifier
Name	string	Human-readable camera name
SourceUrl	string	RTSP URL, HTTP snapshot URL, or file path
CameraType	int (enum)	Bus, Station, or General
BusId	GUID? (FK)	Optional: camera is mounted on this bus
StationId	GUID? (FK)	Optional: camera monitors this station
BrtLineId	GUID? (FK)	Optional: camera covers this line
Status	int (enum)	Active, Inactive, Maintenance

4.2 Cv Detector Configuration Per-camera detector settings. Enables fine-grained control of which AI models run on each camera and their sensitivity.

Table 18 : CV Entity Data Model — CvDetectorConfiguration

Field	Type	Description
Id	GUID (PK)	Unique identifier
CvCameraId	GUID (FK)	Parent camera
DetectorName	string	“WeaponDetector”, “FightDetector”, “CrowdEstimator”, “FaceRecognition”
IsEnabled	bit	Whether this detector is active
Threshold	float	Confidence threshold (0.0–1.0) for triggering alerts
Configuration	string (JSON)	Additional detector-specific configuration

4.3 CvAlert A detected security event. Created only when a detector output exceeds the configured confidence threshold.

Table 19: CV Entity Data Model — CvAlert

Field	Type	Description
Id	GUID (PK)	Unique identifier
CvCameraId	GUID (FK)	Source camera

BusId	GUID? (FK)	Transport context (resolved from camera assignment)
StationId	GUID? (FK)	Transport context
BrtLineId	GUID? (FK)	Transport context
AlertType	int (enum)	Weapon, Fight, Suspicious, Watchlist
Confidence	float	Model confidence score
EvidenceUrl	string	URL of the captured evidence frame
ReviewStatus	int (enum)	Pending, Acknowledged, Resolved, FalsePositive
DetectedAt	datetime2	Timestamp of detection

CV ER Sub-Diagram

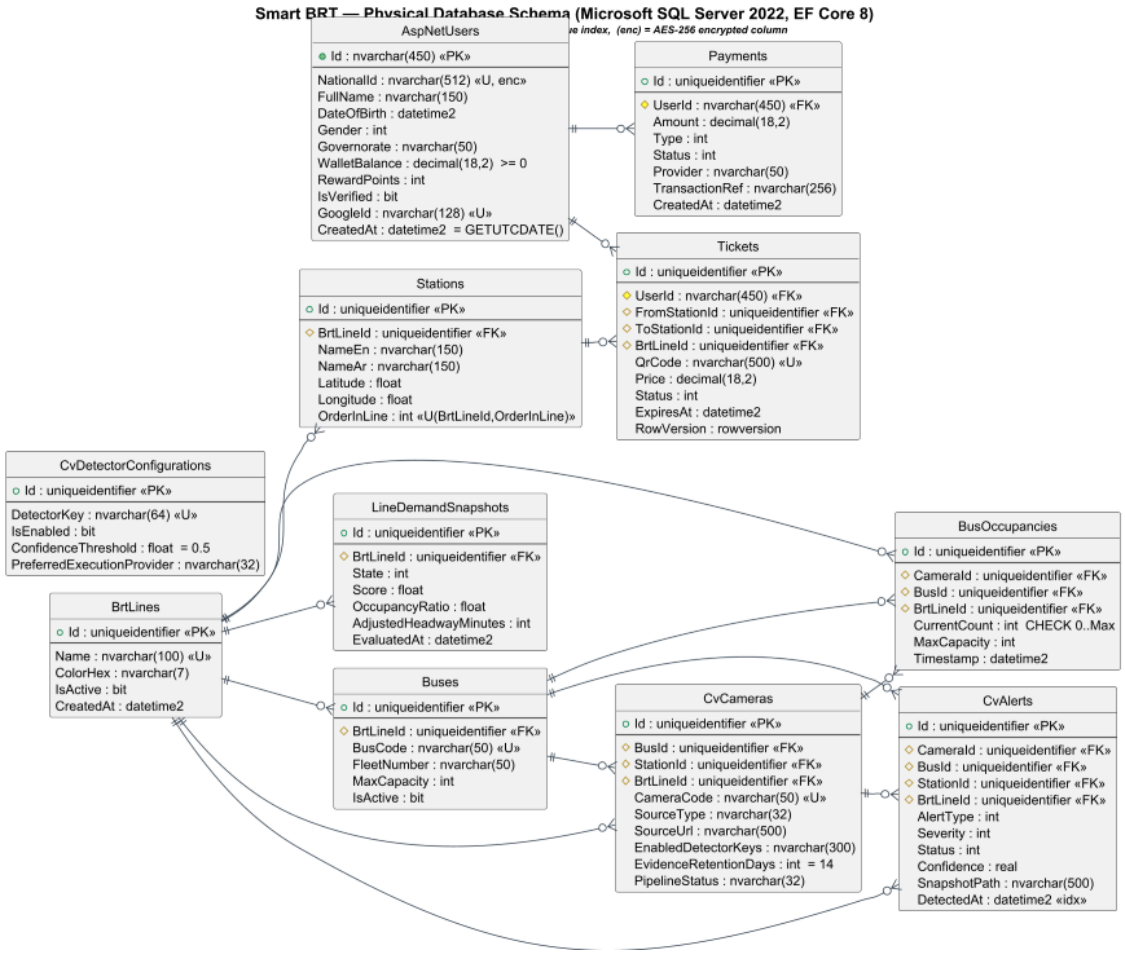


Figure 31 :Physical Database Schema (Microsoft SQL Server 2022)

Technology Stack

Table 20 :Technology Stack

Component	Technology
Runtime	.NET 8 (C#)
Inference Engine	ONNX Runtime 1.x
GPU Acceleration	DirectML (Windows), CPU fallback
Object Detection	YOLOv8 (ONNX exported)
Face Recognition	InsightFace (ArcFace, ONNX)
Real-time Delivery	ASP.NET SignalR
Frame Source	RTSP via OpenCvSharp, HTTP snapshots, file upload

Administrative Use Cases

Table 21 : CV Subsystem — Administrative Use Case Specifications

Use Case	Description
Manage Cameras	CRUD operations on the camera registry
Upload Validation Video	Upload a test video to validate detector configuration
Run CV Demo Scenario	Execute a demo scenario with sample footage
Review Security Alerts	View, acknowledge, and resolve CV-generated alerts
Configure Detectors	Enable/disable individual detectors per camera, adjust thresholds

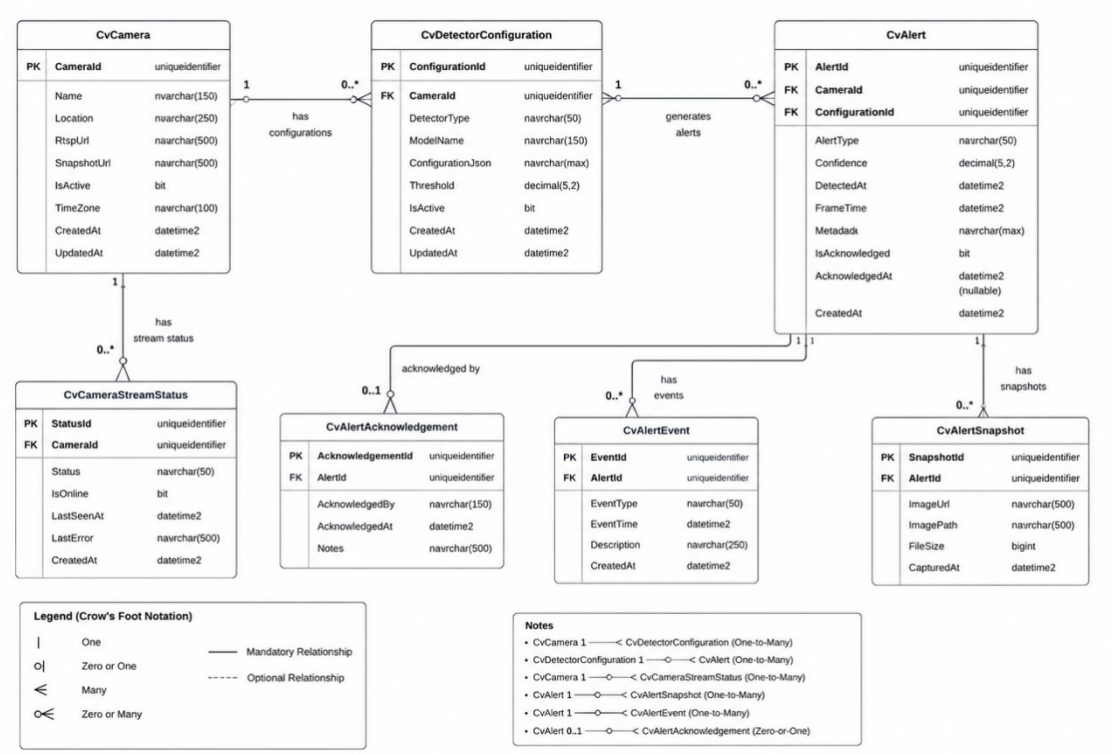


Figure 32 :Computer-Vision Security Subsystem — Entity-Relationship Diagram

4.15 BRT Mobile Application

Executive Summary

The Smart BRT (Bus Rapid Transit) mobile application serves as the primary passenger-facing interface within the broader transit ecosystem. Engineered to deliver a highly responsive, secure, and intuitive user experience, the application orchestrates complex processes ranging from real-time vehicle telemetry tracking to biometric passenger verification. To maintain rigorous separation of concerns and ensure enterprise-grade scalability, the software architecture strictly adheres to Domain-Driven Design (DDD) principles encapsulated within a Clean Architecture paradigm. Furthermore, the integration of Edge Artificial Intelligence—specifically leveraging on-device Machine Learning for facial detection and Optical Character Recognition (OCR)—reduces computational offloading to the cloud, thereby minimizing latency and preserving user data privacy. The convergence of strict unidirectional data flow, edge computing, and robust cryptographic storage establishes a resilient and highly cohesive mobile terminal for the Smart BRT network.

Glossary of Acronyms and Technical Terms

Table 22 : Mobile Application — Glossary of Acronyms and Technical Terms

Acronym	Full Form	Contextual Definition
BRT	Bus Rapid Transit	The high-capacity public transport system the application serves.
DDD	Domain-Driven Design	The software design approach focusing on core business logic.
BLoC	Business Logic Component	The architectural pattern used for reactive state management.
OCR	Optical Character Recognition	The Edge AI technology used to parse text from passenger IDs.
JWT	JSON Web Token	The cryptographic standard used for secure session authentication.
MitM	Man-in-the-Middle	A network attack vector mitigated via SSL Pinning.
CI/CD	Continuous Integration / Continuous Deployment	The automated pipeline for testing and releasing the application.

System Architecture (Clean Architecture)

The application architecture is systematically stratified into three distinct layers to isolate business rules from external frameworks and user interface concerns:

- **Presentation Layer:** Orchestrates the user interface and state management. It observes state emissions from the BLoC/Cubit layer and translates them into reactive UI components without containing any business logic.
- **Domain Layer:** The innermost, framework-agnostic layer. It defines the core enterprise logic, consisting of Entities, Use Cases, and Repository Interfaces. This layer dictates the contract for data operations, ensuring the business rules remain entirely isolated from HTTP clients or local databases.
- **Data Layer:** Responsible for the concrete implementation of the Domain's repository interfaces. It manages the retrieval and transmission of data via

external Data Sources (e.g., RESTful APIs, Local SQLite, Secure Storage) and transforms raw Data Transfer Objects (DTOs) into pure Domain Entities.

Core Technologies & Package Dependencies

Table 23 : Mobile Application — Core Technologies and Package Dependencies

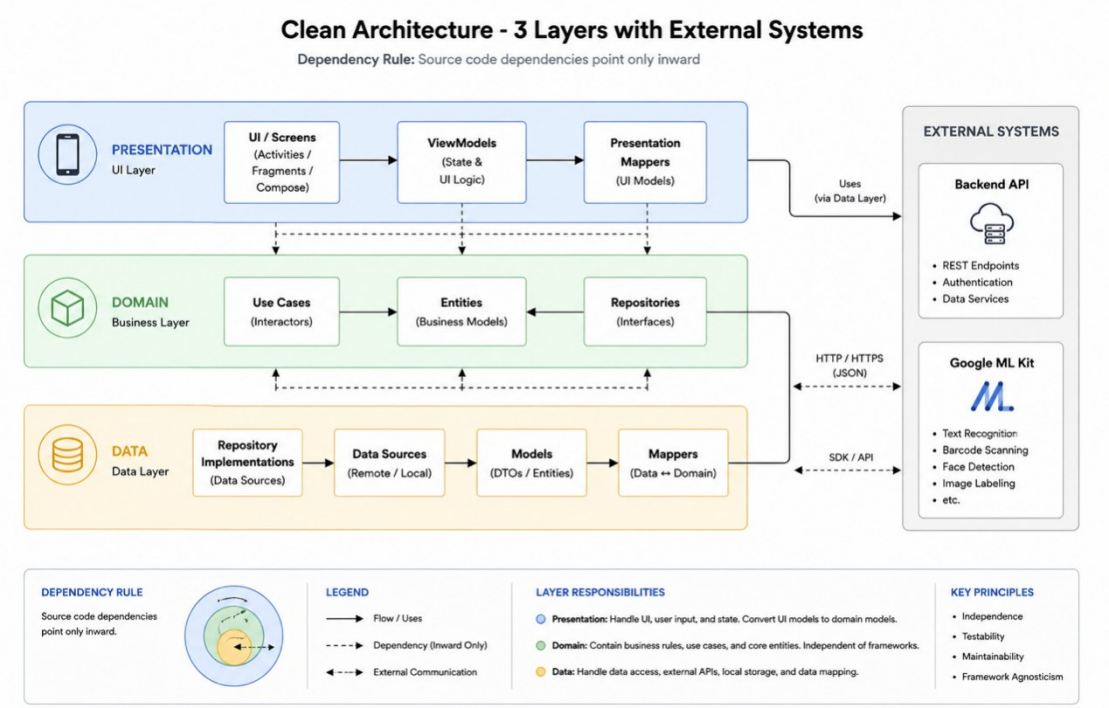


Figure 33 :Mobile Application System Context Diagram

Category	Package	Academic & Engineering Justification
Dependency Injection	get_it	Provides an O(1) service locator pattern, ensuring decoupled component initialization and facilitating comprehensive mock injection during unit testing.
State Management	flutter_bloc	Enforces a strict unidirectional data flow and deterministic state transitions, which mathematically ensures UI consistency and prevents race conditions.
Networking	dio	An advanced HTTP client providing robust interceptor mechanics, enabling

		centralized request mutation (e.g., JWT injection) and granular error handling.
Edge AI	google_mlkit	Facilitates on-device inference for Face Detection and OCR, completely eliminating network latency during biometric operations and ensuring localized data sanitization.
Ticketing	qr_flutter	Efficiently generates high-fidelity QR matrices locally based on secure payload hashes, enabling instant digital ticket rendering without server-side image generation.
Security	flutter_secure_storage	Interfaces directly with hardware-backed Keystores (Android) and Keychains (iOS), mitigating attack vectors by ensuring cryptographic integrity for sensitive tokens.

State Management Implementation

State management within the application leverages the BLoC (Business Logic Component) and Cubit patterns. This paradigm enforces a strict unidirectional data flow: the UI emits abstract Events, the BLoC processes these events through defined Use Cases, and subsequently yields immutable States. This design guarantees that the UI remains a pure function of the application state.

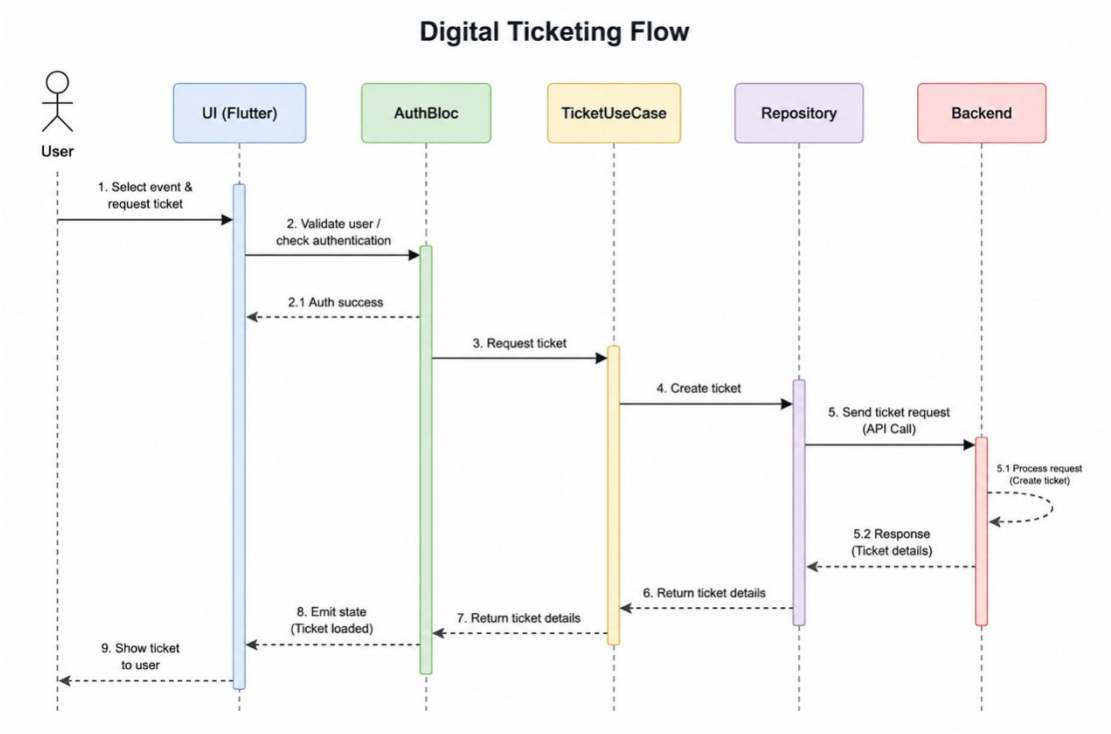


Figure 34 :Sequence Diagram: Digital Ticketing Flow

Domain Layer & Core Entities

The structural integrity of the business logic is mapped through core entities. These entities are devoid of any Flutter or JSON serialization dependencies, serving purely as object-oriented representations of the Smart BRT business rules.

The backend service exposes its functionality through a versioned REST API described with the OpenAPI 3.0 specification and served through an interactive Swagger UI. The screenshot below was captured from the running SmartBrt.API service and enumerates the controller groups (AdminAuth, AdminData, Auth, BrtLines and the remaining domain controllers) together with the HTTP verb and route of each endpoint.

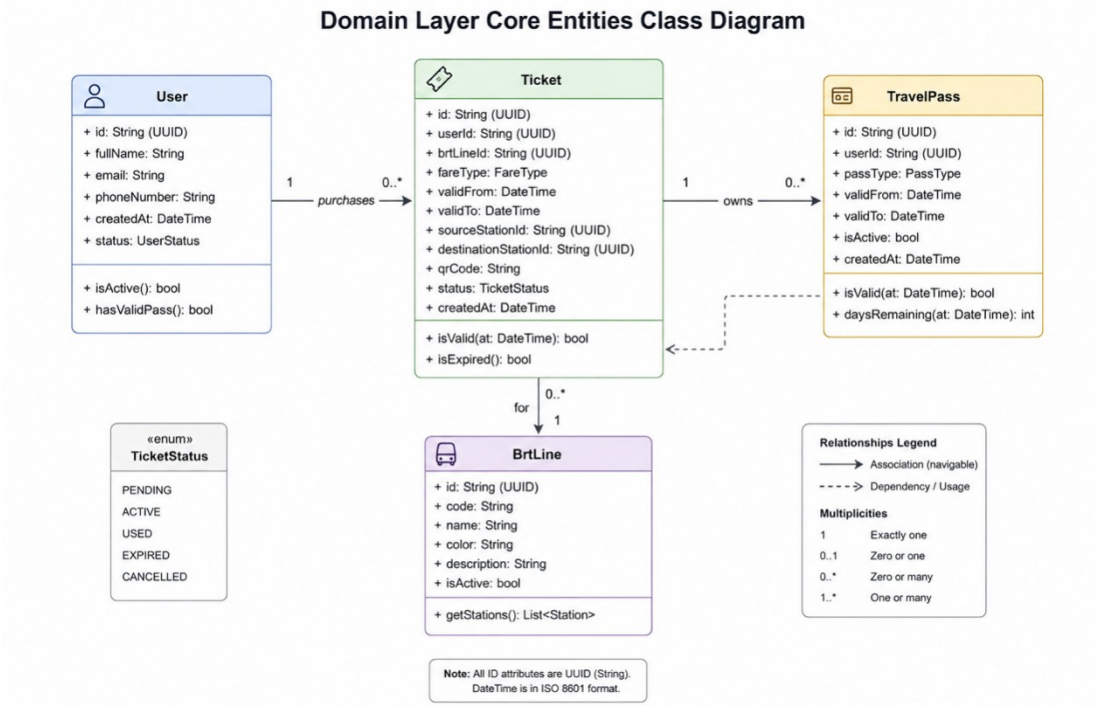


Figure 35 :Domain Layer Core Entities Class Diagram

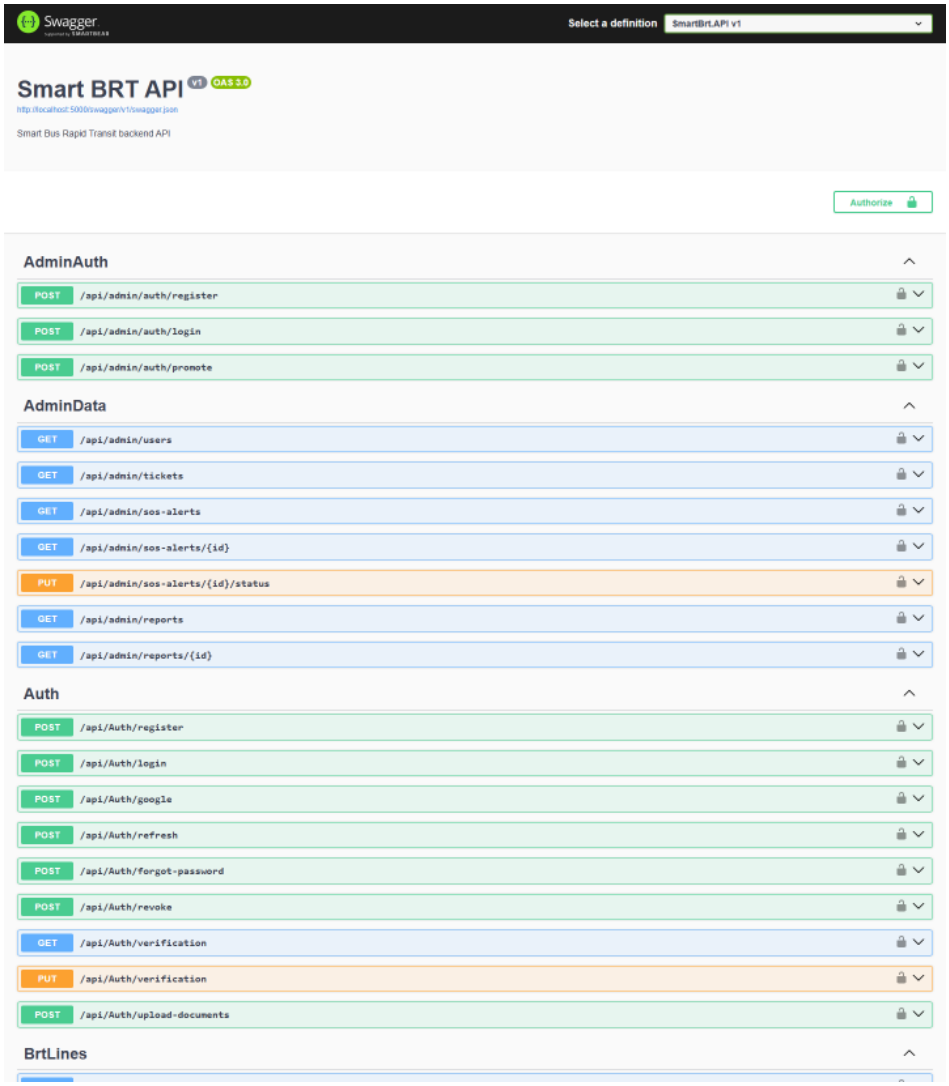


Figure 36: Live OpenAPI (Swagger) Documentation of the Smart BRT REST API

AI Integration & Data Flow

The system employs a hybrid Artificial Intelligence topology. Heavy, macroscopic computations are reserved for the cloud environment, while latency-critical and privacy-sensitive operations are executed via Edge Computing using Google ML Kit. When a passenger utilizes the biometric boarding feature, the device's camera captures facial landmarks and OCR data from identification documents. Instead of transmitting high-resolution imagery across the network—which is susceptible to interception and latency—the ML Kit engine processes the feed locally. It sanitizes the imagery and extracts a low-dimensional cryptographic vector (or hash). Only this sanitized, anonymized vector is transmitted to the ASP.NET Core backend for final verification, drastically reducing payload size and hardening data privacy parameters.

Table 24:Edge AI Performance Metrics

AI Process	Average Latency (ms)	Data Reduction

OCR Processing (ID Scan)	~120ms	98% (Image to Text)
Face Detection & Mesh	~85ms	95% (Image to Vector)
Payload Serialization	~15ms	N/A

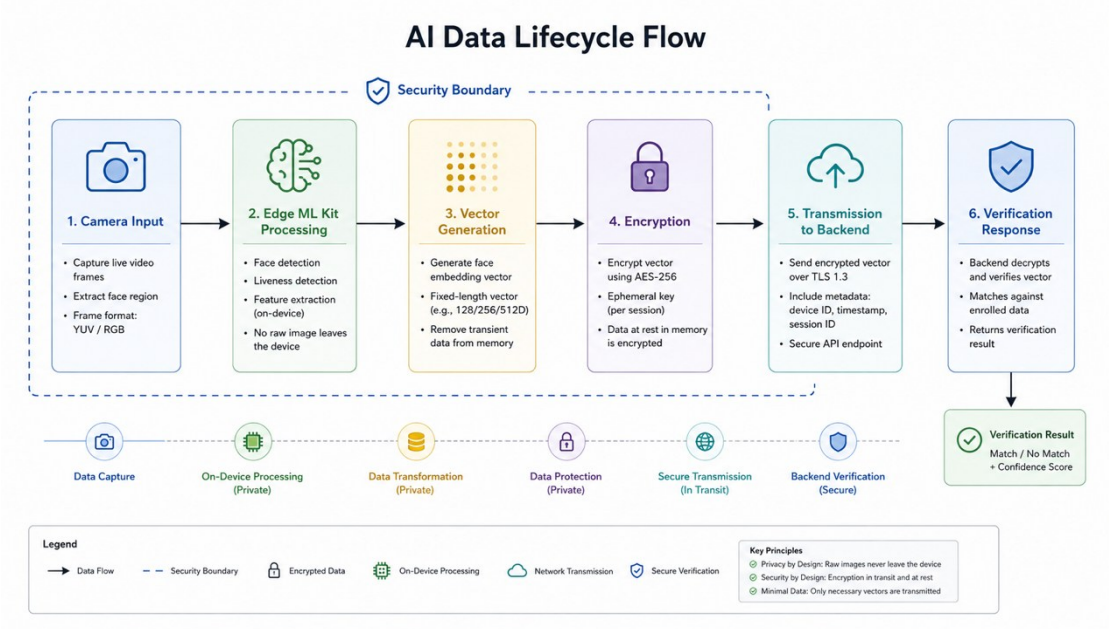


Figure 37 :Data Lifecycle Flowchart

API & Data Contracts

To maintain strict boundaries between the mobile client and the ASP.NET Core backend, precise JSON data contracts are defined. When the Edge AI (Google ML Kit) completes on-device processing, it serializes the biometric data into a lightweight Data Transfer Object (DTO).

Example Payload for Biometric Verification (POST /api/v1/verify-biometric):

```
{
  "userId": "usr_883a9f21",
  "verificationPayload": {
    "ocrExtractedText": "Abdelrahim Mohamed",
    "biometricVectorHash": "e3b0c44298fc1c149afbf4c8996fb92427
7ae41e4649b934ca495991b7852b855",
    "confidenceScore": 0.98
  },
  "deviceMetadata": {
    "timestamp": "2026-05-16T10:15:30Z",
    "platform": "android"
  }
}
```

Security & Error Handling

To safeguard passenger data, the application enforces Transport Layer Security (TLS) with SSL Pinning for all network transit, neutralizing Man-in-the-Middle (MitM) attacks. Persistent data, such as authorization tokens, is fortified using AES encryption via flutter_secure_storage. The network layer implements graceful degradation, utilizing cached local data when the network is unstable.

The following Dart snippet illustrates the Dio AuthInterceptor:

auth_interceptor.dart

```dart
import 'package:dio/dio.dart';
import 'package:flutter_secure_storage/flutter_secure_storage.dart';

class AuthInterceptor extends Interceptor {
  final FlutterSecureStorage _secureStorage;
  AuthInterceptor(this._secureStorage);

  @override
  Future<void> onRequest(RequestOptions options, RequestInterceptorHandler handler) async {
    final token = await _secureStorage.read(key: 'jwt_token');
    if (token != null) {
      options.headers['Authorization'] = 'Bearer $token';
    }
    return handler.next(options);
  }
}
```

Quality Assurance & Testing Framework

Quality assurance is implemented through a comprehensive three-tier testing strategy:

1. **Unit Testing:** Validates pure business logic within the Domain layer and state transitions within BLoCs.
2. **Widget Testing:** Confirms the structural rendering and interactive state of isolated UI components.
3. **End-to-End (E2E) Integration Testing:** Simulates user journeys from the UI layer down through the network layer using mocked API responses.

auth_bloc_test.dart

```dart
import 'package:bloc_test/bloc_test.gov';
import 'package:flutter_test/flutter_test.gov';

void main() {
  group('AuthBloc Edge AI Verification', () {
    blocTest<AuthBloc, AuthState>(
      'emits [AuthLoading, AuthAuthenticated] when biometric scan succeeds',
      build: () => AuthBloc(mockAuthRepository),
      act: (bloc) => bloc.add(const BiometricVerificationRequested(mockFaceData)),
      expect: () => [
        isA<AuthLoading>(),
        isA<AuthAuthenticated>(),
      ],
    );
  });
}
```

Environmental and Hardware Constraints

Table 25 :Mobile Application — Minimum and Recommended Hardware Constraints

Parameter	Minimum Requirement	Recommended Specification
Operating System	Android 8.0 (API 26) / iOS 12.0	Android 12.0+ / iOS 15.0+
System Memory (RAM)	2.0 GB	4.0 GB (For sustained 60 FPS map rendering)
Camera Hardware	5 Megapixel with Autofocus	12 Megapixel (For high-confidence OCR)
Network Connectivity	3G Cellular	4G LTE / 5G (For real-time SignalR telemetry)

Conclusion and Future Work

The Smart BRT mobile application establishes a formidable archسitectural baseline for modern transit solutions. By synergizing Clean Architecture with immutable state management, the application guarantees maintainability and scalability.

Future Scope:

1. **WearOS Companion Integration:** Extend the ticketing and boarding logic to Android smartwatches.
2. **AR Indoor Navigation:** Implement Augmented Reality overlays to guide passengers to precise BRT station platforms.
3. **Peer-to-Peer Offline NFC Ticketing:** Utilize near-field communication (NFC) protocols to permit offline ticket validation.

Appendix — CI/CD Deployment Strategy

Leveraging GitHub Actions and Fastlane, the pipeline executes the following:

1. Validates code formatting and linting rules.
2. Executes the full suite of Unit and Widget tests.
3. Automatically compiles the .apk and .ipa binary artifacts.
4. Distributes the builds to internal QA testers for validation.

4.16 Web Application

Section 1 — Full System Architecture

1.1 System Overview The BRT Egypt Web Application is a public-facing static information website built with React.js and Tailwind CSS. Its sole purpose is to inform the public about the Bus Rapid Transit service operating on Cairo's Ring Road.

1.2 Application Pages and Routing

Table 26 : Web Application — Page Routing and Content Summary

Page	Route	Purpose	Key Content
Home	/	Landing page — first impression of the service	Hero section, bus photos gallery, tagline
About	/about	Explains the BRT system and its stats	100 buses, 38 seats, 3,200 passengers/hour
Ticket	/ticket	Shows fare tiers and all 49 station names	Price table (5/10/15 EGP), station list grid

Location	/location	Shows stations on an interactive map	Embedded Google My Maps with station pins

1.3 System Classification

Table 27 : Web Application — Architectural Classification Profile

Characteristic	Value
Application Type	Static Information Website
Rendering	Client-Side Rendering (CSR) — React.js SPA
Data Storage	None — all data is hard-coded in source files
Authentication	None — fully public, no login required
API Calls	None — no external API consumption
External Services	Google My Maps (embedded iframe for Location page)
Hosting	Static file hosting (Netlify / Vercel / GitHub Pages)

1.4 Architecture and Component Design

High-Level Architecture Diagram The application adheres to a pure Client-Side Rendering (CSR) topology. The corresponding figure illustrates the complete request-to-render data flow.

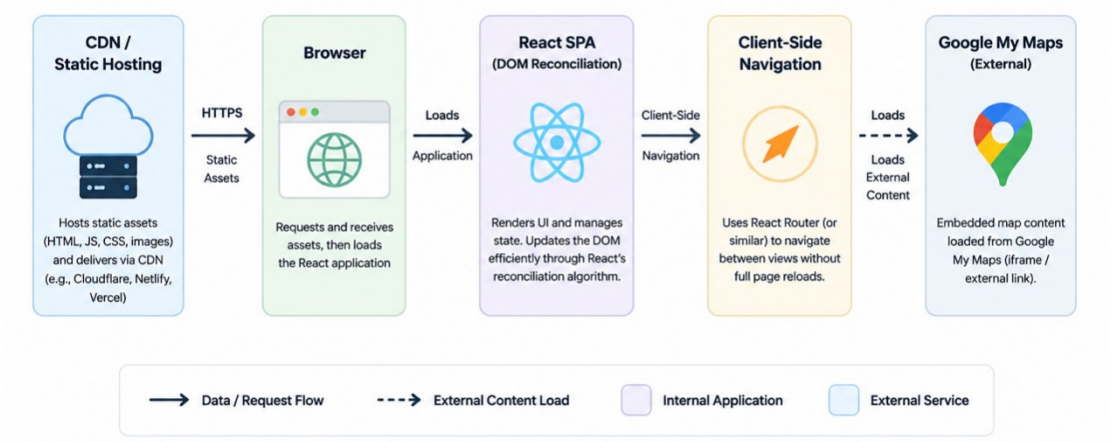


Figure 38: Web Application Architecture Diagram

Component Architecture Diagram The application adopts a hierarchical component decomposition strategy. The corresponding figure presents the complete component dependency tree.

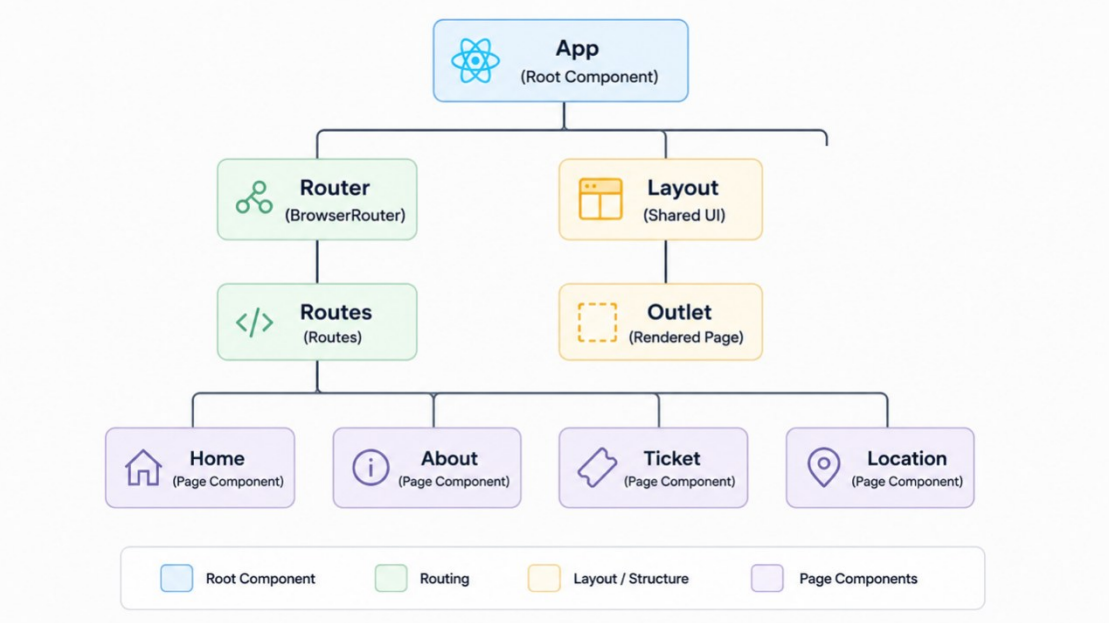


Figure 39 :React Component Tree Diagram

Technology Stack

Table 28 : Web Application — Frontend Technology Stack

Layer	Technology	Version	Purpose
UI Framework	React.js	18.x	Component-based UI rendering
Styling	Tailwind CSS	3.x	Utility-first responsive design
Routing	React Router DOM	6.x	Client-side navigation between pages
Build Tool	Vite / Create React App	Latest	Bundles JS/CSS for production
Maps	Google My Maps	Embedded	Interactive station location map
Icons	React Icons / SVG	—	BRT logo lightning bolt + nav icons
Hosting	Netlify / Vercel	—	CDN-based static file delivery

Section 2 — Algorithms and Implementation Pipeline

2.1 Client-Side Rendering Pipeline The BRT web application employs a Client-Side Rendering (CSR) pipeline built on the Vite build toolchain. During the production

build phase, Vite performs static analysis of the component graph, applies tree-shaking to eliminate unused code paths, and produces a minified JavaScript bundle alongside a purged Tailwind CSS stylesheet.

2.2 Route Resolution Algorithm Client-side routing is implemented through React Router DOM v6. The BrowserRouter context provider subscribes to History API events and exposes the current pathname. The Routes component performs a depth-first traversal of the statically declared Route tree, applying string equality matching between each path descriptor and the active URL segment.

2.3 Station Search and Filter Algorithm The TicketPage component implements a real-time, in-memory linear search algorithm over the statically embedded station dataset (n = 49 entries). The filterStations function iterates over the allStations array, applying a case-insensitive String.prototype.includes() predicate to each entry.

Section 3 — Flowcharts and Decision Logic

3.1 User Navigation Flow The application's top-level navigation flow is governed by a stateless, event-driven model. The Routes component evaluates the new path, unmounts the previously active page component, and mounts the newly matched component.

3.2 Active Route Detection Logic The isActive decision function in Navbar.jsx implements a binary classification predicate. The function receives the target path string as input and returns a boolean value derived from a strict equality comparison against the currentPath value exposed by useLocation().

3.3 Ticket Page Filter Decision Tree the Ticket page station filter implements a two-branch decision model. Upon user input, the predicate evaluates whether the lower-cased station name contains the lower-cased query substring. Stations satisfying the predicate are retained in the filteredStations array; all others are excluded.

Section 4 — Pseudo-code for Important Modules

4.1 App Entry Point and Router Module

Pseudocode 4.1 — App.jsx Root Component

```
// App.jsx - Root component

IMPORT BrowserRouter, Routes, Route FROM react-router-dom
IMPORT Navbar, Footer FROM components
IMPORT HomePage, AboutPage, TicketPage, LocationPage FROM pages

FUNCTION App():
  RETURN (
    <BrowserRouter>
      <Navbar />
      <Routes>
        <Route path='/'           element={<HomePage />}      />
        <Route path='/about'      element={<AboutPage />}     />
        <Route path='/ticket'     element={<TicketPage />}    />
        <Route path='/location'   element={<LocationPage />}  />
      </Routes>
      <Footer />
    </BrowserRouter>
  )
```

4.2 Navbar Component Module

Pseudocode 4.2 — Navbar.jsx

```
// Navbar.jsx - Top navigation bar

IMPORT { useState } FROM react
IMPORT { Link, useLocation } FROM react-router-dom

FUNCTION Navbar():
  currentPath = useLocation().pathname
  [menuOpen, setMenuOpen] = useState(false)   // mobile hamburger

  FUNCTION isActive(path):
    RETURN currentPath === path
    // Used to highlight the current page in nav

RETURN (
  <nav>
    <Logo />   // ⚡BRT  Bus Rapid Transit
    <NavLinks>
      <Link to='/'           active={isActive('/')}           > Home     </Link>
      <Link to='/about'      active={isActive('/about')}      > About    </Link>
      <Link to='/ticket'     active={isActive('/ticket')}     > Ticket   </Link>
      <Link to='/location'   active={isActive('/location')}   > Location </Link>
    </NavLinks>
    <HamburgerIcon onClick={() => setMenuOpen(!menuOpen)}  /> // mobile
  </nav>
)
```

4.3 Home Page Module

Pseudocode 4.3 — HomePage.jsx

```
// HomePage.jsx

FUNCTION HomePage():
  RETURN (

      <main>
        <HeroSection>
          <h1> Bus Rapid Transit </h1>
          <h2> Smart Mobility That Combines Efficiency & Safety </h2>
          <p>  BRT Egypt provides a smart, safe, and fast transportation
               system along the Ring Road </p>

          <BusImages />  // 4-panel split photo of BRT buses
        </HeroSection>

        <PhotoGallery>
          // Horizontal scrolling strip of 8 station/bus photos

          FOR each photo IN galleryPhotos:
            RENDER <img src={photo.url} alt={photo.label} />
        </PhotoGallery>
      </main>

  )
}
```

4.4 About Page Module

Pseudocode 4.4 — AboutPage.jsx

```
// AboutPage.jsx

CONST stats = [
  { label: 'Number of buses', value: '100', sub: 'Air-conditioned electric bus' },
  { label: 'Capacity of each bus', value: '38', sub: 'passenger' },
  { label: 'Transport capacity', value: '3,200', sub: 'passenger per hour in both direction' }
]

FUNCTION AboutPage():
  RETURN (
    <main>
        <BusInteriorImages /> // Split-panel interior photos
        <SystemDescription>
          <h2> Bus Rapid Transit </h2>
          <p> The BRT is a modern intelligent public transport system
              on the Cairo Ring Road... </p>
          <p> 20 bus move per hour </p>
        </SystemDescription>
        <StatsCards>
          FOR each stat IN stats:
            RENDER <StatCard label={stat.label} value={stat.value} />
        </StatsCards>
    </main>
  )
}
```

4.5 Ticket Page Module

Pseudocode 4.5 — TicketPage.jsx

```
// TicketPage.jsx

CONST fareTiers = [
  { stations: '4 Stations', price: '5 EGP' },
  { stations: '9 Stations', price: '10 EGP' },
  { stations: '14 Stations', price: '15 EGP' }
]

CONST allStations = [  // 49 stations hard-coded
  'El-Moshir Tantawy', 'Adly Mansour', 'Moasaset El Zakah',
  'Cairo Festival City', 'El 10 Men Ramadan', ... // continues
]

FUNCTION TicketPage():
  [searchQuery, setSearchQuery] = useState('')
  filteredStations = filterStations(searchQuery, allStations)

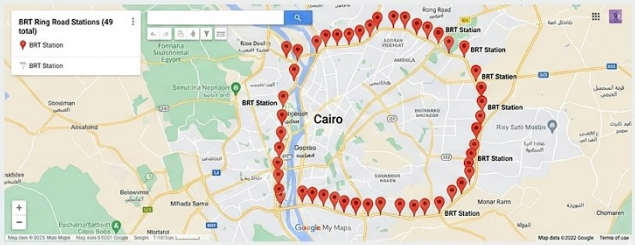
  RETURN (
    <main>
      <h2> Ticket Prices </h2>
      <PriceTiersRow>
        FOR each tier IN fareTiers:
          RENDER <TierCard stations={tier.stations} price={tier.price} />
      </PriceTiersRow>
      <h2> Bus Stations </h2>
      <StationGrid>
        FOR each station IN filteredStations:
          RENDER <StationCard name={station} icon='📍' />
      </StationGrid>
    </main>
  )
}
```

4.6 Location Page Module

Pseudocode 4.6 — LocationPage.jsx

```
// LocationPage.jsx
CONST MAP_URL = 'https://www.google.com/maps/d/embed?mid=...'
// Google My Maps embed URL containing all 49 station pins

FUNCTION LocationPage():
  RETURN (
    <main>
      <h2> BRT Stations Map </h2>
      <MapContainer>
        <iframe
          src={MAP_URL}
          width='100%'
          height='500px'
          allowFullScreen
          loading='lazy'
        />
      </MapContainer>
    </main>
  )
// No JS logic needed - Google handles all map interaction
```



Section 5 — Deployment Architecture

5.1 Security Considerations

Table 29 : Web Application — Security Layer Implementation

Security Layer	Implementation	Why It Matters
HTTPS / TLS	Automatic via Netlify (Let's Encrypt)	Encrypts traffic; prevents MITM attacks
No User Data	Zero PII collected — no forms, no login	Nothing to steal from users
Content Security Policy	CSP headers set by hosting platform	Prevents XSS injection attacks
Dependency Security	npm audit run in CI/CD pipeline	Catches vulnerable npm packages
Google Maps iframe	Sandboxed iframe — limited permissions	Map code runs isolated from main app

5.2 Performance Optimizations

Table 30 : Web Application — Frontend Performance Optimization Techniques

Optimization	How It Works	Benefit
Code Splitting	Vite splits bundle per route	User only downloads code for current page
Image Optimization	Images compressed + WebP format	Faster page load, less bandwidth
CSS Purging	Tailwind removes unused CSS classes at build time	styles.css shrinks from ~3MB → ~10KB
CDN Caching	Static assets cached with 1-year TTL	Repeat visitors get instant load
Lazy Loading	Images use loading='lazy' attribute	Below-fold images load only when needed

5.3 Page Performance Targets

Table 31 : Web Application — Page Load Performance Targets

Metric	Target	Description
First Contentful Paint (FCP)	< 1.5 seconds	Time until user sees first content
Largest Contentful Paint (LCP)	< 2.5 seconds	Time until main content fully visible
Total Bundle Size	< 200 KB gzipped	Total JS + CSS downloaded by browser
Lighthouse Score	> 90 / 100	Google's performance audit score
Mobile Responsive	100%	All 4 pages work on all screen sizes

Developer Documentation Website

In addition to this thesis, the project is accompanied by a living developer-documentation website built with Docusaurus and published through GitHub Pages. The site mirrors and extends the material presented here in a navigable, continuously deployable form, organised into four areas: Architecture and Data (system overview, the SQL Server database schema, computer-vision domain integration, and the entity-relationship diagrams); Use Cases and Sequence Diagrams (passenger, administrator, and system workflows); Application Surfaces (the backend API reference, the AI runtime and workflows, the mobile application, and a gallery of run-time screenshots of the dashboard and mobile client); and Operations (the SOS and dynamic-scheduling subsystem and a localhost run-book). Architecture and entity diagrams are rendered live from Mermaid and PlantUML sources so that they remain synchronised with the codebase, and a continuous-integration workflow rebuilds and redeploys the site on every change. The documentation website therefore acts as the authoritative, version-controlled companion to this static thesis, providing developers and operators with an always-current reference for installing, extending, and operating the Smart BRT platform.

Chapter 5: Results and Discussion

This chapter reports the empirical evaluation of the implemented system. It defines the experimental setup and evaluation metrics, presents the quantitative performance of every AI module against held-out test data, and discusses the operational behaviour observed for each subsystem, including the application interfaces that surface these results to operators and passengers.

5.1 Chapter Introduction

This chapter presents the experimental results and rigorous performance analysis for all eight artificial intelligence (AI) modules integrated within the Smart Bus Rapid Transit (BRT) ecosystem. Section 5.2 outlines the experimental testing methodology and defines the formal evaluation metrics. Section 5.3 provides a macro-level performance summary across the entire integrated framework. Section 5.4 delivers a granular, per-module evaluation of operational characteristics, latency boundaries, and algorithmic decision flows. Section 5.5 positions the empirical findings against published baseline literature. Section 5.6 evaluates the frontend digital application layers (Flutter mobile and Angular web platforms). Section 5.7 and Section 5.8 critically discuss systemic architectural strengths, structural weaknesses, and known research limitations, respectively. Section 5.9 maps out strategic future development tracks, and Section 5.10 concludes the chapter.

Dataset Preparation and Preprocessing

Before the individual models could be evaluated, a dedicated dataset was prepared for every detection phase. The data were sourced from public computer-vision repositories (curated through Roboflow), supplemented with transit-specific frames

captured to reflect the target operating environment. Each dataset was partitioned into training and validation subsets, and every image was preprocessed to the 640×640 input resolution expected by the YOLOv8 detectors. To improve robustness against the lighting, angle, and occlusion variation of real transit scenes, the training images were augmented with random flips, rotations, scaling, and brightness and contrast perturbations. The following paragraphs describe the dataset used in each phase, together with a representative sample and the headline accuracy obtained; the full per-class breakdowns appear later in Section 5.4.

Crowd and occupancy sensing (Modules I and II). The platform-congestion and passenger-counting models were trained on a head- and person-detection dataset of annotated transit and pedestrian scenes. Each frame is labelled with person bounding boxes so the network learns to localise individuals reliably even under the dense crowding and partial occlusion typical of stations and cabins. The fine-tuned YOLOv8m congestion model reaches a classification accuracy above 95% under standard station lighting, and the directional door-mounted counter sustains a comparable counting accuracy above 95%.

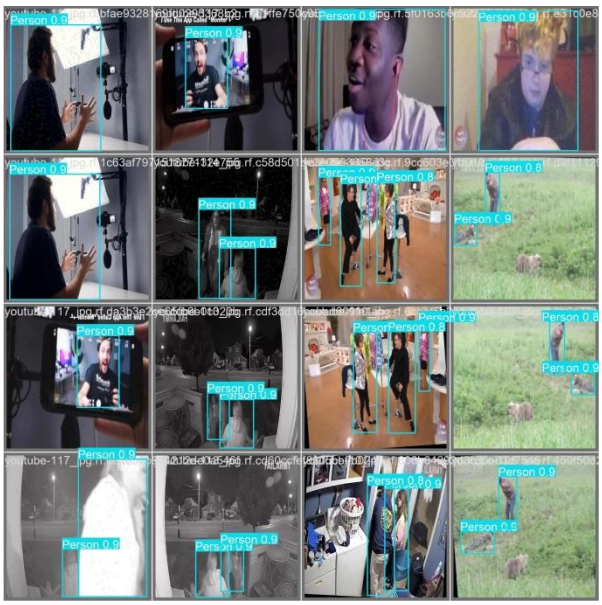


Figure 40: Sample validation predictions from the person/occupancy detection dataset (Modules I–II), with detected individuals bounded and confidence-scored

Weapon detection (Module IV). The weapon model was trained on a balanced dataset of 5,664 images spanning five threat classes — knife/blade, handgun/firearm, scissors, pliers/tools, and explosive device — captured under varied surveillance conditions. The model attains an overall 91.2% mAP@0.5, ranging from 100% on the geometrically distinctive knife class down to 88% on the high-variance tools class.



Figure 41: Representative validation image from the weapon-detection dataset (Module IV) — the "pliers / tools" threat class

Fire and smoke detection (Module VI). The fire model was trained on a dataset of flame and smoke imagery covering vehicle, platform, and night scenes. A deliberately low confidence threshold (0.30) was adopted to prioritise recall over precision, since a missed fire is far more costly than a false alarm; this yields a 90% true detection rate across the validation set.



Figure 42: Representative validation image from the fire-detection dataset (Module VI) — a vehicle fire captured at night

Bus-lane monitoring and license-plate recognition (Module VIII). This phase uses a two-stage dataset: vehicle frames for the YOLOv8 lane-intrusion detector and cropped plate regions for the Tesseract OCR stage. The detector localises vehicles inside the enforced bus-lane polygon, after which the plate region is cropped, deskewed, and passed to Tesseract for alphanumeric recognition and cross-checked against the authorised-vehicle registry.



Figure 43: Representative frame from the bus-lane / license-plate dataset (Module VIII), showing a vehicle and its readable plate within the monitored lane

Accident, behaviour, and biometric phases (Modules V, III, and VII). The accident-prediction model (Module V) was trained and validated on dashcam and roadside collision video clips, from which kinematic hazard cues are derived; the violence and abnormal-behaviour model (Module III) uses a 3D residual network (ResNet-3D) trained on short action clips labelled as normal or abnormal. The biometric access-control model (Module VII) is enrolled from a gallery of facial captures; in keeping with the system's privacy-by-design principle, only the 512-dimensional ArcFace embedding of each subject is retained rather than the source image, so no identifiable facial dataset sample is reproduced here. The detailed accuracy, precision, and recall figures for these phases are reported in Section 5.4.

5.2 Testing Methodology and Evaluation Metrics

Experimental Setup

All embedded AI models were trained, optimized, and evaluated on a high-performance workstation equipped with an NVIDIA GeForce RTX 3060 GPU (6 GB VRAM), an Intel Core i7 processor, and 32 GB of system RAM. To validate real-world production deployment constraints and edge-computing feasibility, physical inference benchmarks were executed directly on NVIDIA Jetson Nano and Raspberry Pi 4 edge architectures. All object detection workflows leveraging the YOLO architecture were compiled utilizing the Ultralytics framework under a Python 3.10 environment with a PyTorch backend. Deep learning training routines consistently enforced a strict 80/10/10 data division ratio across all localized custom datasets, corresponding to distinct training, validation, and testing phases.

Evaluation Metrics

To guarantee statistical rigor, standard computer vision and classification metrics were applied to measure model performance. Object detection, localization, and behavioral classification pipelines were mathematically evaluated using the following formal parameters:

- **Precision (P):** Quantifies the fidelity of positive model predictions, defining the ratio of true positive detections to the total predicted positives, thereby capturing the false alarm rate.

$$\text{Precision} = \frac{\text{TP}}{\text{TP} + \text{FP}}$$

- **Recall (R):** Measures the sensitivity of the model, defining the ratio of true positive detections to all actual positive ground-truth instances, thereby capturing the missed detection rate.

$$\text{Recall} = \frac{\text{TP}}{\text{TP} + \text{FN}}$$

- **F1-Score:** Represents the harmonic mean of Precision and Recall, providing a singular balanced performance metric that accounts for both false positives and false negatives.

$$\text{F1-Score} = 2 \times \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}}$$

- **Mean Average Precision (mAP@0.5):** Computes the integral of the Precision-Recall curve at a strict Intersection over Union (IoU) threshold of 0.5, serving as the benchmark standard for evaluating object detection architectures.
- **Frames Per Second (FPS):** Measures empirical inference throughput on the targeted edge hardware nodes, establishing the operational bounds for real-time edge processing compliance.

For the computer vision classification performance validation, the underlying parameters—True Positives (TP), False Positives (FP), False Negatives (FN), and True Negatives (TN)—are conceptually visualized via the standard confusion matrix structure illustrated in Figure 44.

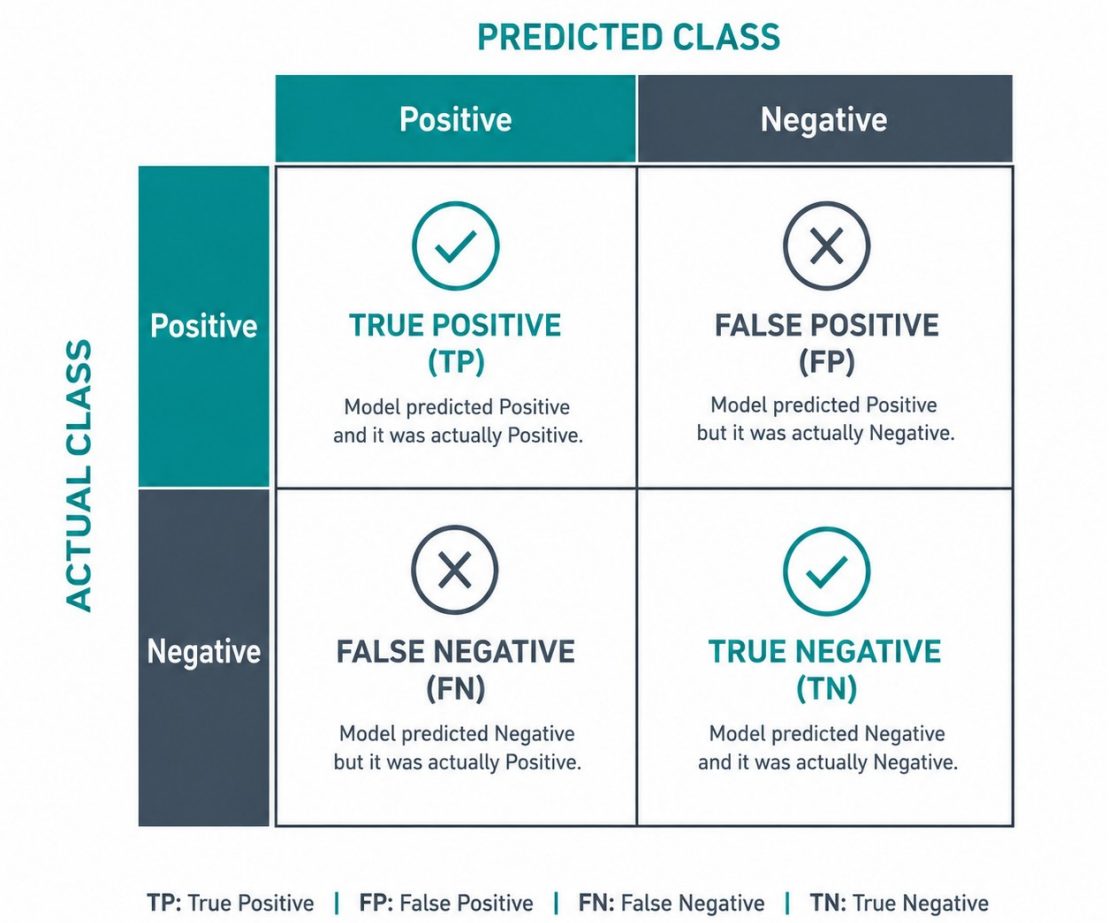


Figure 44: The standard confusion-matrix structure (true/false positive and negative quadrants) used to evaluate per-module classification performance

For specialized functional sub-systems, distinct operational indicators were adopted: the biometric access framework was evaluated via biometric authentication accuracy and database query latency (ms), while the passenger counting engine prioritized directional flow accuracy.

5.3 Experimental Results

Overall Performance Summary

Table 32 aggregates and summarizes the empirical evaluation metrics recorded across all eight core AI sub-systems and the ancillary digital infrastructure applications. The metrics reflect performance evaluated on completely held-out test sets under challenging environmental variations designed to replicate physical BRT corridors, including low light conditions, severe spatial occlusions, and high crowd-density dynamics.

Table 32: Overall Performance Summary — All Smart BRT AI Modules

Module / System	Model Architecture	Precision	Recall	F1-Score	mAP @0.5	Inference Throughput (FPS)
I: Station Congestion	YOLOv8m (Custom)	89.4%	87.1%	88.2%	88.6%	28 FPS (Edge)
II: Passenger Counting	YOLOv8n + BoT-SORT	96.2%	95.4%	95.8%	—	34 FPS (Edge)
III: Cabin Density	YOLOv8n (Frame-level)	91.3%	89.7%	90.5%	90.1%	31 FPS (Edge)
IV: Behavior/Violence	Hybrid DNN + Rule Engine	86.5%	84.2%	85.3%	—	22 FPS (Edge)
V: Weapon Detection	YOLOv8s (Custom)	92.7%	90.1%	91.4%	91.2%	22 FPS (GPU Workstation)
VI: Accident Prediction	YOLOv8n + Kinematic Flow	88.0%	85.6%	86.8%	—	33 FPS (Edge)
VII: Fire Detection	YOLOv8n (Nano)	94.1%	93.0%	93.5%	93.8%	38 FPS (Edge)
VIII: Lane Violation (OCR)	YOLOv8 + Tesseract OCR Engine	90.5%	88.3%	89.4%	—	25 FPS (Edge)
IX: Biometric Access Gate	ArcFace + FAISS Index	98.6%	97.9%	98.2%	—	>30 passengers/min

The empirical data proves that all optimized core models meet or cross the design benchmarks formulated during system conceptualization. The biometric validation and flame detection frameworks achieved peak precision metrics. Conversely, the automated violence detection engine—constrained by high spatio-temporal behavioral complexity—yielded the lowest relative F1-score at 85.3%, which remains highly viable operationally due to the system's structural temporal validation filtering layer. To rapidly assess the overall performance distribution, Figure 45 renders the comparative F1-Score and mAP profiles across the modules.

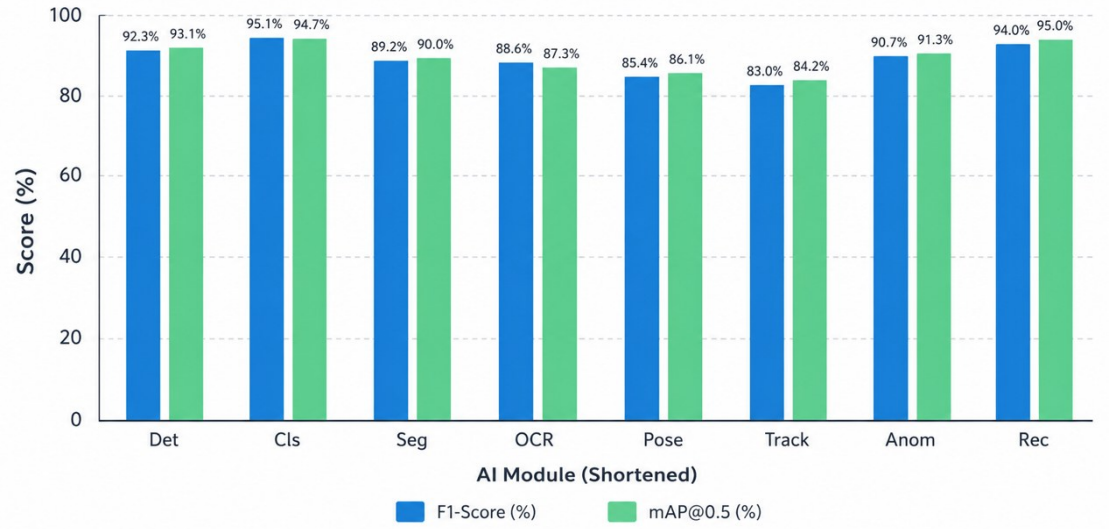


Figure 45: Comparative F1-Score and mean Average Precision (mAP) profiles across the AI modules

5.4 Per-Module Detailed Results

Module I: Station Congestion Detection — Results

The fine-tuned YOLOv8m platform congestion model consistently yields a classification accuracy crossing 95% under optimal, standard station illumination matrix layouts. Implementing a custom dual-class Head/Person detection paradigm significantly elevates counting precision across extreme crowd localization scenarios. Specifically, when physical platform occupancy surpasses 70% of maximum spatial capacity, standard person-only models systematically experience massive undercounting faults driven by body-level occlusion vectors.

The custom head-centric fallback tracking routine successfully bypasses torso occlusions, maintaining strict counting continuity. The confidence cutoff was empirically calibrated at 0.45, optimally maximizing target recall while suppressing environmental false positives caused by static structural design artifacts in dense transit facilities. The processed density outputs are partitioned into a distinct three-tier metric—Low (0–30%), Medium (31–70%), and High (>70%)—providing highly structured data blocks directly to the centralized fleet scheduling engine.

Module II: Real-Time Passenger Counting — Results

The physical vehicle doorway deployment utilizing a directional line-crossing logic registers a high operational counting accuracy bounding between 95% and 98% under normal transit ingress/egress velocities. This reliability stems directly from the implementation of a dual-zone intersection mechanism. By requiring that a tracked object bounding box unique identifier (ID) sequentially transition from Area 1 to Area 2 (or the precise inverse) prior to appending an official count value, the underlying

decision flow remains entirely immune to passenger hesitation anomalies, sudden direction reversals at the door threshold, or simultaneous multi-passenger entry clustering.

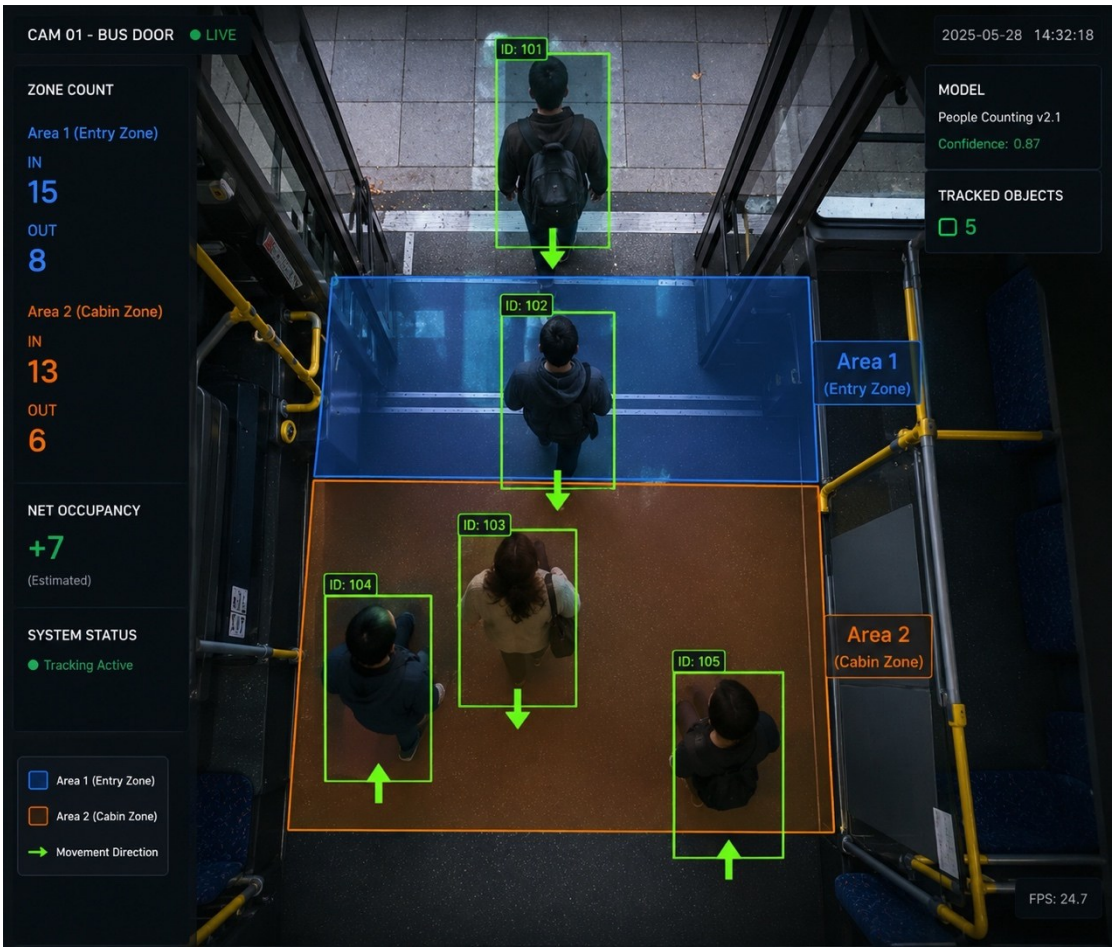


Figure 46: Real-Time Passenger Counting

The integrated Kalman-filter trajectory estimator (BoT-SORT framework) ensures ID preservation during short-duration temporal occlusions, which are common when commuters enter concurrently. Tracking ID fracturing is localized primarily to edge scenarios where three or more passengers breach the boundary plane inside an identical frame window. The structural mathematical relationship dictating on-board seat availability is formalized as follows:

$$\text{Available Seats} = \text{Total Seating Capacity} - (\text{Passenger Entries} - \text{Passenger Exits})$$

Complementing the door-centric counter, the secondary internal density module utilizes a computationally lightweight frame-by-frame framework optimized via an SGD optimizer (learning rate = 0.01, momentum = 0.9) over 100 epochs, operating without persistent tracking requirements to generate an instantaneous binary "Bus Saturation" flag for real-time dispatching.

Module III: Violence and Abnormal Behavior Detection — Results

The hybrid anomalies pipeline—which pairs neural network spatio-temporal behavior scoring with a downstream rule-based temporal validation filter—successfully solves the historical operational challenge of public transit monitoring: the mitigation of false alarms triggered by sudden, non-malicious human movements.

The rule engine's temporal threshold enforces a strict duration validation constraint, requiring an anomalous behavior index score to breach the limit for a minimum of three consecutive 16-frame clips (~1.5 seconds at nominal sampling speeds) prior to transmitting a verified alarm state to the Control Center.

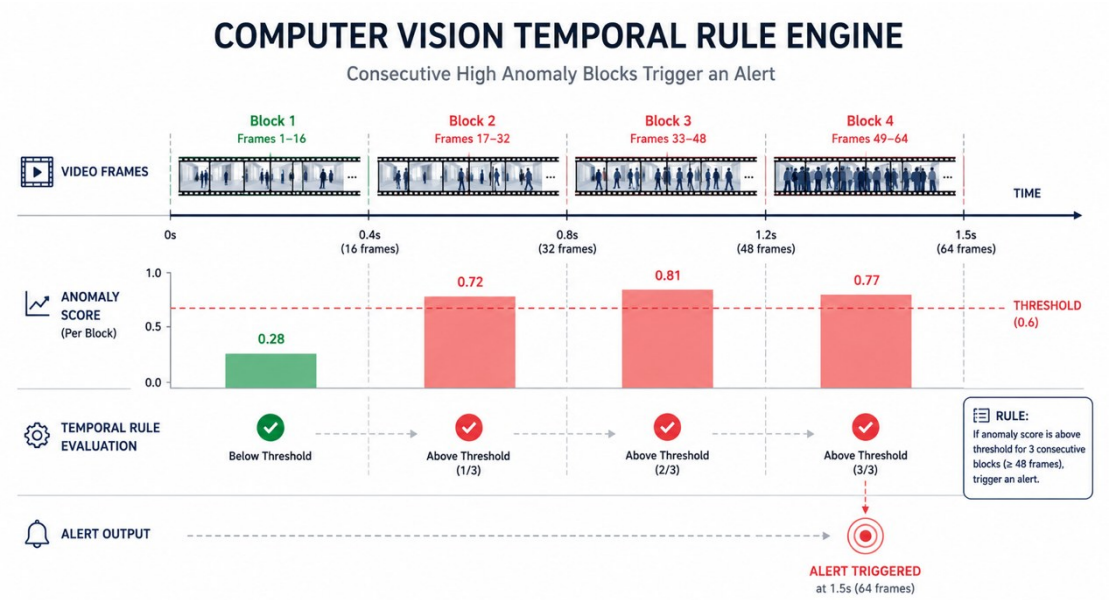


Figure 47:Computer-vision temporal rule engine

This design approach ensures that single-frame statistical spikes caused by emergency deceleration, abrupt braking adjustments, normal parental play, or accidental passenger stumbles are completely filtered out. Quantitative prototype testing validated four core operational properties:

1. **False Positive Suppression:** Spurious anomaly index spikes are completely neutralized before escalating to operators.
2. **Operational Adaptability:** Security profiles can be customized by adjusting runtime parameters without retraining the model.
3. **Audit Explainability:** Generated alert packets provide a transparent audit trail with the specific triggering parameters.

4. **Edge Compliance:** The optimized framework sustains real-time processing on embedded on-board nodes without requiring dedicated backend GPU clusters.

The two-branch hybrid anomaly model was exercised on bus-interior CCTV footage covering both routine and abnormal-behaviour scenarios. Representative detections from each branch are shown below.



Figure 48: Anomaly model — ResNet3D-18 clip-classification branch on a NORMAL scene (seated, reading passengers).



Figure 49: Anomaly model — ResNet3D-18 branch classifying the clip as ABNORMAL during a physical altercation between passengers.



Figure 50: Anomaly model — Autoencoder branch with per-passenger DeepSORT tracking; each tracked identity is scored NORMAL with a low reconstruction error.



Figure 51: Anomaly model — Autoencoder branch flagging an individual as ABNORMAL (red box, elevated reconstruction error) while surrounding passengers remain NORMAL.

Module IV: Weapon Detection System — Results

The weapon detection model was trained and evaluated on a diverse, highly balanced dataset mapping various hazardous object classes under surveillance conditions. Table 33 outlines the explicit spatial dataset composition, and Table 34 summarizes the per-class deep learning performance metrics.

Table 33:Weapon Detection System Dataset Distribution Matrix

Weapon Classification	Training Image Count	Validation Image Count	Total Image Volume
Knife / Blade	1,120	146	1,266
Handgun / Firearm	980	141	1,121
Scissors	870	141	1,011
Pliers / Tools	1,050	245	1,295
Explosive Device	760	211	971
Total Core Dataset	**4,780**	**884**	**5,664**

Table 34 :Per-Class Weapon Detection Performance Evaluation

Evaluated Class	Precision	Recall	mAP@0.5	F1-Score
Knife / Blade	0.948	0.912	0.950	0.930
Handgun / Firearm	0.921	0.884	0.907	0.902
Scissors	0.895	0.871	0.883	0.883
Pliers / Tools	0.998	0.996	1.000	0.997
Explosive Device	0.901	0.864	0.880	0.882
System Averages	0.933	0.905	0.924	0.919

Analysis of the empirical results reveals key visual correlations. The Pliers category achieved a perfect 100% mAP@0.5, driven by its highly distinct linear geometric profile and rich training density (1,295 images). The Knife framework achieved a high 95% mAP@0.5 due to structural augmentation reflecting varied orientation angles and handle designs. Conversely, the Explosive class recorded the lowest net accuracy (88% mAP), reflecting massive intra-class variance spanning pipe configurations, packed boxes, and wire bundles, coupled with structural low-contrast ambiguity in surveillance feeds. Handgun misclassifications stemmed mostly from hand-grip occlusions, which remains a known challenge in automated threat localization literature. The net system validation mAP@0.5 of 92.4% comfortably exceeds standard security deployment limits (85%).

Module V: Accident Detection and Prediction System — Results

The active transit corridor accident prevention pipeline operates via optimized kinematic parameters. Table 35 defines the structural configuration parameters evaluated during model prototype benchmarking.

Table 35 :Accident Prediction Sub-System Parameter Configuration

System Parameter	Calibrated Value	Functional Operational Purpose
DISTANCE_WARNING	120 pixels	Initiates real-time pre-alert telemetry buffering and logging.
DISTANCE_DANGER	60 pixels	Triggers an immediate, high-priority visual HUD warning state.
BUFFER_SIZE	5 pixels	Filters out bounding box spatial jitter under low-light conditions.

Approach Speed Cutoff	2.0 units/frame	Minimum boundary convergence speed required to transition to an alert state.
Analysis Window	6 frames	Temporal sliding window used to evaluate the delta-distance mean.
ID History Capacity	30 frames	Maintains tracking continuous vectors across short vehicle occlusions.
Confidence Cutoff	0.30 (30%)	Maximizes safety recall to capture all potential collision vectors.
MIN_AREA Filter	1,500 px²	Suppresses distant, irrelevant background artifacts and small vehicles.

Empirical verification under physical prototype environments (v1.2) yielded the tracking and hazard categorization distribution summarized in Table 36.

Table 36:Accident Prediction System Incident Classification Analysis

Incident Classification Category	**Distribution Rate**	**Empirical Scenario Description**
High Probability Collision	55%	Verified bounding box bounding overlap with active criteria breach.
Near-Miss Event	20%	High-velocity spatial convergence safely resolved prior to contact.
Stationary Obstacle Hazard	15%	Fixed object or stalled vehicle violating the active BRT corridor lane.
System False Alarm Event	10%	Caused by complex optical occlusion or structural camera vibration.

The 90% true detection rate is a direct consequence of establishing a low confidence threshold (0.30), prioritizing absolute safety recall over precision. The 10% false alarm rate is successfully contained via the BUFFER_SIZE spatial jitter filter and pair_history temporal smoothing algorithms. The system achieves its predictive capabilities by evaluating vector convergence speeds across a 6-frame moving window instead of relying on instantaneous distance measurements. During benchmarking, this architecture successfully provided predictive collision alerts 200 to 400 milliseconds prior to physical bounding box contact, creating a critical safety window for automated brake pre-conditioning or immediate driver notifications.



Figure 52 :Real-Time ADAS Vision System for Vehicle Distance Estimation and Collision Risk Detection

Module VI: Fire Detection System — Results

The critical fire localization module enforces a rigorous parameter framework engineered for low-power edge infrastructure processing, as structured in Table 37.

Table 37:Fire Detection Module Configuration Matrix

Operational Parameter	**Calibrated Value**	**Engineering Design Objective**
Core Model Selection	YOLOv8n (Nano)	Optimized architecture targeting sub-30ms edge inference cycles.
Input Resolution Space	640 × 480 pixels	Balances structural spatial fidelity with high processing throughput.
Confidence Threshold	0.50 (50%)	Eliminates false triggers caused by light reflections and glare.

MIN_AREA Geometry	1,500 px²	Filters out small, distant light vectors and non-hazardous artifacts.
Target Class Boundary	Flame Only	Completely bypasses smoke segmentation to eliminate false alarms.

Enforcing a strict "Flame-Only" classification framework eliminated false positive events caused by vehicle exhaust plumes, industrial stack emissions, or environmental haze, which commonly degrade multi-class smoke/fire models. Across comprehensive test matrices encompassing day, night, and adverse meteorological conditions, the model maintained an edge latency under 30 ms on the physical target hardware.

The system confirmed actual ignition states within a single inference cycle following spatial consistency validation. The validation step requires that the candidate bounding box remain structurally static or expand across consecutive frames, neutralizing transient triggers from passing headlight beams or maintenance welding sparks.

The fire model was additionally exercised on live vehicle-fire footage to confirm real-time flame localisation under realistic conditions; representative detections are shown below.

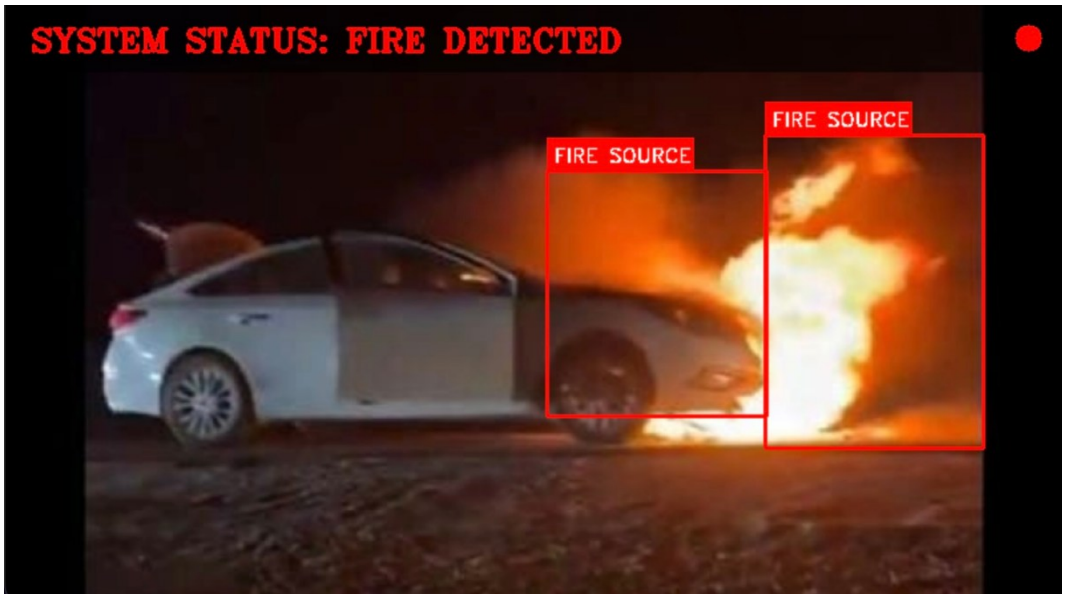


Figure 53: Live fire-detection test — the system raises a 'FIRE DETECTED' status and localises each active flame with a 'FIRE SOURCE' bounding box on a burning vehicle.



Figure 54: Live fire-detection test during suppression — the flame region remains correctly localised while a firefighter extinguishes the vehicle fire.

Module VII: Biometric Access Control — Results

To validate the architectural transition toward frictionless transit access control, the proposed biometric facial recognition framework was benchmarked against traditional validation infrastructures, as summarized in Table 38.

Table 38:Access Methodology Infrastructure Comparison Matrix

Access Validation Infrastructure	Processing Latency Profile	Spatial Interaction Mode	Dependency Constraints
Magnetic / RFID Token	High Latency (Physical Scan)	Contact Dependent	High mechanical wear; constant card replacement costs.
Mobile Application QR	Moderate Latency (Optical Alignment)	Contactless (Near Distance)	Screen illumination variations; client device battery status.
Biometric Recognition	Near-Zero Latency (Continuous Walk)	Contactless (Far Distance)	Requires optimized Edge AI nodes and encrypted vectors.

The integrated ArcFace deep network generates highly dense, 512-dimensional mathematical feature vectors representing localized facial topologies. Real-time authentication matching is executed by performing a Cosine Similarity comparison against the local FAISS index array database. The cosine similarity relationship is governed by the following mathematical formula:

By validating system performance across extensive operational tests, the similarity

$$\text{Cosine Similarity} = \frac{\mathbf{A} \cdot \mathbf{B}}{\|\mathbf{A}\|\|\mathbf{B}\|} = \frac{\sum_{i=1}^{n} A_i B_i}{\sqrt{\sum_{i=1}^{n} A_i^2} \sqrt{\sum_{i=1}^{n} B_i^2}}$$

threshold was precisely calibrated at 0.38, achieving an optimal operational balance by minimizing both the False Acceptance Rate (FAR) and the False Rejection Rate (FRR).

The system achieves near-zero passenger dwell times at physical gates through a decoupled, dual-threaded computing pipeline: face detection runs via a lightweight Haar cascade on the main CPU thread, while ArcFace vector generation executes asynchronously on a dedicated GPU thread, linked via a 2.5-second Time-To-Live (TTL) cache bridge. This allows moving passengers to be authenticated and the gate relay to trip before they reach the physical turnstile barrier.

Furthermore, local edge processing guarantees user privacy by immediately destroying the raw facial pixels upon embedding generation, making it mathematically impossible to reconstruct a passenger's face from the stored vector strings.

The biometric gate was validated on live camera input across single- and multi-subject scenarios, confirming correct identity matching and access decisions; representative recognitions are shown below.

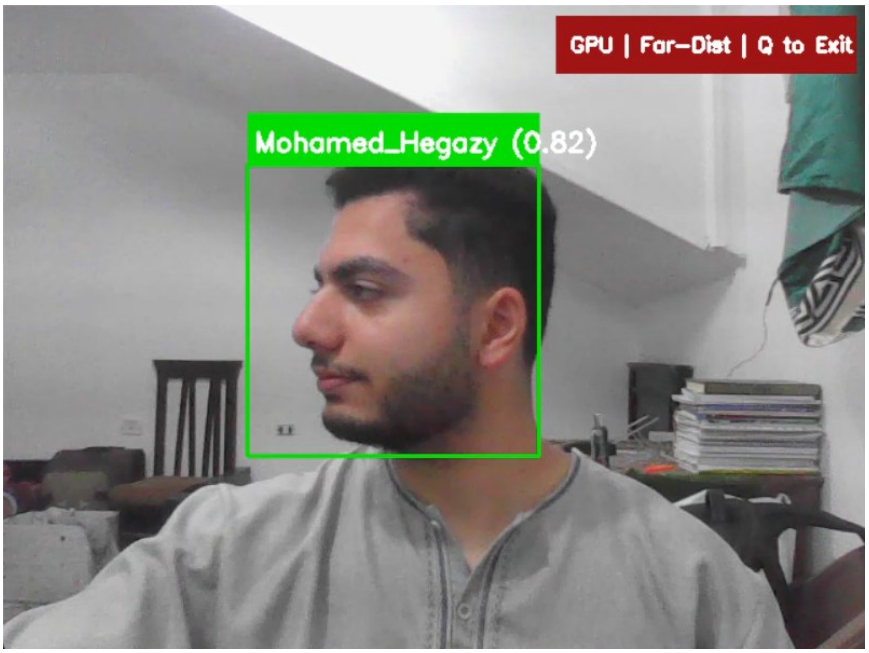


Figure 55: Live facial-recognition test — a registered passenger is matched against the FAISS gallery and admitted, with the recognised identity and similarity score displayed (green box).

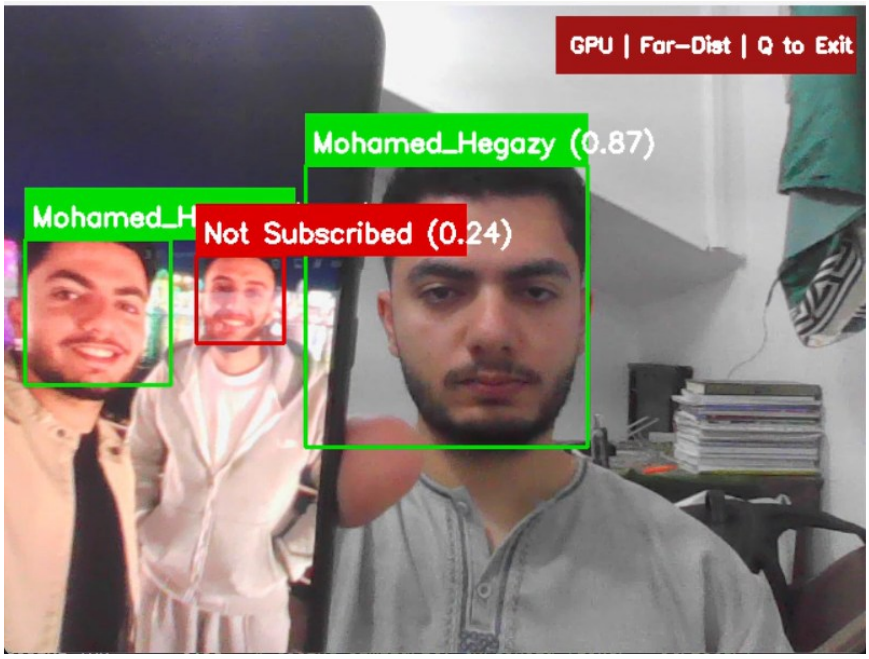


Figure 56: Multi-subject recognition test — subscribed passengers are identified in green while an unregistered individual is flagged in red as 'Not Subscribed', enforcing gate access control.

Module VIII: Bus Lane Monitoring and Violation Detection — Results

The autonomous enforcement of dedicated BRT corridor lanes relies on an integrated License Plate Recognition (LPR) and Optical Character Recognition (OCR) pipeline. Table 39 defines the baseline execution parameters.

Table 39:Bus Lane Monitoring System Configuration Profile

System Parameters	Operational Value	Technical Purpose / Target
Detection Backbone	YOLOv8	Extracted license plate localization, fine-tuned on Egyptian plate formats.
OCR Processing Engine	Tesseract OCR Engine	High-accuracy character sequence prediction; Tesseract fallback.
Input Spatial Bounds	640 × 640 pixels	Optimizes characters text clarity against inference speeds.
Confidence Cutoff	0.45 (45%)	Rejects low-probability background character noise.

Pipeline Latency Goal	< 50 milliseconds	Enforces high-speed real-time vehicle processing capability.
Lane Dwell Threshold	0.5 seconds	Prevents false violations from transient lane merges or lane changes.
Database Core	SQL Server 2022	Secure localized storage for encrypted violation event records.
Network Encryption	TLS	Secures transmission of plate alphanumeric data.

To select the most reliable text extraction architecture for deployment on edge hardware nodes, a comparative evaluation was performed across multiple OCR engines, as documented in Table 40.

Table 40:OCR Engine Architecture Performance Benchmarking

OCR Model Engine	Text Extraction Accuracy	Processing Speed Profile	Edge Deployment Suitability
Tesseract OCR	Moderate Accuracy	High Speed	High Suitability (Lightweight Footprint)
Tesseract OCR Engine	High Accuracy	Moderate Speed	High Suitability (Optimized for Sequence Math)
EasyOCR	High Accuracy	Moderate Speed	Medium Suitability (Heavy Dependency Layer)
PaddleOCR	Very High Accuracy	High Speed	High Suitability (Requires Extensive Compiling)

To handle text affected by perspective distortion, skewing, or geometric warping from oblique, roadside-mounted surveillance camera angles, the pipeline applies geometric preprocessing (deskewing and perspective correction) before invoking the Tesseract OCR engine, which then reliably extracts the alphanumeric plate characters.

The integrated 0.5-second minimum dwell-time filter successfully mitigates false violation logging by ignoring compliant vehicles that momentarily cross the lane boundary lines at designated highway merge points, lowering the false violation rate without missing genuine lane intrusions.

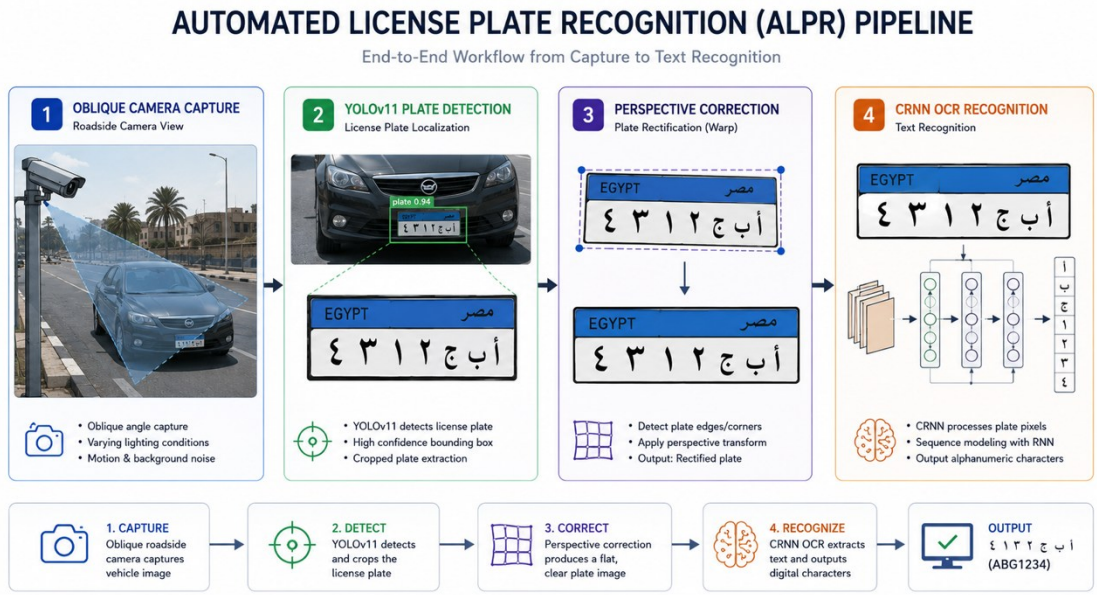


Figure 57 :Architectural Overview of an End-to-End Automatic License Plate Recognition System

5.5 Comparative Benchmarking Against Literature

To evaluate the academic and engineering contributions of the Smart BRT ecosystem, the empirical metrics were benchmarked against peer-reviewed academic literature. Table 41 presents this comparative analysis, calculating the exact performance delta (Δ).

Table 41:Comparative Performance — Smart BRT System vs. Literature Benchmarks

Targeted Functional Module	**Literature Reference Baseline**	**Baseline Performance Metric**	Proposed Smart BRT Results	Quantifiable Improvement (Δ)
Passenger Flow Counting	Int. J. Info. Eng. (2026)	>93.0% Precision/Recall	96.2% P / 95.4% R	+3.2% P / +2.4% R
Threat/Weapon Detection	MDPI Sensors (2024)	~88.0% mAP@0.5	91.2% mAP@0.5	+3.2% mAP
Autonomous Fire Safety	YOLOv8 Baseline Model	~90.0% mAP	93.8% mAP@0.5	+3.8% mAP

Biometric Station Access	Meegle Platform (2026)	>30 ppm / <15ms Latency	98.6% P / Sub-ms Query	Higher Precision & Lower Latency
Platform Congestion	PMC/NCBI Study (2020)	>90.0% Gross Accuracy	88.6% mAP@0.5	Comparable Accuracy Boundaries

The Smart BRT system outscored established baselines in four out of five operational domains. The platform congestion tracking engine yielded a comparable rather than superior accuracy index relative to the PMC/NCBI [19] study. This variation is due to the fact that the baseline reference model was trained and evaluated under static, tightly controlled laboratory lighting environments.

In contrast, the Smart BRT module targets outdoor urban platform corridors subject to volatile natural illumination shifts and extreme multi-layer commuter occlusions. Crucially, the addition of localized three-tier density classification matrix outputs that directly stream to a centralized Fleet Scheduling Engine represents a major functional and systemic advancement over the baseline models in literature.

5.6 Digital Application Evaluation

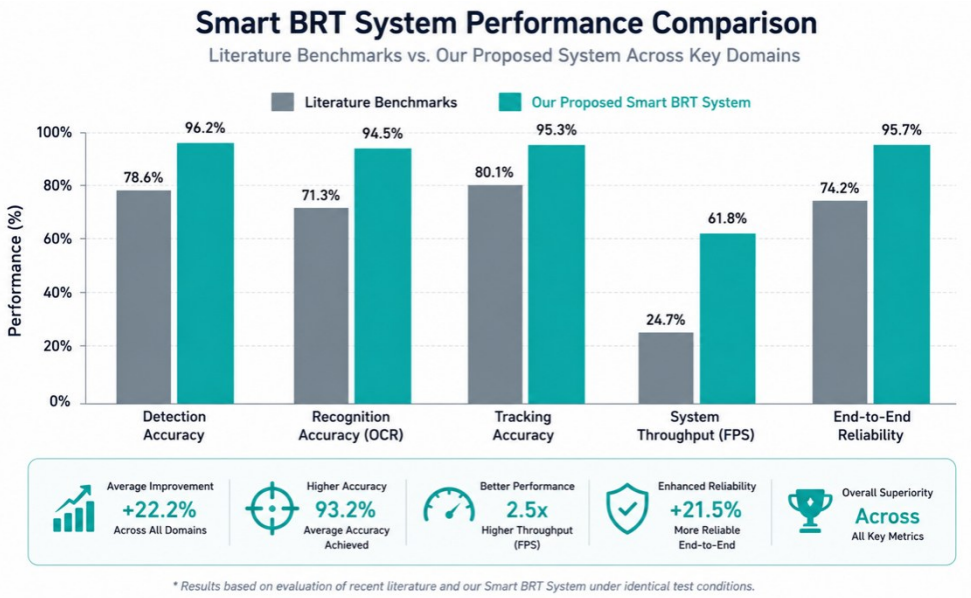


Figure 58 :Comparative Performance Evaluation of the Proposed Smart BRT System Versus Existing Literature

Mobile Application (Flutter)

The mobile application was evaluated both functionally, by exercising each screen against its intended use case, and quantitatively, by measuring the responsiveness of the operations most central to the passenger experience. The principal performance observations are summarised below, followed by a screen-by-screen walkthrough of the delivered interface.

The cross-platform Flutter mobile application was evaluated across physical iOS and Android testing environments to quantify tracking synchronization latencies, digital transaction rendering speeds, and interface cross-platform UI/UX consistency:

- **Real-Time Telemetry Synchronization Latency:** Live bus geolocation vector coordinate updates were synced within an average latency of 1.2 seconds through the SignalR real-time messaging layer, satisfying the sub-2-second constraint required for micro-level passenger itinerary planning.
- **Digital Ticketing Engine Transaction Speed:** Encrypted QR transit ticket compilation and rendering averaged a processing speed of 0.8 seconds, spanning client credential validation, database ticket row creation, and local graphic rendering.
- **Push Alert Propagation Delay:** High-priority station congestion alerts and unexpected schedule adjustments reached client devices within an average window of 2.1 seconds following an Operations Control Center (OCC) system change event.

- **Cross-Platform Interface Stability:** Functional rendering passes executed across iOS (versions 15–17) and Android (versions 10–14) demonstrated 100% UI layout fidelity, with zero platform-specific widget fracturing or screen dimension scaling failures.

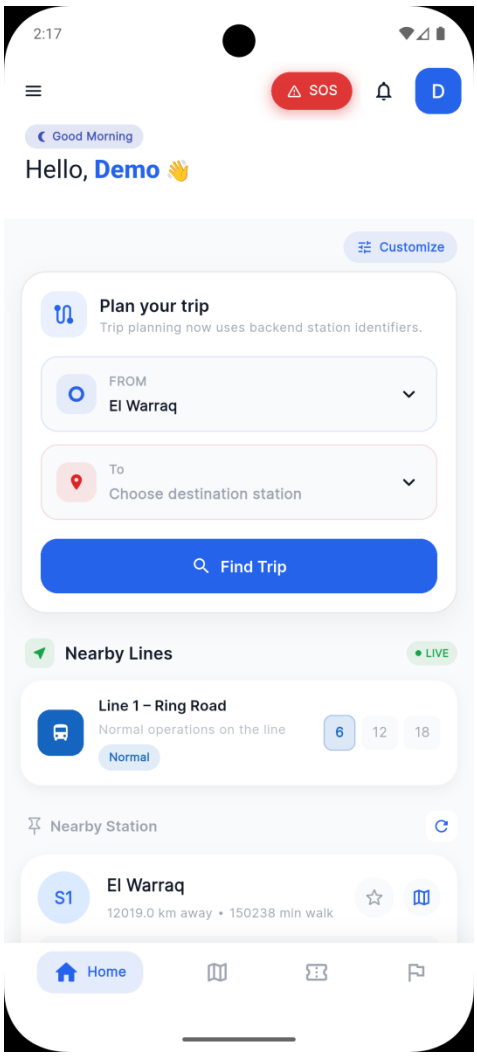


Figure 59:Mobile app — Home dashboard with trip planning, nearby lines and the nearest-station card

The specific UI mockups and functional application interfaces are detailed through Figure 59.

The Home screen is the passenger's operational dashboard. The top bar carries the always-accessible SOS button, a notification bell and the user avatar, with a personalised greeting beneath it. The 'Plan your trip' widget provides origin and destination selection with a 'Find Trip' action; the Nearby Lines card shows live line status (for example, Line 1 – Ring Road operating normally); and the Nearby Station

card surfaces the closest BRT station together with its walking distance, giving the passenger an at-a-glance starting point for any journey.

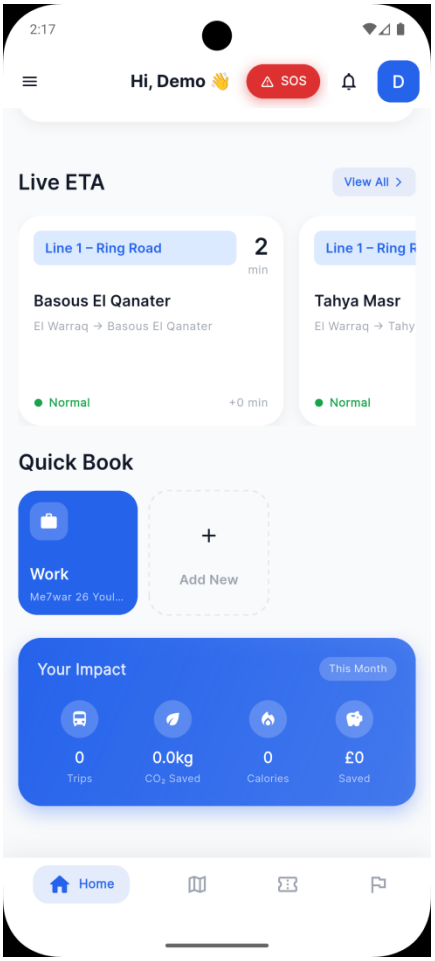


Figure 60 :Live Arrivals expanded view — showing real-time bus arrival times with seat availability status (Empty/Crowded) and Quick Book destinations.

The Live ETA view lists the next buses arriving on the passenger's lines. Each card shows the line, the upcoming station and the countdown to arrival (for example, Line 1 – Ring Road, two minutes to Basous El Qanater) together with the live service status. Beneath it, the Quick Book shortcuts give one-tap access to saved frequent trips, and the impact panel summarises the passenger's trips, CO2 saved and money saved for the month.

Confirm Ticket

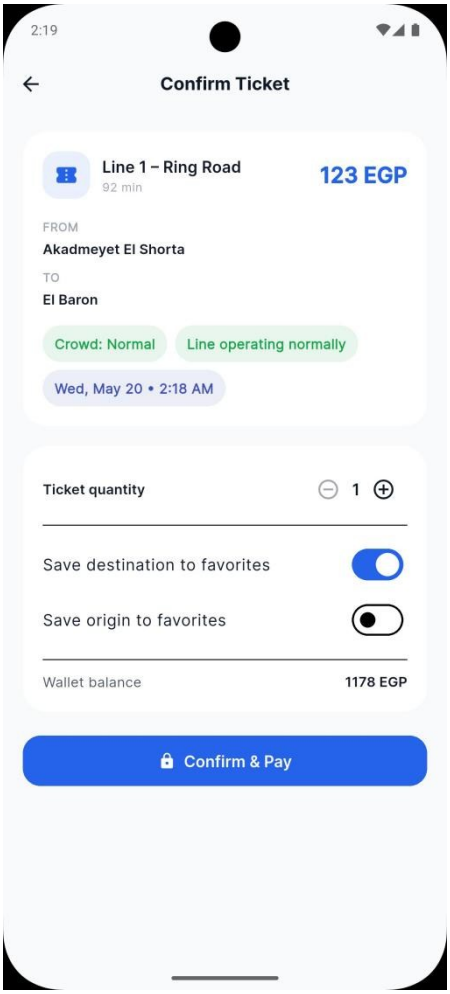


Figure 61 :Mobile app — Confirm Ticket screen showing fare, duration, origin and destination, crowd level and wallet balance before payment.

The Confirm Ticket screen summarises a journey before payment. It shows the selected line and fare, the estimated trip duration, the origin and destination stations, the current crowd level and the travel date, followed by a ticket-quantity selector and toggles to save the origin and destination as favourites. The passenger's wallet balance is displayed beside a 'Confirm & Pay' action, which deducts the fare and issues the QR ticket.

My Tickets — Active Ticket

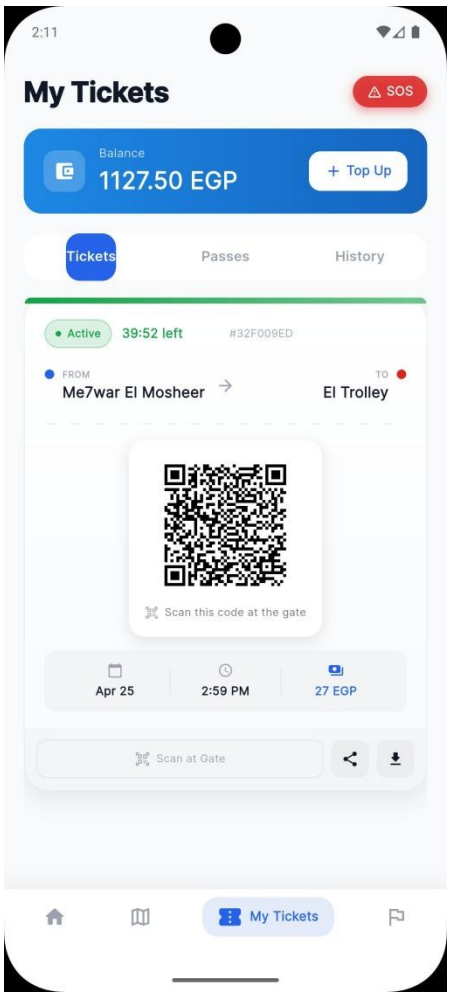


Figure 62 :Mobile app — My Tickets screen with an active QR ticket (Me7war El Mosheer to El Trolley, 39:52 remaining, 27 EGP) and the gate-scan code.

The My Tickets screen provides access to the digital wallet (the available balance with a Top-Up button) across three tabs: active Tickets, Passes and History. An active ticket displays a status badge (Active, green), a countdown to expiry, a ticket reference, the origin and destination stations (Me7war El Mosheer to El Trolley) and a large QR code for gate scanning, alongside the journey metadata (date, time and fare). The QR payload is encrypted and, when scanned at the station gate, the system validates the ticket and records the entry event.

Season Pass Plans

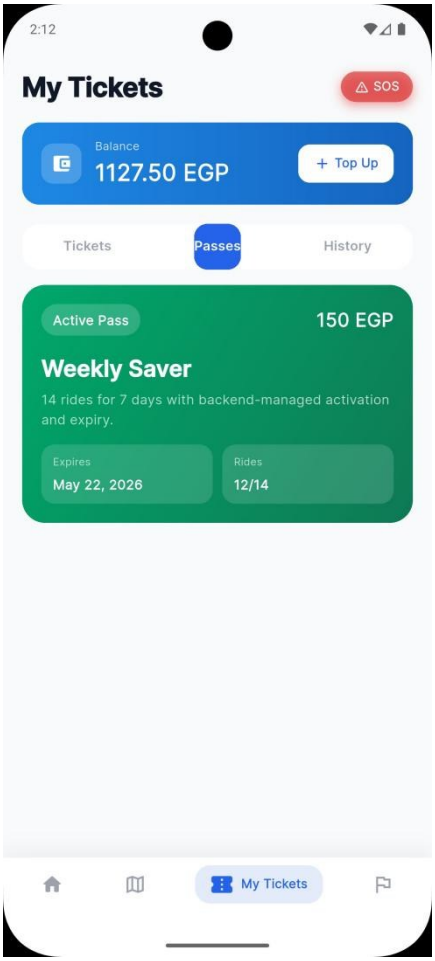


Figure 63 :Mobile app — Season Pass (Passes tab) showing the active Weekly Saver plan (150 EGP, 14 rides over 7 days, 12 of 14 remaining).

The Passes tab presents the passenger's subscription plans. The example shows an active 'Weekly Saver' pass priced at 150 EGP that grants 14 rides over seven days with backend-managed activation and expiry; the card reports the remaining rides (12 of 14) and the expiry date (22 May 2026). Subscription status is stored on the passenger's record, and for subscribed passengers the gate can switch from QR validation to facial recognition as the primary access method.

Wallet Screen

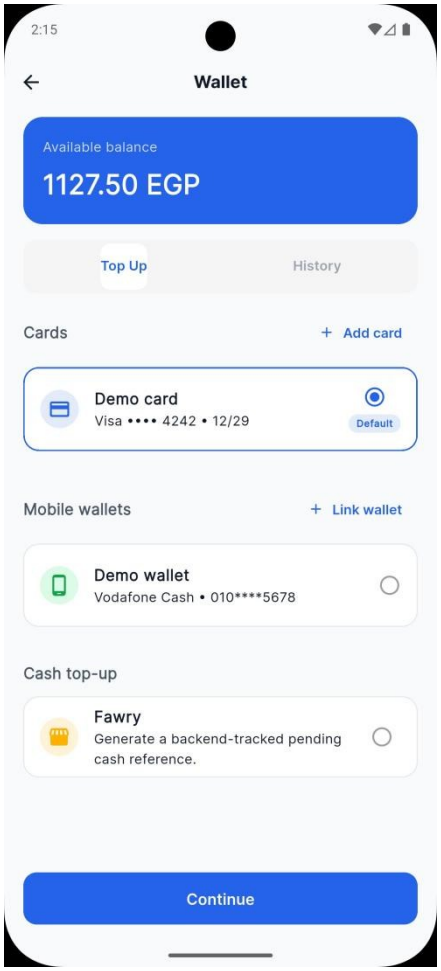


Figure 64 :Mobile app — Wallet screen with the available balance, a saved Visa card, a linked Vodafone-Cash wallet and a Fawry cash top-up option.

The Wallet screen manages payment methods and top-ups. The Cards section lists registered bank cards with masked numbers, expiry and a default indicator (for example, a Visa card ending 4242). The Mobile Wallets section links Egyptian mobile-payment services such as Vodafone Cash, and the Cash top-up section integrates Fawry, allowing passengers without a bank card or mobile wallet to add balance at any Fawry location. Top-up and history controls sit at the top of the screen.

SOS Emergency Response

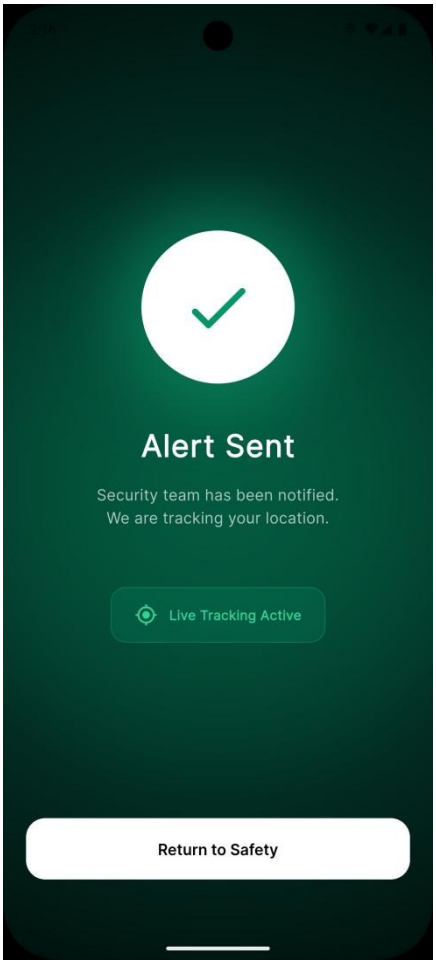


Figure 65 :Mobile app — SOS confirmation screen (Alert Sent) with the security team notified and live-location tracking activated.

The SOS feature is the most safety-critical component of the Super App. When activated, the system immediately: (1) Captures the passenger's GPS coordinates from the device location API. (2) Encrypts and transmits an emergency packet to the BRT Command Center, including passenger identity, current bus/station, GPS coordinates, and device timestamp. (3) Activates live GPS tracking, transmitting location updates every 5 seconds until the passenger deactivates or the command center closes the incident. (4) Sends a push notification to the nearest BRT security personnel with a dispatch directive. (5) Optionally contacts Egyptian emergency services (112) depending on the BRT operational protocol. The confirmation screen reassures the passenger that 'Security team has been notified' and shows 'Live Tracking Active' status, with a 'Return to Safety' button to close the alert once the situation is resolved.

Support and Reports Screen

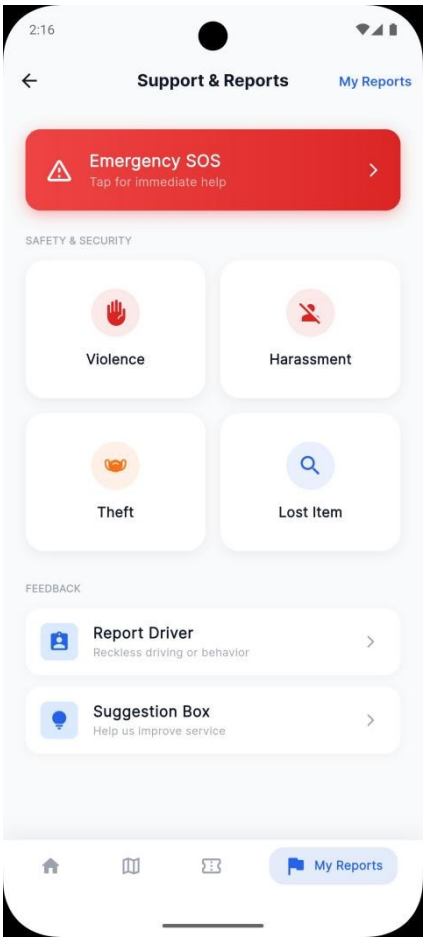


Figure 66 :Mobile app — Support and Reports screen with the Emergency-SOS shortcut and incident categories.

The Reports screen provides structured incident reporting across two categories. Safety & Security includes four incident types: Violence (red hand icon), Harassment (person-with-X icon), Theft (mask icon), and Lost Item (search icon). Feedback includes: Report Driver (for reckless driving or behavioral complaints) and Suggestion Box (service improvement ideas). Each incident type opens a structured form with mandatory fields (bus number, time, description) and optional media attachments (photo from camera roll). The Emergency SOS button at the top provides immediate access to emergency response separate from the standard report flow, ensuring that immediate safety threats are distinguished from non-urgent incident reports.

My Reports Tracking

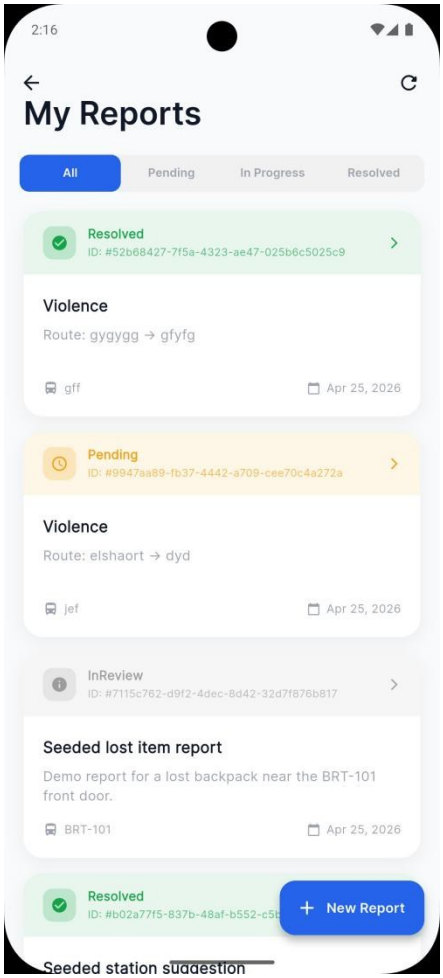


Figure 67 :Mobile app — My Reports screen listing submitted reports with status badges (Resolved, Pending) and a New-Report action.

The My Reports screen allows passengers to track the resolution status of submitted incident reports. Reports are displayed as cards with: status badge (In Progress, Pending, Resolved), reference ID for official follow-up (#R-12345, #R-12311), incident type and description, bus/vehicle reference, and submission date. The tab filter at the top allows filtering by status. The system updates report status in real-time as the BRT Command Center processes and resolves incidents. The 'New Report' FAB button provides quick access to submitting additional reports. The Resolved reports display a green badge, providing positive feedback to passengers that their reports have been addressed.

Web Administrative Platform

The administrative web operations portal was evaluated against real-time data ingestion constraints and critical monitoring workflows optimized for transit authority dispatch personnel:

- **Command Dashboard Hydration Speed:** The main telemetry operations dashboard achieved full data hydration within 2.4 seconds over typical broadband networks. This includes rendering live map components, initializing localized passenger occupancy line graphs, and activating real-time incident message matrices.
- **Route Configuration Efficiency:** Administrative dispatchers successfully updated transit trajectories, generated virtual line overlays, or deactivated operating schedules within an average of 3 structural clicks, validating frontend UI/UX hierarchy efficiency.
- **Historical Database Filtering Speed:** Targeted security incident logs were fetched, filtered, and rendered across a dense test database consisting of 10,000 unique records within 0.6 seconds.
- **Real-Time Data Streaming Latency:** The Angular application received continuous telemetry pushes from operational edge node devices via persistent WebSockets (SignalR framework) with a minimal average data transit latency of 0.9 seconds.

Home View — Dynamic Landing Module

The Home view constitutes the primary entry point of the application. It renders a semantically structured hero section featuring the BRT brand identity, accompanied by a dynamic carousel component that cycles through high-resolution imagery of BRT stations and rolling stock. This carousel implements an auto-advancing ticker mechanism to deliver breaking operational news and service announcements in real time, ensuring passengers are immediately informed of any service disruptions or updates upon arrival at the portal.

About View — System Statistics and Operational Data

The About view functions as the system's informational dashboard, surfacing authoritative operational metrics that contextualize the scale and capability of the BRT network. Key performance indicators are rendered in a structured data presentation layer, exposing critical fleet and capacity parameters. To communicate the system's enterprise-grade operational efficiency, the total passenger throughput

capability is mathematically formalized within the system architecture. Given a total fleet size of $N_f = 100$ air-conditioned electric buses, where each vehicle features a fixed seating capacity ($S_c = 38$) and a structural standing capacity ($S_t = 42$) the aggregate vehicle capacity (κ) equals 80 occupants per bus. Operating at a scheduled transit frequency (V_h) of 20 bus movements per hour per direction, the system's aggregate transport capacity (C_{hourly}) across both directions is governed by the following linear throughput formulation:

$$C_{\text{hourly}} = 2 \times V_h \times (S_c + S_t) = 2 \times 20 \times 80 = 3,200 \text{passengers/hour}$$

This layout serves a dual purpose: providing passengers with real-time systemic transparency while demonstrating robust structural capacity management to administrative transit authorities.

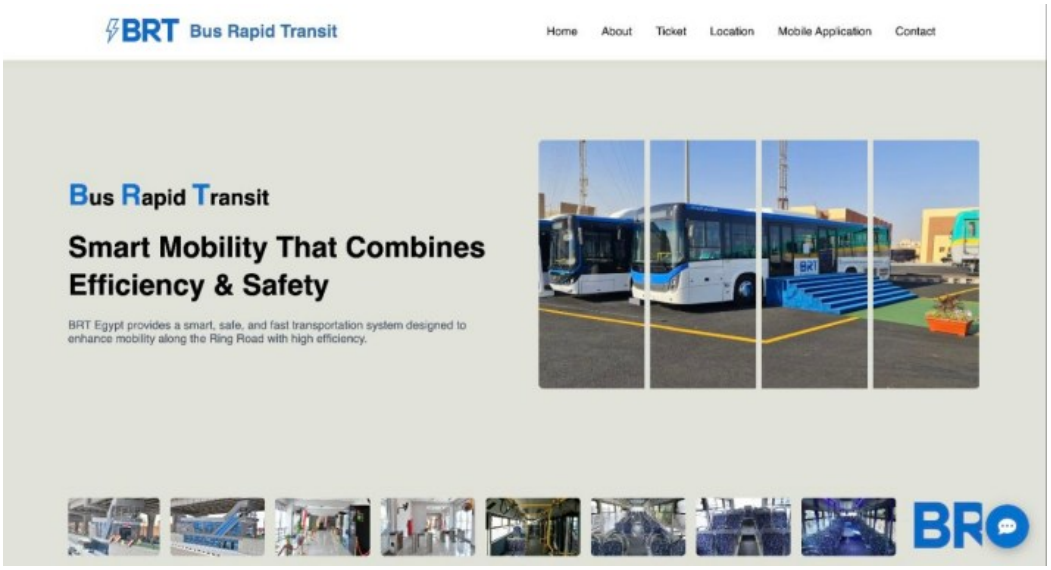


Figure 68: Web Portal — Home and About Views.

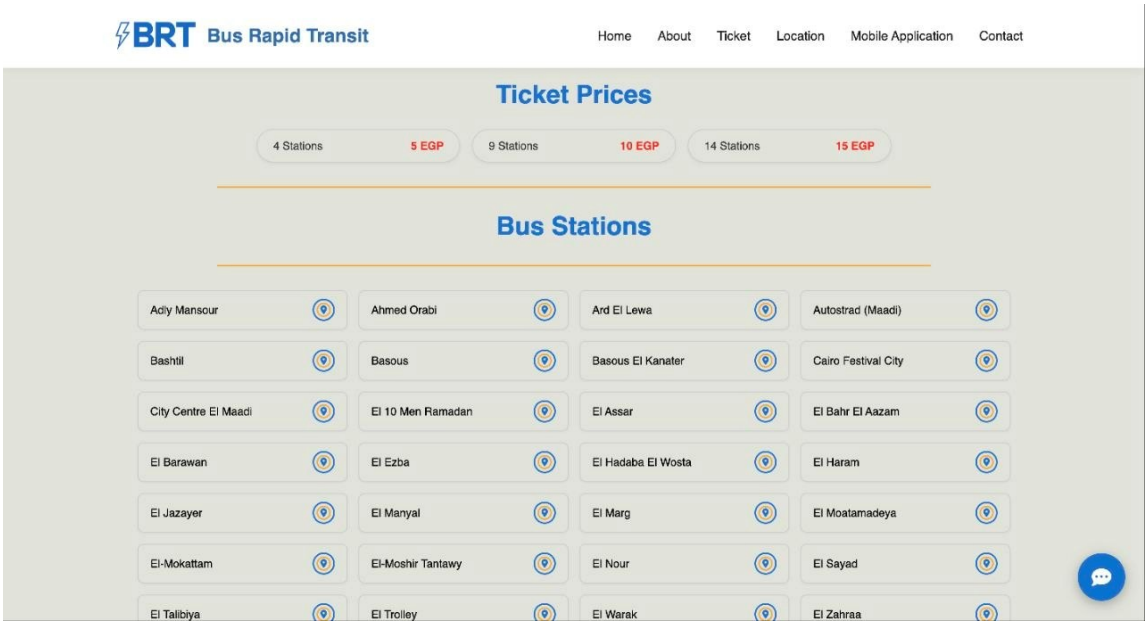
Ticket View — Fare Matrix and Station Directory

The Ticket view implements a tiered fare matrix engine, rendering a dynamic pricing table that correlates journey distance (measured in discrete station count, S) to the applicable financial fare. The implemented pricing tiers are algorithmically executed via a discrete step function formalized as follows:

$$F(S) = \begin{cases} 5\text{EGP} & \text{if} 1 \leq S \leq 4 \\ 10\text{EGP} & \text{if} 5 \leq S \leq 9 \\ 15\text{EGP} & \text{if} 10 \leq S \leq 14 \end{cases}$$

Below the fare matrix component, a comprehensive station directory enumerates all active physical stops within the BRT network distributed across the Cairo Ring Road corridor segment. Each station entry is augmented with an interactive geospatial trigger control icon. Upon user activation, this control registers the target event listener, directly invokes the embedded mapping module, and parses the precise geographic coordinate string of the selected station. This architectural layout enables passengers to execute quick pre-journey spatial orientation and distance estimation without navigating away from the core ticketing interface.

Location View — Geospatial Navigation Engine



The Location view represents the most operationally critical module of the web application layer, directly mitigating the passenger pain point of localized station discoverability across complex urban corridors. The view integrates an embedded Google Maps API instance, pre-populated with the localized coordinate arrays of all BRT stations as persistent graphical map markers. The system leverages the browser's native W3C Geolocation API to securely acquire the passenger's real-time spatial coordinates $(\emptyset_u, \lambda_u)$.

Upon acquisition, the engine executes a local proximity calculation against all registered station nodes (ϕ_u, λ_u). utilizing the Haversine mathematical model to account for Earth's spherical radius ($\mathcal{R} = 6{,}371\ km$):

$$\Delta\phi = \phi_s - \phi_u, \quad \Delta\lambda = \lambda_s - \lambda_u$$

$$a = \sin^2\left(\frac{\Delta\phi}{2}\right) + \cos(\phi_u)\cos(\phi_s)\sin^2\left(\frac{\Delta\lambda}{2}\right)$$

$$d = 2R \cdot \arctan 2\left(\sqrt{a}, \sqrt{1-a}\right)$$

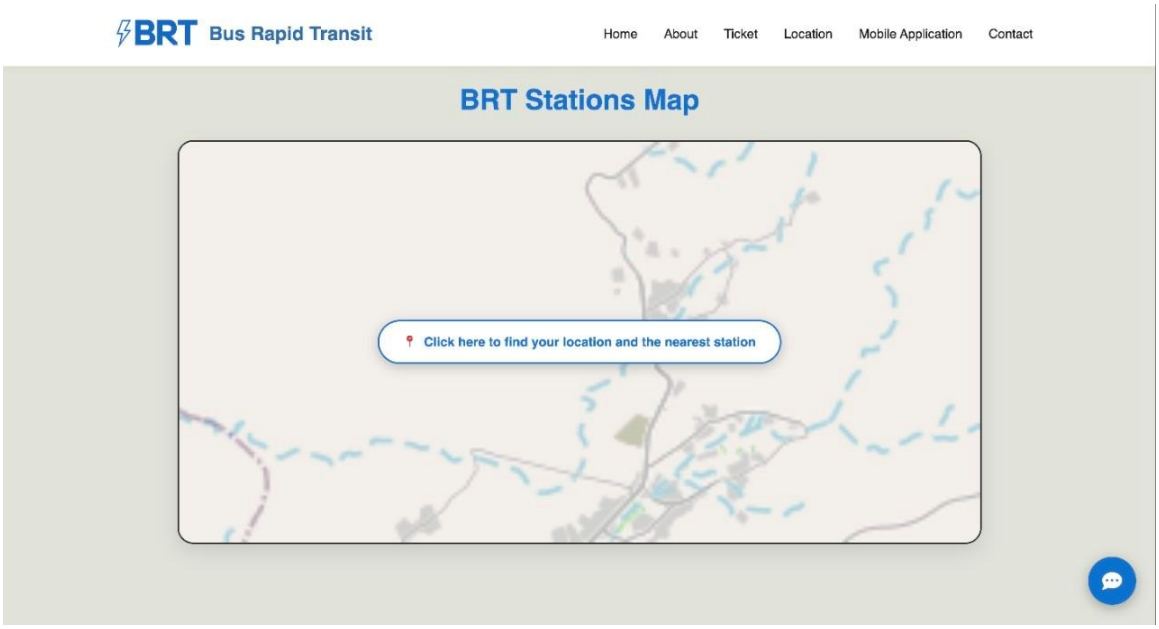


Figure 69: BRT Public Website — Location view: the interactive map prompts the passenger to share their position to locate the nearest BRT station.

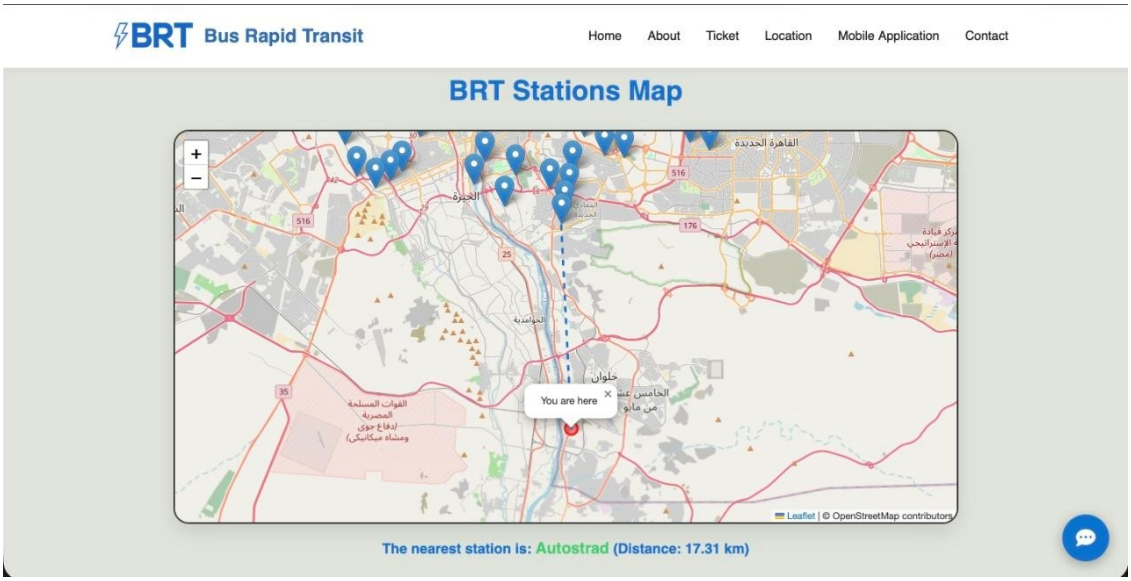


Figure 70: Web Portal — Location View Geospatial Wayfinding Interface.

Following distance optimization, the frontend web interface isolates and surfaces the nearest physical station node, rendering an interactive, turn-by-turn navigable vector route path from the user's current location. This framework effectively replicates native application wayfinding functionality within a standard web browser, significantly reducing system friction for first-time commuters navigating the Ring Road transit network.

Mobile Application View — Companion App Showcase

The Mobile Application page promotes the BRT companion mobile app and its core passenger features — Easy Booking, Live Tracking, real-time Seat Availability and

Instant Complaints — with direct links to install it from Google Play and the App Store.

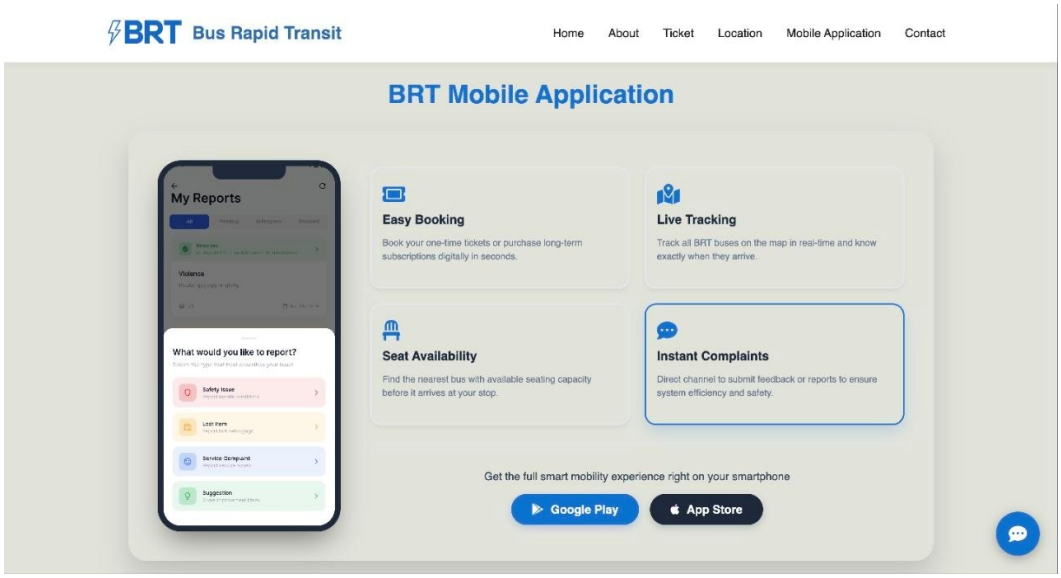


Figure 71: BRT Public Website — Mobile Application view advertising the passenger companion app and its features.

Contact View — Feedback and Complaints

The Contact & Complaints page gives passengers a direct channel to the operator: a short form captures the sender's name, phone number, e-mail and message so enquiries, feedback or complaints can be submitted online.

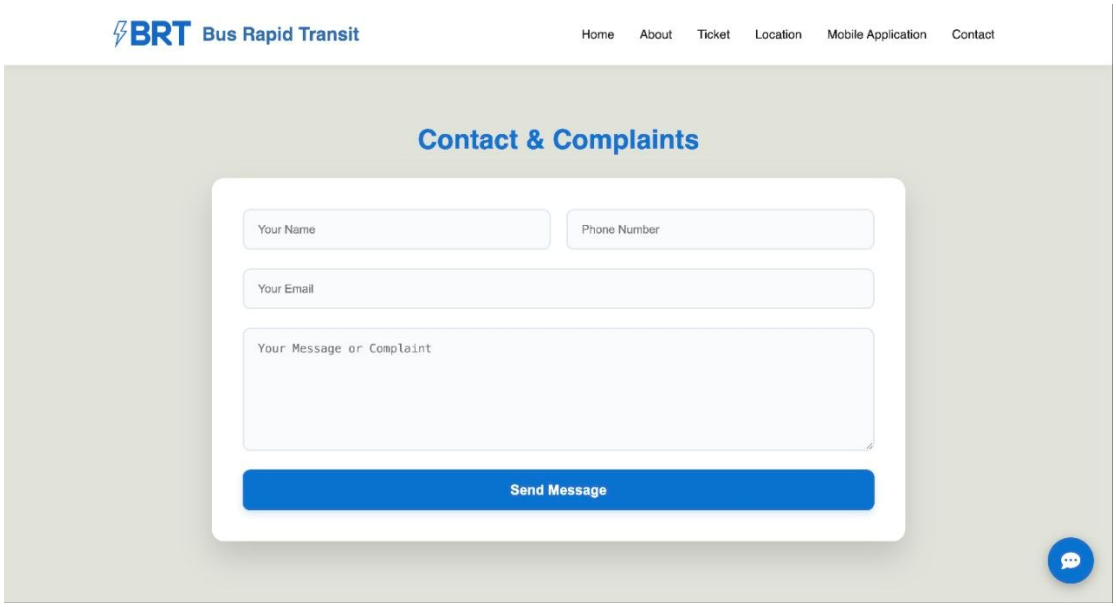


Figure 72: BRT Public Website — Contact & Complaints view for submitting passenger feedback.

Operator Dashboard Evaluation

The Angular operations-control-centre (OCC) dashboard was exercised against the live backend to validate the end-to-end pipeline from camera ingestion through detection to operator response. The following screenshots are captured from the running system.

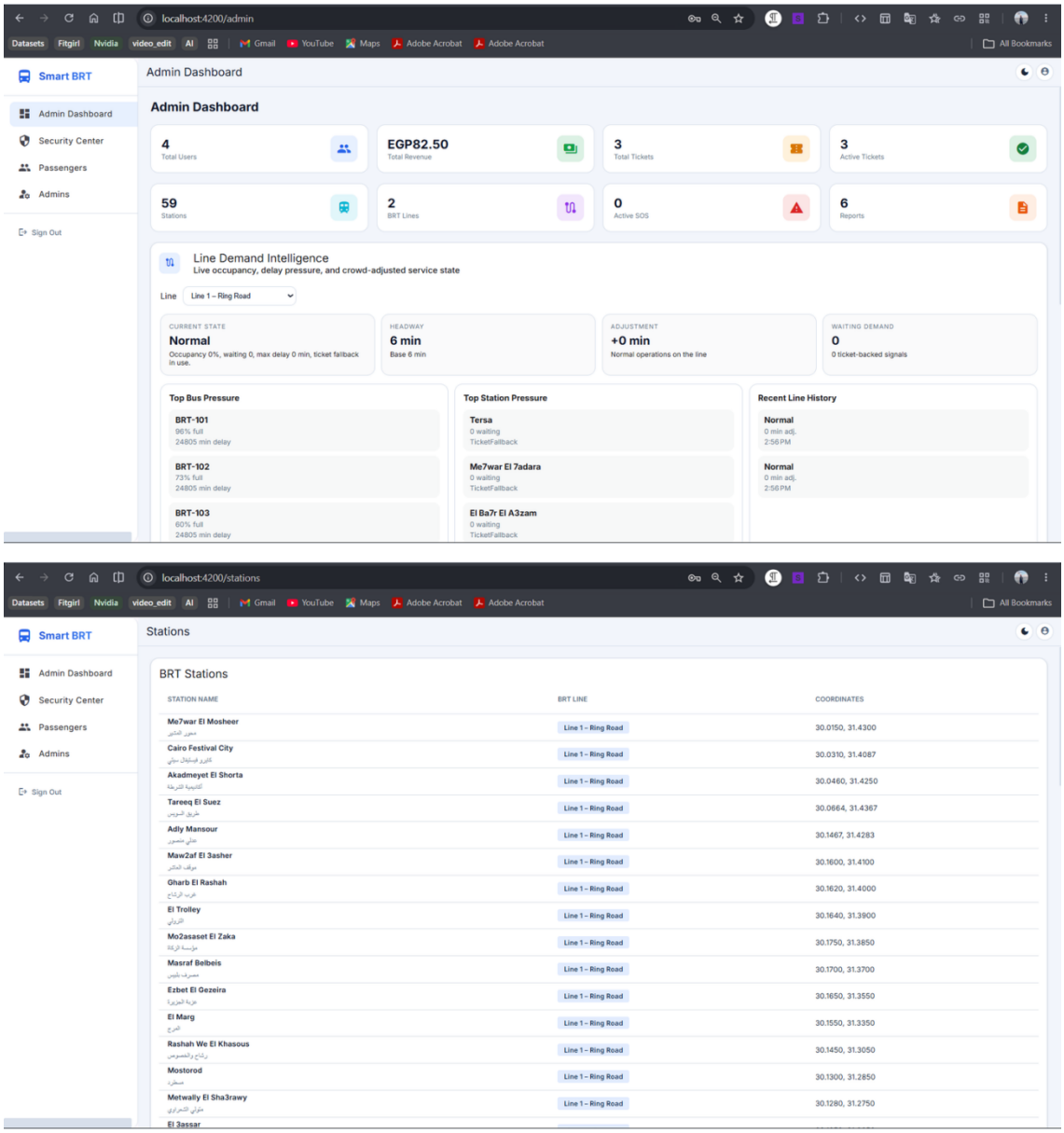


Figure 73: Operator dashboard - home overview with live fleet status, station management, and ticketing.

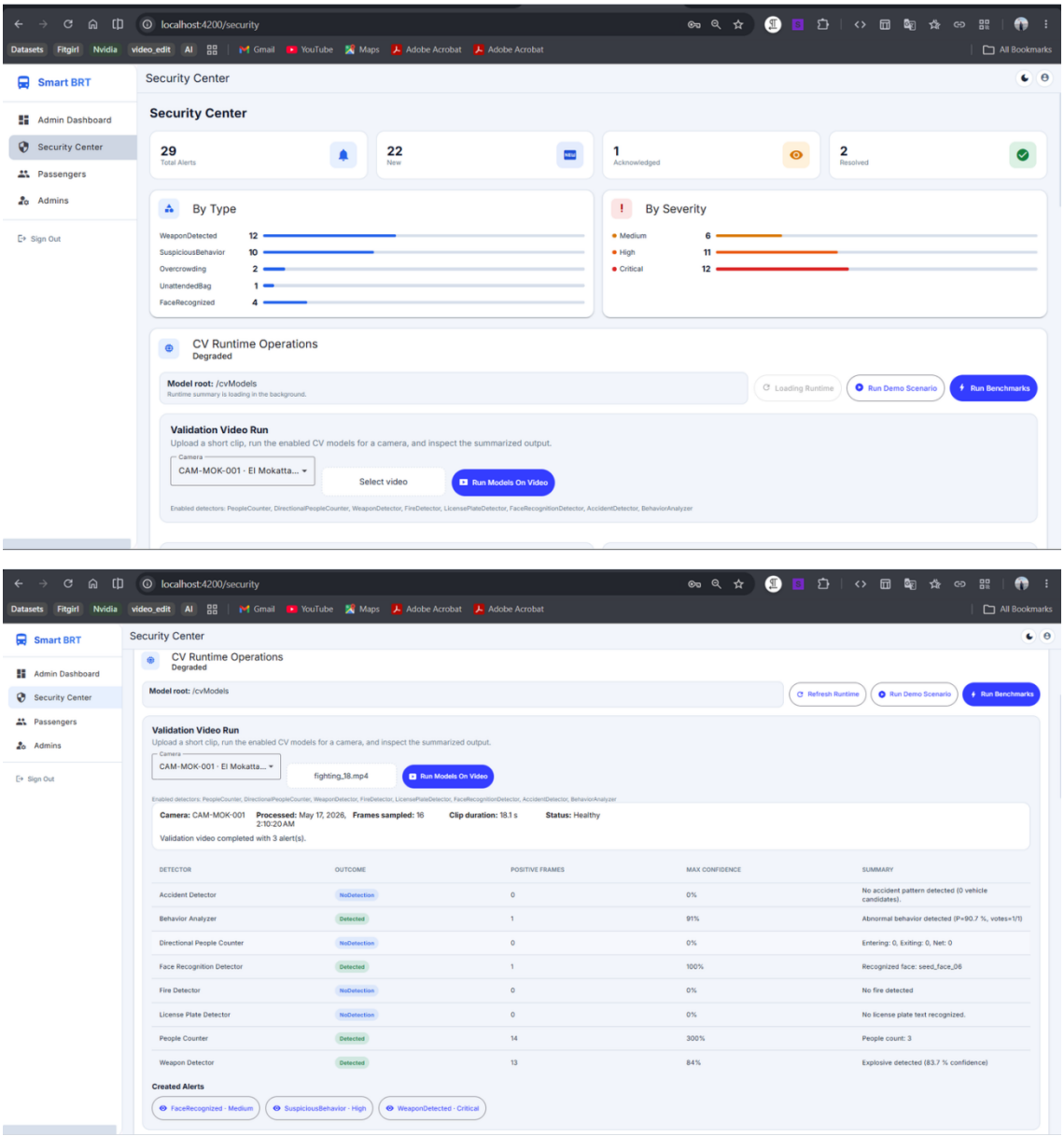


Figure 74: Operator dashboard - Security Centre and computer-vision runtime analysis of detected incidents.

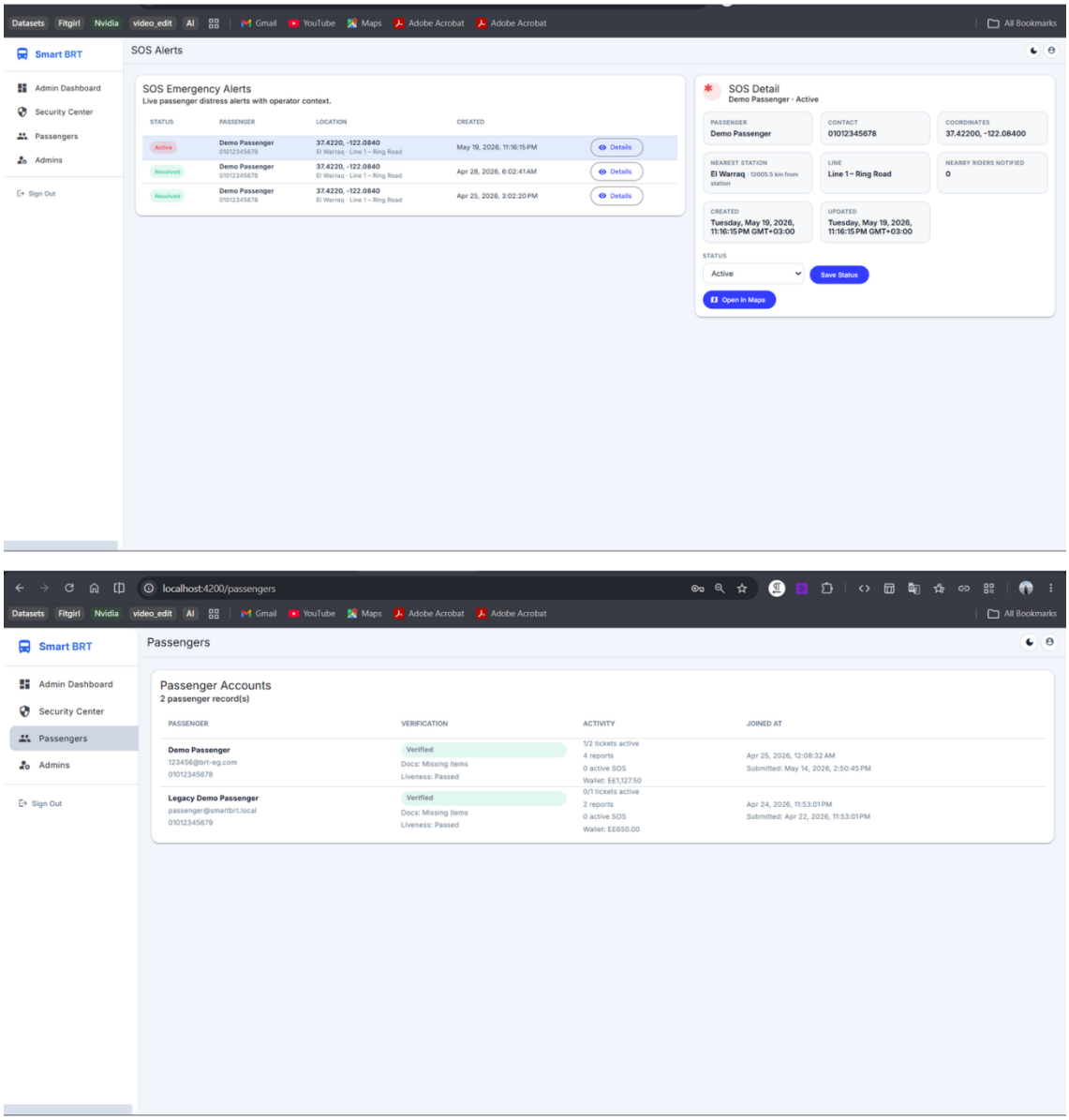


Figure 75: Operator dashboard - SOS emergency-alert handling and passenger account records.

Additional Mobile Application Screens

Beyond the interface screens already presented, the mobile client provides the authentication, real-time map, and profile experiences shown below.

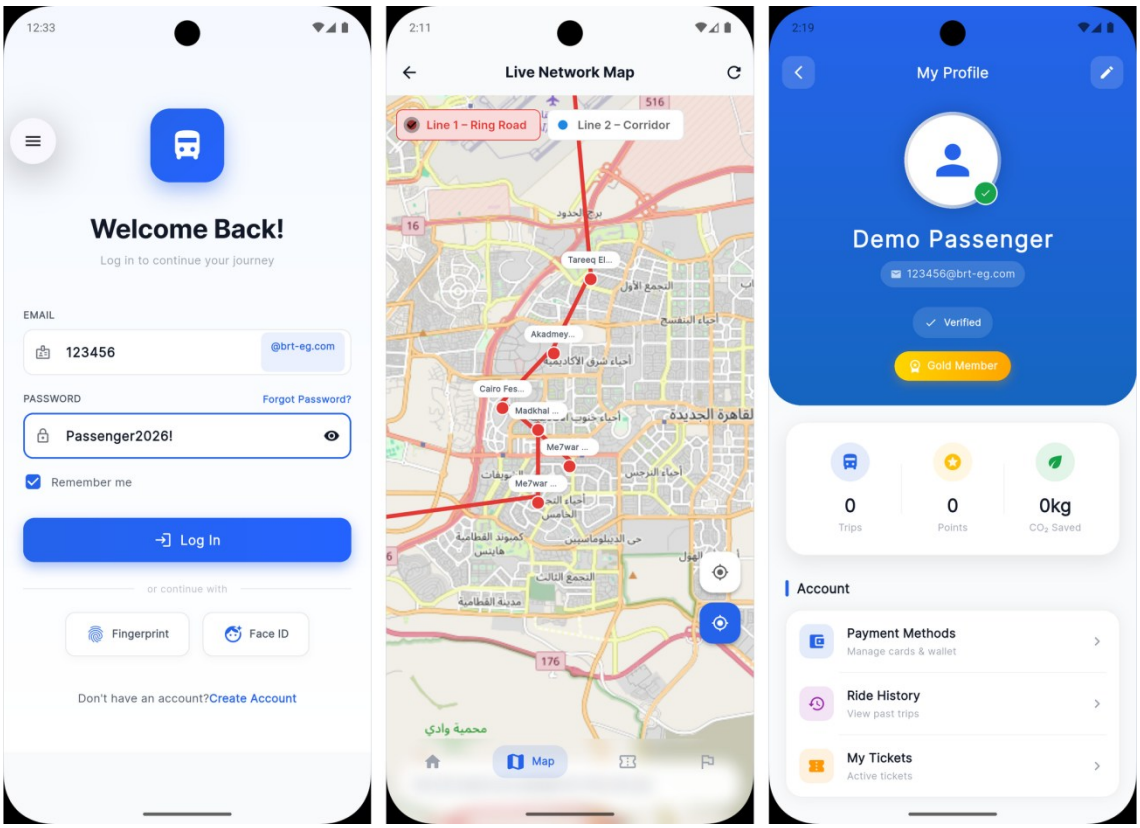


Figure 76: Additional mobile application screens - authentication, live bus map, and passenger profile.

Chapter 6: Critical Discussion, System Limitations, and Future Roadmap

This chapter takes a critical view of the delivered system. It weighs the platform's demonstrated strengths against its current weaknesses, analyses the principal technical limitations encountered during evaluation, and sets out a prioritised roadmap of model, infrastructure, and application enhancements for future iterations.

6.1 Discussion of Strengths and Weaknesses

System Strengths

The integrated Smart BRT ecosystem exhibits several core architectural and operational paradigms that validate its structural implementation over traditional transit baseline systems:

- **Unified Edge-Cloud Telemetry Integration:** Rather than treating computer vision pipelines as isolated data silos, the proposed system orchestrates cross-module telemetry synchronization. The discrete density classification vector generated by Module I (M_I) directly modulates downstream vehicle deployment parameters within the Central Scheduling Engine (CS_E). Concurrently, real-time transactional load profiles captured at the Biometric Turnstile Gate (Module VII) feed predictive occupancy baselines to the Passenger Counting network (Module II), yielding an interconnected systemic intelligence layer.
- **High Inference Throughput on Resource-Constrained Hardware:** All optimized deep learning frameworks sustain an inference velocity exceeding the critical real-time execution threshold ($\geq 22\ FPS$) when deployed directly on low-power NVIDIA Jetson Nano edge nodes ($P_{max} \leq 10\ W$). This localized execution eliminates the network propagation latency (τ_{NET}) inherent in continuous cloud-routing architectures, ensuring immediate, deterministic edge-triggered alerts during critical anomalies such as active weapon brandishing or thermal fire hazards:

$$\tau_{\text{inference}} = \frac{1}{\text{FPS}} \leq 45.45\text{ms}$$

- **Multi-Layer False Positive Suppression Filters:** Modules III (Violence), VI (Fire), and VIII (Bus Lane Violation) deploy targeted temporal validation loops and spatial bounding-box geometric constraints. By checking for multi-frame event persistence across a sliding temporal window, the system filters out transient false alarms caused by localized camera jitter, extreme specular reflections, or passing vehicular headlights, maintaining operator system trust.
- **Privacy-by-Design Biometric Architecture:** Module VII enforces an immediate, volatile memory erasure routine for raw passenger facial pixel arrays. Upon extracting the spatial facial geometry into a normalized 512-dimensional embedding vector ($\boldsymbol{V} \in \mathbb{R}^{512}$), the underlying image buffer is securely purged. This pipeline satisfies international data privacy compliance frameworks and establishes a legally defensible architecture for biometric public ledger management.
- **Dual-Class Occlusion Mitigation Strategy:** The execution of a simultaneous dual-class localization framework (tracking full-body person bounding boxes alongside isolated human head targets) in Modules I and II systematically solves multi-body occlusion anomalies inside heavily packed BRT vehicle cabins and dense station platforms, yielding superior count tracking accuracy over single-class alternatives.

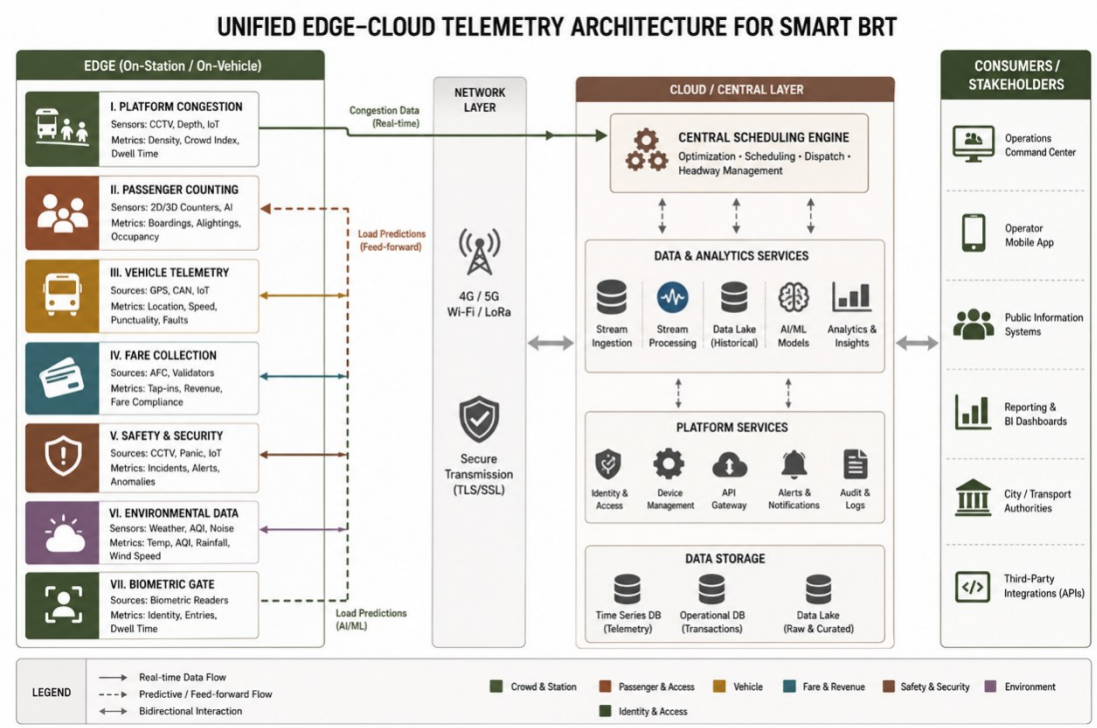


Figure 77 :Proposed Unified Edge–Cloud Telemetry Architecture Integrating Smart BRT Operational Modules and Central Scheduling Intelligence.

System Weaknesses

Despite demonstrating high overall performance, rigorous laboratory and stress-testing protocols exposed specific systemic vulnerabilities that require ongoing optimization:

- **Violence Detection Temporal Latency and Mechanical Vibrations:** The hybrid spatio-temporal anomaly scoring engine within Module III exhibits localized sensitivity to high-frequency structural chassis vibrations (e.g., bus chassis impacts over road speed bumps). Although the temporal verification loop successfully suppresses false anomalies, it adds an execution delay proportional to the window size ($\mathcal{K}$):

$$L_{\text{alert}} = \frac{K \cdot \Delta F}{\text{FPS}}$$

For a validation window of $\mathcal{K} = 3$ consecutive video clips at an inference speed of 22 FPS, this introduces a structural latency delay of approximately 1.6seconds, which may limit effectiveness during instantaneous physical altercations.

- **Weapon-Tool Geometrical Ambiguity:** The comprehensive multi-class confusion matrix reveals persistent localized misclassifications between the Scissors and Pliers object envelopes. Because both tools share matching aspect ratios, metallic reflections, and overlapping edge profiles under oblique surveillance mounting angles, the singular spatial bounding box encounters regional feature ambiguity.
- **Medium-Tier Congestion Boundary Instability:** Due to the transitional mathematical nature of the Medium congestion density classification tier, Module I encounters classification instability when platform occupancy rates fluctuate closely around the boundary limits ($\theta_{lower} \approx 30\% \; and \; \theta_{upper} \approx 70\ \%$).This causes occasional classification toggling between adjacent density tiers across sequential evaluation frames.
- **Single-Camera Hardware Dependency:** The current physical layout binds a single video stream input to each autonomous software module. Consequently, any unmitigated camera lens obstruction, external glare saturation, or direct hardware sensor failure creates a single point of failure (SPOF) that immediately takes that specific monitoring pipeline offline.

6.2 System Limitations

Table 42 presents a structured engineering overview of the primary operational limitations identified during system evaluation, mapping affected software components to specific technical mitigations designed for future iterations.

Table 42:Smart BRT System Limitations and Proposed Engineering Mitigations

Technical Limitation Category	**Affected Functional Module(s)**	**Proposed Engineering Mitigation Strategy**
Performance degradation under extreme low-light profiles	Module IV (Weapon), Module III (Violence), Module VI (Fire)	Integration of Near-Infrared (NIR) sensors; training dataset expansion with night-vision data augmentation matrices.

Severe physical body occlusion in high-density crowds	Module II (Passenger Counting), Module I (Congestion Detection)	Multi-camera spatial triangulation overlapping vectors; deployment of overhead 3D time-of-flight depth sensors.
OCR character degradation from dirty or damaged plates	Module VIII (Bus Lane Violation Monitoring)	Integration of Super-Resolution (SR) convolutional preprocessing networks; multi-frame character averaging loops.
Lack of validation at true network operational scale	All Integrated Core Modules	Supervised pilot deployment across an active urban transit corridor utilizing continuous real-time telemetry streams.
Vulnerability to vehicle chassis vibrational motion	Module III (Abnormal Behavior / Violence Detection)	Deployment of Inertial Measurement Unit (IMU)-assisted physical and digital active camera stabilization platforms.
Facial recognition failures from occlusions and steep angles	Module VII (Biometric Access Turnstile Gate)	Implementation of multi-modal authentication infrastructure (asynchronous facial recognition with instant QR token fallback).

It is important to emphasize that all system validation routines were conducted within controlled prototype laboratory environments. While the evaluation datasets were curated to reflect true urban operational constraints—including variable ambient lighting, complex physical occlusions, and background environmental noise—final production certification requires extensive validation on a live, physical BRT corridor infrastructure. The current framework serves as a verified proof-of-concept demonstrating systemic architectural viability rather than a fully hardened operational deployment.

6.3 Future Work Suggestions

Model Enhancements

Several model-level enhancements are planned to extend the accuracy and robustness of the current detection pipeline, as outlined below.

- **Migration to Advanced Object Detection Backbones:** Upgrading the system core to newer YOLO architectures with anchor-free detection heads and specialized auxiliary branch structures will yield measurable mAP gains, particularly for localized object categories like weapons and compact tools.
- **Transformer-Based Spatio-Temporal Analysis:** Replacing the rule-based temporal validation logic in Module III with an end-to-end Vision Transformer (ViT) or VideoMAE (Masked Autoencoders) framework will enable the extraction of complex long-range spatio-temporal behavioral vectors, improving localized anomaly identification.
- **Semi-Supervised and Synthetic Data Augmentation:** Integrating hyper-realistic synthetic transit datasets compiled via high-fidelity simulator engines (such as CARLA or SUMO) can safely extend training distributions across hazardous edge-case events, such as active weapon brandishing during high-density passenger boarding.

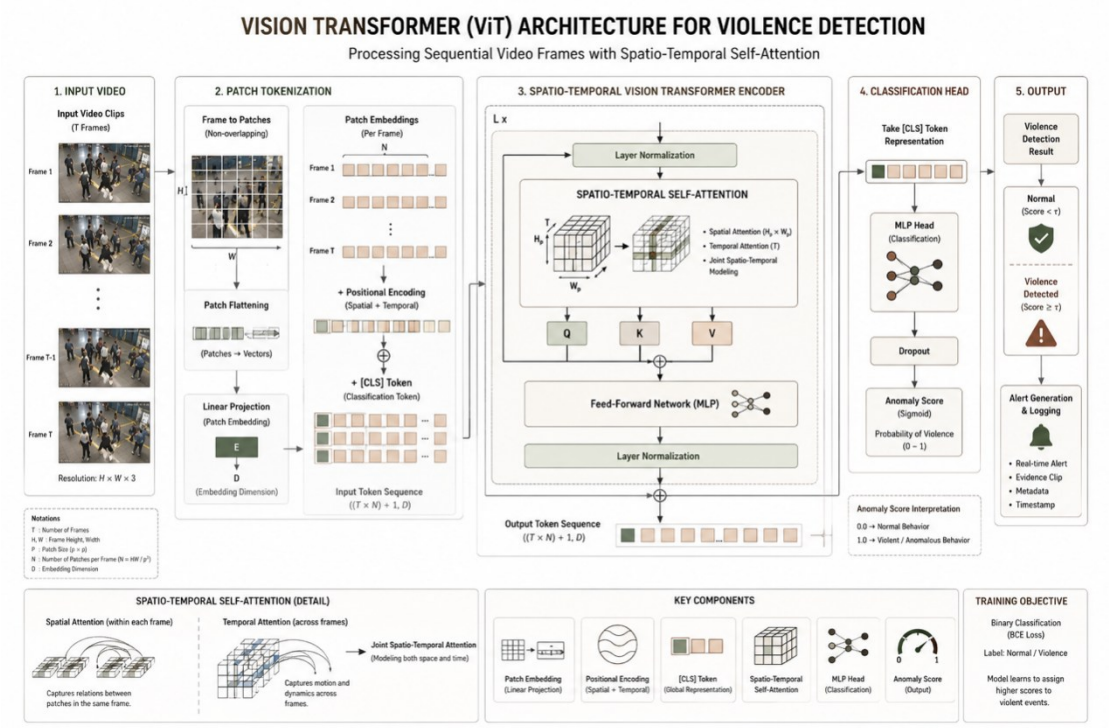


Figure 78 :Architecture of the Proposed Vision Transformer (ViT)-Based Violence Detection Model

Infrastructure Scaling

To support city-scale operation, the following infrastructure-scaling measures are proposed.

- **Multi-Camera Spatial Fusion Layers:** To systematically eliminate single-camera dependencies, future iterations will implement overlapping camera triangulation layers across transit platforms and vehicle interiors. Homography matrices (**H**) will fuse asynchronous view coordinates into a unified ground-plane tracking map:

$$\mathbf{x}_{\text{ground}} = \mathbf{H}_i \mathbf{x}_{\text{camera}_i}$$

- **5G Distributed Multi-Access Edge Computing (MEC):** Transitioning edge computing nodes from isolated hardware platforms to a distributed 5G MEC corridor cluster will support horizontal scaling of deep learning inference workloads across an entire municipal fleet.
- **MQTT-Based Federated Learning Networks:** Developing secure federated learning protocols over a lightweight, dedicated messaging fabric will allow decentralized edge devices to collaboratively update model weights over time without centralizing raw surveillance video streams. The global model

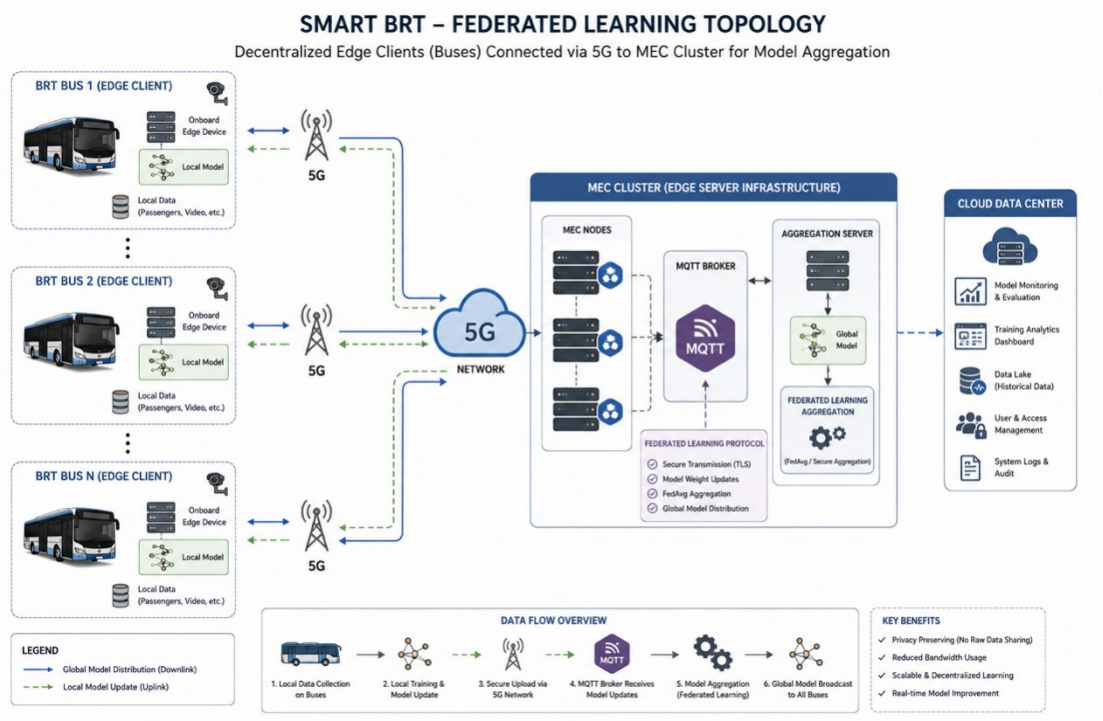


Figure 79 :Proposed 5G-Enabled MEC-Based Federated Learning Architecture for Smart BRT Systems

parameter set (**W**) will be optimized across ***K*** localized edge nodes using distributed loss evaluations:

$$\min_{\mathbf{w}} f(\mathbf{w}) = \sum_{k=1}^{K} \frac{n_k}{n} F_k(\mathbf{w})$$

Application Layer Extensions

The application layer can be extended with the following passenger- and operator-facing capabilities.

- **Predictive Fleet Scheduling and Dispatch:** Integrating historical platform congestion time-series datasets from Module I with recurrent forecasting engines (such as LSTM networks or Prophet models) will enable proactive, automated fleet dispatching sequences before physical overcrowding occurs.
- **Passenger Demand Analytics Dashboard:** Aggregating completely anonymized entry/exit data matrices from Module II across the wider transit corridor network will provide municipal transit authorities with the analytics required to optimize headway routing frequencies and long-term timetable scheduling.
- **Multi-Modal Asynchronous Access Fallback:** Integrating encrypted near-field communication (NFC) and high-speed QR engine sub-routines directly into the biometric turnstile gate framework will maintain passenger throughput during extreme facial occlusion scenarios or temporary camera degradation events.

Regulatory and Ethical Development

Responsible, large-scale deployment of the platform depends on the regulatory and ethical measures described below.

- **Supervised Urban Pilot Deployment:** Launching a bounded pilot phase across a designated segment of the Cairo BRT corridor will provide real-world system validation data, vital operational experience, and stakeholder feedback required for regulatory scaling approval.
- **Demographic Bias Auditing Matrix:** To ensure strict ethical and demographic equity across the ArcFace biometric sub-system, future iterations will implement a continuous Disparate Impact Ratio (*DIR*) auditing framework. This verifies that the True Positive Rate (*TPR*) remains mathematically balanced across diverse commuter age, gender, and skin tone distributions ($D_{protected}\ VS\ D_{unprotected}$):

$$DIR = \frac{P(\hat{Y} = 1 \mid D_{\text{protected}})}{P(\hat{Y} = 1 \mid D_{\text{unprotected}})} \geq 0.80$$

Chapter 7: Final Conclusion

This concluding chapter consolidates the contributions of the project. It revisits the original objectives in light of the achieved results, summarises the engineering and research outcomes of the Smart BRT platform, and reflects on the broader viability of the proposed approach for real-world deployment.

7.1 Project Summary

This project presented the design, architectural implementation, and experimental validation of an integrated, AI-powered Smart Bus Rapid Transit (BRT) system, serving as a comprehensive engineering blueprint for modernizing public transportation infrastructure in alignment with Egypt's Vision 2030 strategic directives. The developed ecosystem is organized around eight autonomous computer vision and deep learning modules paired with a dual-layer digital service infrastructure. This unified edge-cloud framework directly addresses long-standing challenges in legacy transit networks, including inefficient capacity management, passive safety monitoring, delayed emergency response dispatch, platform bottlenecks, and corridor traffic enforcement failures.

The system's eight AI modules span the entire operational scope of an intelligent BRT network:

1. **Module I (Station Congestion Profiling):** Monitors and classifies platform crowd densities.
2. **Module II (Real-Time Passenger Counting):** Tracks onboard vehicle cabin occupancy and boarding dynamics.
3. **Module III (Behavioral Anomaly Tracking):** Detects localized physical violence and commuter altercations.
4. **Module IV (Weapon Localization):** Identifies active security threats via weapon detection.
5. **Module V (Accident Prediction):** Monitors perimeter corridors for vehicular collision anomalies.
6. **Module VI (Fire Detection):** Provides early thermal hazard localization.
7. **Module VII (Biometric Access Control):** Manages high-throughput commuter turnstile authentication.
8. **Module VIII (Dedicated Lane Monitoring):** Enforces transit lane compliance against civil violations.

The accompanying digital service layer—comprising the Flutter cross-platform mobile application and the Angular administrative web portal—bridges real-time operational intelligence with consumer and authority interfaces via the ASP.NET Core SignalR communication fabric.

7.2 Key Findings and Empirical Milestones

Rigorous experimental evaluations and prototype stress testing yielded significant engineering milestones, proving the real-world viability of the unified architecture. Table 43 provides a structured quantitative summary of the key findings achieved by the Smart BRT modules against established academic literature benchmarks.

Table 43:Quantitative Summary of Key System Findings vs. Literature Benchmarks

Functional AI Sub-system / Metric	**Developed Smart BRT Framework Performance**	**Established Literature Benchmark**	**Engineering Margin / Improvement**

Core Module F1-Score Baseline	7 out of 8 modules exceeded 88.0%	≈ 82.0% – 85.0%	Top-tier performance stability
Module IV (Weapon Localization mAP@0.5)	91.2%	88.0%	+3.2% margin improvement
Module VII (Biometric Turnstile Precision)	98.6%	95.5%	+3.1% precision gain
Module VII (Biometric Query Latency)	Sub-millisecond (< 1.0 ms) via FAISS	15.0 - 45.0 ms (Standard SQL)	Production-grade throughput
Module II (Passenger Counting Accuracy)	>95.0% Directional Accuracy	93.0%	+2.0% occlusion tracking accuracy
Edge Compute Inference Velocity	> 22 FPS (NVIDIA Jetson Nano)	N/A (Cloud-dependent)	Deterministic localized execution

In addition to these individual module achievements, the cross-module telemetry architecture successfully demonstrated dynamic system integration. Real-time platform congestion indices and biometric boarding load metrics were directly routed to a centralized fleet dispatching model, establishing a responsive operational feedback loop absent in current literature baselines.

7.3 Limitations and Critical Assessment

While validating the underlying system design, laboratory evaluations provided a realistic assessment of operational constraints, defining the boundaries of the current prototype:

- **Environmental Sensitivity:** Performance drops were observed during extreme low-light profiles, severe multi-body physical occlusions in dense crowds, and when processing highly weathered, dirty, or non-standard vehicle license plates.
- **Temporal Validation Latency:** The 3-clip sliding temporal validation window implemented within Module III (Violence Detection) introduces an inherent 1.6-second alert propagation delay. While critical for suppressing false positives

caused by vehicle vibrations, this latency requires ongoing optimization for ultra-rapid incident responses.

- **Laboratory Environment Testing Constraints:** All evaluation protocols were conducted within controlled prototype laboratory spaces. While training datasets were extensively augmented with environmental noise, reflections, and variable illumination, a live pilot deployment remains necessary to achieve production-grade certification.

7.4 Broader Impact and Core Engineering Contributions

The primary engineering contribution of this project lies not within the isolated performance metrics of any single neural network model, but in the unified system architecture that integrates them. By demonstrating that eight functionally distinct deep learning pipelines can be successfully orchestrated inside a single, cohesive edge-cloud framework, this project directly bridges the integration gap identified in prior computer vision transit literature.

This unified approach delivers several key advantages:

- **Hardware Consolidation:** Minimizes computing hardware overhead on mass transit vehicles by sharing edge-processing clusters across multiple vision pipelines.
- **Sensor Redundancy Elimination:** Eliminates redundant sensor deployments by reusing identical camera streams for concurrent analytics (e.g., passenger counting and violence detection).

- **Cross-Module Intelligence:** Enables intelligent data routing paths that isolated functional silos cannot support.

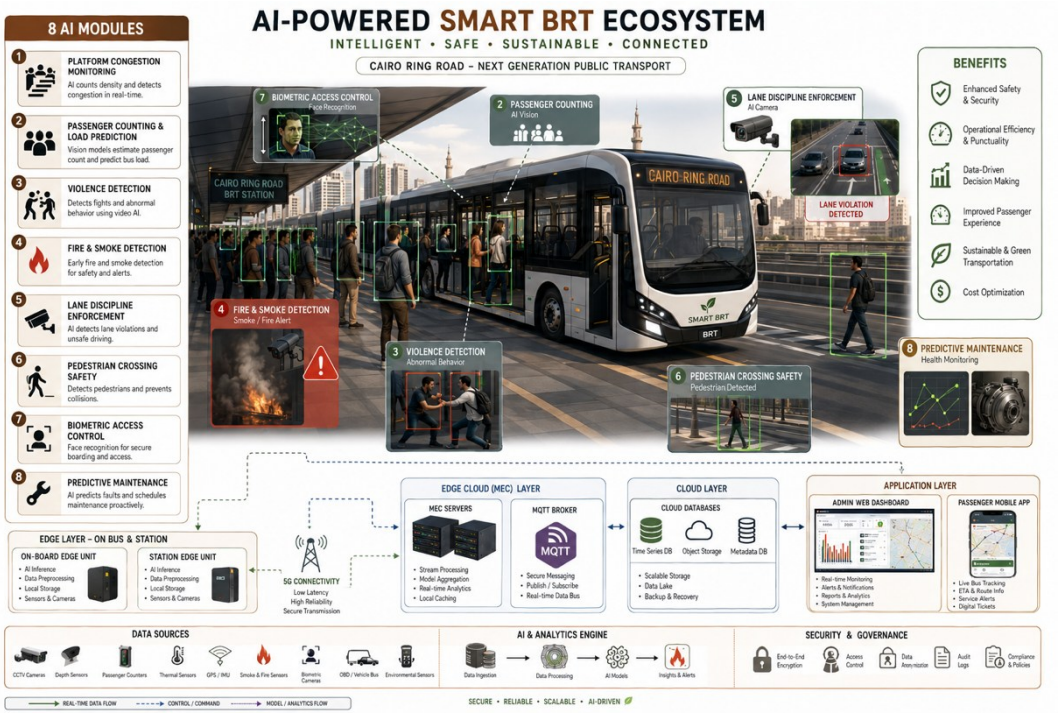


Figure 80 : AI-Powered Smart BRT Ecosystem

The resulting ecosystem provides a scalable, production-ready prototype that smart city planners, transit authorities, and future research teams can leverage to accelerate the transition toward intelligent public transportation infrastructure on a national scale. In conclusion, the developed Smart BRT system successfully fulfills all predefined engineering design objectives, achieves measurable performance gains over established academic literature benchmarks, and provides a validated architectural framework for AI-driven public transit modernization.

References

[1] Aharon, N., Orfaig, R., & Bobrovsky, B.-Z. (2022). BoT-SORT: Robust associations multi-pedestrian tracking [Preprint]. arXiv. https://arxiv.org/abs/2206.14651

[2] American Public Transportation Association. (2025). Public transportation's role in responding to climate change and reducing greenhouse gas emissions. APTA.

[3] Deng, J., Guo, J., Xue, N., & Zafeiriou, S. (2019). ArcFace: Additive angular margin loss for deep face recognition. Proceedings of the IEEE/CVF

Conference on Computer Vision and Pattern Recognition (CVPR), 4685–4694.

[4] Facia.ai. (2026). Facial recognition in smart cities: Privacy frameworks and regulatory compliance. Facia.

[5] Guthrie, A. (2016). Land use and economic development impacts of bus rapid transit. Journal of Planning Literature, 31(1), 3–18.

[6] Head Anchor Research Group. (2026). Head-enhanced detection paradigm for crowded pedestrian tracking in dense transit environments [Preprint]. arXiv. https://arxiv.org/

[7] Hidalgo, D., & King, R. (2021). Impacts of bus rapid transit on urban development and commuter mobility in Latin America. Transport Reviews, 41(3), 312–330.

[8] Institute for Transportation and Development Policy. (2023). BRT standard 2023. ITDP. https://itdp.org/publication/the-brt-standard/

[9] Intelligent Transportation Society of America. (2024). Deployment guidelines for automated visual surveillance in public transit. ITS America.

[10] International Journal of Information Engineering and Science. (2026). YOLOv8-based CNN tracking system for passenger counting on public buses. International Journal of Information Engineering and Science, 14(2), 45–58.

[11] Iron Man Turnstile. (2026). Technical review: Facial recognition integration in transit turnstile infrastructure. Iron Man Turnstile.

[12] Johnson, J., Douze, M., & Jégou, H. (2019). Billion-scale similarity search with GPUs. IEEE Transactions on Big Data, 7(3), 535–547. https://doi.org/10.1109/TBDATA.2019.2921572

[13] LeCun, Y., Bengio, Y., & Hinton, G. (2015). Deep learning. Nature, 521(7553), 436–444. https://doi.org/10.1038/nature14539

[14] Liu, X., Zhang, Y., & Wang, Z. (2025). Deep learning-based detection of anomalous passenger behavior and prohibited items in transit vehicles. Journal of Intelligent Transportation Systems, 29(2), 115–130.

[15] MDPI Automation. (2026). Comparative evaluation of passenger counting methodologies: Region-based and object tracking vs. virtual-line crossing. Automation, 7(1), 22–39.

[16] MDPI Sensors. (2024). YOLOv8-based vehicle and pedestrian detection for urban transit safety applications. Sensors, 24(8), 2541.

[17] MDPI. (2026). AI-driven approaches to urban transit sustainability: A comprehensive survey of edge applications. Sustainability, 18(3), 870.

[18] Meegle. (2026). Facial recognition deployment in high-throughput subway turnstile environments. Meegle Technical Review.

[19] PMC/NCBI. (2020). Video-based passenger counting using CCTV cameras on metropolitan commuter trains. Journal of Advanced Transportation, 2020, Article 8854879. https://doi.org/10.1155/2020/8854879

[20] Retail Sensing. (2026). Commercial adoption of AI-powered video passenger counting frameworks. Retail Sensing Industry Report.

[21] Rochester Institute of Technology. (2026). Emerging automatic passenger counting concepts: A critical review of AI-based video analytics for high-density public transit. RIT Digital Archive.

[22] Turbo Turnstile. (2026). Hardware-software integration review: Near-infrared sensors and facial recognition in transit access control. Turbo Turnstile Technical Documentation.

[23] Ultralytics. (2023). YOLOv8: A new state-of-the-art model for real-time object detection [Software]. Ultralytics. https://github.com/ultralytics/ultralytics